

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

Intelligent System Development · Assignment 03

Deep learning viết tay với NumPy và ba mô hình ML cho mỗi category

Mã Giảng
sinh viên
viên: hướng
B23DCVT10 dẫn:
· Dinh
Lớp: Que
Nguyễn B23VT01 Tran,
· Ph.D.,
Đại Nhóm: Assoc.
Dũng 02 Prof.

Học kỳ I.2026

Tóm tắt

Báo cáo dựng lại một mạng nơ-ron nhiều lớp chỉ bằng NumPy: lan truyền xuôi, binary cross-entropy, lan truyền ngược theo quy tắc chuỗi và gradient descent đều viết tay, không dùng TensorFlow, PyTorch, Keras hay scikit-learn. Bốn kiến trúc được huấn luyện trên cùng một tập dữ liệu, cùng một seed khởi tạo và cùng một quy trình đo, nên khác biệt giữa chúng chỉ đến từ kiến trúc. Mô hình tốt nhất theo ROC-AUC là M2 Wider với 0.8043.

Bốn chương sau mở rộng phạm vi ra cả ba category của dự án, để mỗi category có đủ ba mô hình machine learning và một mô hình deep learning. Hai mô hình ML được bổ sung là KNN và AdaBoost, lớp hai họ thuật toán chưa từng xuất hiện trong dự án. Hai mạng nơ-ron được bổ sung, cũng viết tay bằng NumPy, phù bài toán hồi quy giá nhà và bài toán văn bản e-commerce, trong đó ma trận nhúng từ được huấn luyện bằng lan truyền ngược thay vì để ngẫu nhiên cố định.

Mục lục

1. Bài toán và phạm vi

Deep Learning là phép hợp thành của các hàm có tham số. Thay vì học một ánh xạ trực tiếp từ đầu vào sang nhãn, mô hình dựng nhiều phép biến đổi nối tiếp rồi học tham số của chúng.

$$\hat{y} = f_3(f_2(f_1(X))) = \sigma(\text{ReLU}(\text{ReLU}(XW_1 + b_1)W_2 + b_2)W_3 + b_3)$$

Bài toán là phân loại nhị phân nguy cơ tiểu đường từ bộ khảo sát BRFSS 2015. Đầu vào là tám chỉ số sức khỏe, đầu ra là một xác suất. Ràng buộc của bài là không dùng framework học sâu nào: mọi phép toán của mạng, kể cả đạo hàm, đều viết bằng NumPy.

01 Biểu diễn <i>8 đặc trưng, chuẩn hoá z</i>	02 Mạng NumPy <i>forward, loss, backward</i>	03 Huấn luyện <i>gradient descent lô nhỏ</i>	04 Đánh giá <i>4 mô hình, 3 ngưỡng</i>
---	---	---	---

Bốn kiến trúc được so sánh

Mô hình	Kiến trúc	Kích hoạt lớp ẩn	Tham số	Mục đích
M1 Baseline	8 - 16 - 8 - 1	ReLU	289	Kiến trúc gốc của bài giảng, dùng làm mốc so sánh cho ba mô hình còn lại.
M2 Wider	8 - 32 - 16 - 1	ReLU	833	Nhân đôi bề rộng cả hai lớp ẩn, số tham số tăng gần ba lần.
M3 Shallow	8 - 16 - 1	ReLU	161	Bỏ hẳn lớp ẩn thứ hai, chỉ còn một tầng biểu diễn.
M4 No ReLU	8 - 16 - 8 - 1	tuyến tính	289	Giữ nguyên ba lớp nhưng thay ReLU bằng hàm đồng nhất.

Cả bốn dùng chung seed khởi tạo 42 và seed xáo trộn 7, cùng học suất 0.05, cùng 60 epoch và cùng lô 512 dòng.

Khoảng cách giữa mô hình tốt nhất và kém nhất theo ROC-AUC chỉ là 0.0077. Đây là kết quả đáng chú ý nhất của báo cáo và sẽ được giải thích ở mục 8: với tám đặc trưng bảng đã chuẩn hoá, phần lớn tín hiệu là tuyến tính, nên chiều sâu và bề rộng mang lại rất ít.

2. Biểu diễn dữ liệu

Bài giảng dùng bộ Pima với đúng tám biến đầu vào. Bộ BRFSS ở đây có 21 biến, nên phải chọn ra tám. Tiêu chí chọn là trị tuyệt đối hệ số tương quan với nhãn nhị phân, xếp hạng đầy đủ 21 biến rồi lấy tám biến đầu. Cách chọn này đơn giản và kiểm chứng được, nhưng chỉ bắt được quan hệ tuyến tính đơn biến.

Tám đặc trưng được chọn

Hạng	Đặc trưng	tương quan
1	GenHlth	0.2823
2	HighBP	0.2603
3	BMI	0.2112
4	DiffWalk	0.2081
5	HighChol	0.2044
6	Age	0.1855
7	HeartDiseaseorAttack	0.1672
8	PhysHlth	0.1587

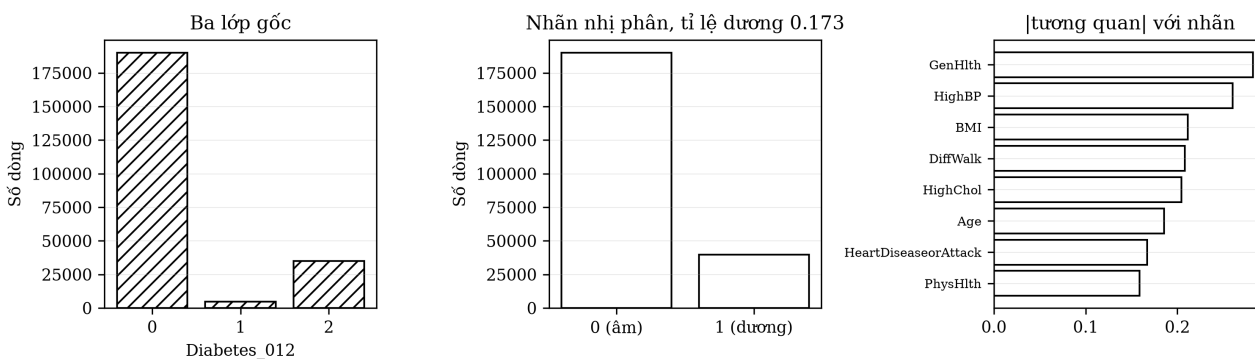
Bốn biến bị loại ngay sau ngưỡng: *Income* (0.1478), *Education* (0.1085), *PhysActivity* (0.1024), *Stroke* (0.0977).

Nhân nhị phân giữ đúng quy ước của các ứng dụng trước, y bằng 1 khi *Diabetes_012* > 0, tức gộp tiền tiểu đường và tiểu đường vào lớp dương. Sau khi bỏ 23,899 dòng trùng hoàn toàn, còn 229,781 dòng, chia 183,824 train và 45,957 test theo tỉ lệ 80 trên 20 với seed 42.

Tỉ lệ lớp dương là 0.1729 trên train và 0.1729 trên test. Đây là bài toán lệch lớp, nên accuracy một mình không nói lên điều gì: đoán toàn lớp âm đã cho accuracy khoảng 0.827.

$$x' = (x - \mu_{train}) / \sigma_{train}$$

Chuẩn hoá dùng trung bình và độ lệch chuẩn tính riêng trên tập train rồi áp lại cho test. Nếu tính trên toàn bộ dữ liệu thì thông tin của tập test đã rò sang lúc huấn luyện, và mọi con số đánh giá sau đó đều lạc quan hơn thực tế.



Phân bố ba lớp gốc, nhân nhị phân dùng để huấn luyện, và thứ hạng tương quan của tám đặc trưng được chọn.

Hình f1Observation: lớp tiền tiểu đường rất mỏng so với hai lớp còn lại, và sau khi gộp nhị phân thì lớp dương vẫn chỉ chiếm khoảng một phần sáu. **Interpretation:** tín hiệu dương hiếm, mạng sẽ có xu hướng nghiêng về lớp âm để hạ nhanh hàm mất mát. **ML implication:** phải đọc recall và ROC-AUC thay vì accuracy, và ngưỡng quyết định 0.5 không nhất thiết là ngưỡng nên dùng khi triển khai.

3. Mạng nơ-ron viết tay

Toàn bộ mạng gồm bốn phép toán, không có phép nào cần thư viện học sâu.

3.1 Lan truyền xuôi

$$Z_k = H_{k-1}W_k + b_k \quad H_k = g(Z_k)$$

Hàm g là ReLU ở lớp ẩn và sigmoid ở lớp ra. ReLU đưa phi tuyến vào mạng: thiếu nó, mọi phép xếp chồng tuyến tính vẫn quy về một phép tuyến tính duy nhất. Sigmoid ép đầu ra về khoảng mở $(0, 1)$ để đọc như một xác suất.

3.2 Hàm mất mát

$$L = -(1/N) \sum [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

Xác suất được cắt về khoảng $[1e-8, 1 - 1e-8]$ trước khi lấy log, nếu không một dự đoán tự tin tuyệt đối và sai sẽ cho $\log(0)$ và làm hỏng cả vòng huấn luyện.

3.3 Lan truyền ngược

Ghép sigmoid với binary cross-entropy cho một kết quả rất gọn ở lớp cuối. Đạo hàm của hàm mất mát theo đầu vào của sigmoid không còn chứa đạo hàm sigmoid nữa:

$$\partial L / \partial Z_L = (\hat{y} - y) / N$$

Từ đó gradient lan ngược bằng quy tắc chuỗi, mỗi lớp chỉ cần hai phép nhân ma trận và một phép nhân từng phần tử với đạo hàm của hàm kích hoạt:

$$dW_k = H_{k-1}^T \delta_k \quad \delta_{k-1} = (\delta_k W_k^T) \odot g'(Z_{k-1})$$

Lan truyền ngược không phải một phép thuật riêng của học sâu. Nó chỉ là quy tắc chuỗi áp cho một hàm hợp thành, thực hiện theo thứ tự ngược để dùng lại kết quả trung gian mà lan truyền xuôi đã lưu.

3.4 Cập nhật tham số

$$W \leftarrow W - \eta \partial L / \partial W$$

Hàm cập nhật trả về bộ tham số mới thay vì sửa tại chỗ, nên mỗi bước huấn luyện là một phép biến đổi rõ ràng từ trạng thái cũ sang trạng thái mới. Cách viết này tốn thêm một lần cấp phát mỗi bước nhưng loại hẳn một lớp lỗi khó tìm khi gỡ rối gradient.

4. Mô hình M1 Baseline

Kiến trúc gốc của bài giảng, dùng làm mốc so sánh cho ba mô hình còn lại. Hai lớp ẩn ReLU tạo ra hai tầng biểu diễn trung gian trước khi sigmoid quy về xác suất.

0.8378	0.2926	0.8042	8s
ACCURACY · TEST	F1 · NGUỖNG 0.5	ROC-AUC · TEST	THỜI GIAN HUẤN LUYỆN

Cấu hình và quá trình huấn luyện của M1

Hạng mục	Giá trị
Kiến trúc	8 - 16 - 8 - 1
Kích hoạt lớp ẩn	ReLU
Số tham số học được	289
Kích thước biểu diễn trung gian	H1: (183,824, 16), H2: (183,824, 8)
Học suất, epoch, lô	0.05 · 60 · 512
Số lần cập nhật tham số	21,600
Loss train đầu và cuối	0.3854 → 0.3704
Loss trên tập test	0.3702
Thời gian huấn luyện	7.9 giây

Chênh lệch accuracy giữa train và test là -0.0018, và loss test (0.3702) so với loss train (0.3704) lệch -0.0002. Khoảng cách hẹp như vậy cho thấy mô hình chưa quá khớp, phần thiếu nằm ở sức biểu diễn chứ không ở việc học thuộc dữ liệu.

Kết quả của M1 trên tập test theo ba ngưỡng quyết định

Ngưỡng	Accuracy	Precision	Recall	F1	TP	FN	FP	TN
0.3	0.8007	0.4364	0.5252	0.4767	4,172	3,772	5,389	32,624
0.5	0.8378	0.5940	0.1941	0.2926	1,542	6,402	1,054	36,959
0.7	0.8286	0.7161	0.0140	0.0274	111	7,833	44	37,969

Hạ ngưỡng từ 0.5 xuống 0.3 nâng recall từ 0.1941 lên 0.5252 và đổi lại precision giảm từ 0.5940 xuống 0.4364, tức bắt thêm 2,630 ca dương thật nhưng báo động nhầm thêm 4,335 ca. Nâng lên 0.7 đẩy recall xuống 0.0140. Với bài sàng lọc bệnh, bỏ sót tổn kém hơn báo động nhầm, nên ngưỡng thấp là lựa chọn hợp lý hơn 0.5.

Đây là mốc tham chiếu. Hai lớp ẩn ReLU dựng được hai tầng biểu diễn, nhưng recall tại ngưỡng 0.5 chỉ ở mức 0.1941: mạng hạ hàm mất mát nhanh nhất bằng cách nghiêng về lớp âm chiếm đa số, đúng như dự đoán từ phân bố lớp.

5. Mô hình M2 Wider

Nhân đôi bề rộng cả hai lớp ẩn, số tham số tăng gần ba lần. Câu hỏi cần trả lời là thêm sức chứa có đổi lấy được chất lượng hay không.

0.8372

ACCURACY · TEST

0.2814

F1 · NGUỖNG 0.5

0.8043

ROC-AUC · TEST

14s

THỜI GIAN HUẤN LUYỆN

Cấu hình và quá trình huấn luyện của M2

Hạng mục	Giá trị
Kiến trúc	8 - 32 - 16 - 1
Kích hoạt lớp ẩn	ReLU
Số tham số học được	833
Kích thước biểu diễn trung gian	H1: (183,824, 32), H2: (183,824, 16)
Học suất, epoch, lô	0.05 · 60 · 512
Số lần cập nhật tham số	21,600
Loss train đầu và cuối	0.3810 → 0.3699
Loss trên tập test	0.3700
Thời gian huấn luyện	13.7 giây

Chênh lệch accuracy giữa train và test là -0.0008, và loss test (0.3700) so với loss train (0.3699) lệch +0.0002. Khoảng cách hẹp như vậy cho thấy mô hình chưa quá khớp, phần thiếu nằm ở sức biểu diễn chứ không ở việc học thuộc dữ liệu.

Kết quả của M2 trên tập test theo ba ngưỡng quyết định

Ngưỡng	Accuracy	Precision	Recall	F1	TP	FN	FP	TN
0.3	0.8000	0.4349	0.5237	0.4752	4,160	3,784	5,406	32,607
0.5	0.8372	0.5934	0.1844	0.2814	1,465	6,479	1,004	37,009
0.7	0.8289	0.7314	0.0161	0.0315	128	7,816	47	37,966

Hạ ngưỡng từ 0.5 xuống 0.3 nâng recall từ 0.1844 lên 0.5237 và đổi lại precision giảm từ 0.5934 xuống 0.4349, tức bắt thêm 2,695 ca dương thật nhưng báo động nhầm thêm 4,402 ca. Nâng lên 0.7 đẩy recall xuống 0.0161. Với bài sàng lọc bệnh, bỏ sót tổn kém hơn báo động nhầm, nên ngưỡng thấp là lựa chọn hợp lý hơn 0.5.

So với M1, ROC-AUC cao hơn 0.0001 và F1 lệch -0.0112, trong khi số tham số tăng từ 289 lên 833 và thời gian huấn luyện tăng từ 8s lên 14s. Thêm sức chứa không mua được chất lượng ở bài này, vì giới hạn nằm ở lượng thông tin trong tám đặc trưng chứ không ở khả năng biểu diễn của mạng.

6. Mô hình M3 Shallow

Bỏ hẳn lớp ẩn thứ hai, chỉ còn một tầng biểu diễn. Đây là phép thử xem chiều sâu có thực sự cần cho tám đặc trưng bằng hay không.

0.8374

ACCURACY · TEST

0.2913

F1 · NGUỖNG 0.5

0.8037

ROC-AUC · TEST

6s

THỜI GIAN HUẤN LUYỆN

Cấu hình và quá trình huấn luyện của M3

Hạng mục	Giá trị
Kiến trúc	8 - 16 - 1
Kích hoạt lớp ẩn	ReLU
Số tham số học được	161
Kích thước biểu diễn trung gian	H1: (183,824, 16)
Học suất, epoch, lô	0.05 · 60 · 512
Số lần cập nhật tham số	21,600
Loss train đầu và cuối	0.3780 → 0.3707
Loss trên tập test	0.3704
Thời gian huấn luyện	5.8 giây

Chênh lệch accuracy giữa train và test là -0.0016, và loss test (0.3704) so với loss train (0.3707) lệch -0.0003. Khoảng cách hẹp như vậy cho thấy mô hình chưa quá khớp, phần thiếu nằm ở sức biểu diễn chứ không ở việc học thuộc dữ liệu.

Kết quả của M3 trên tập test theo ba ngưỡng quyết định

Ngưỡng	Accuracy	Precision	Recall	F1	TP	FN	FP	TN
0.3	0.8006	0.4357	0.5199	0.4741	4,130	3,814	5,349	32,664
0.5	0.8374	0.5903	0.1934	0.2913	1,536	6,408	1,066	36,947
0.7	0.8292	0.7123	0.0196	0.0382	156	7,788	63	37,950

Hạ ngưỡng từ 0.5 xuống 0.3 nâng recall từ 0.1934 lên 0.5199 và đổi lại precision giảm từ 0.5903 xuống 0.4357, tức bắt thêm 2,594 ca dương thật nhưng báo động nhầm thêm 4,283 ca. Nâng lên 0.7 đẩy recall xuống 0.0196. Với bài sàng lọc bệnh, bỏ sót tổn kém hơn báo động nhầm, nên ngưỡng thấp là lựa chọn hợp lý hơn 0.5.

Bỏ hẳn một lớp ẩn làm ROC-AUC thấp hơn 0.0005 so với M1, với chỉ 161 tham số, bằng khoảng 56% của M1, và nhanh hơn 1.4 lần. Nếu phải chọn một mô hình để triển khai thì đây là ứng viên hợp lý nhất: chất lượng ngang ngửa với chi phí thấp nhất.

7. Mô hình M4 No ReLU

Giữ nguyên ba lớp nhưng thay ReLU bằng hàm đồng nhất. Khi đó ba phép biến đổi tuyến tính xếp chồng vẫn chỉ là một phép tuyến tính, nên mạng suy biến thành hồi quy logistic dù vẫn có 289 tham số.

0.8328	0.2839	0.7967	7s
ACCURACY · TEST	F1 · NGUỖNG 0.5	ROC-AUC · TEST	THỜI GIAN HUẤN LUYỆN

Cấu hình và quá trình huấn luyện của M4

Hạng mục	Giá trị
Kiến trúc	8 - 16 - 8 - 1
Kích hoạt lớp ẩn	tuyến tính (hàm đồng nhất)
Số tham số học được	289
Kích thước biểu diễn trung gian	H1: (183,824, 16), H2: (183,824, 8)
Học suất, epoch, lô	$0.05 \cdot 60 \cdot 512$
Số lần cập nhật tham số	21,600
Loss train đầu và cuối	$0.3786 \rightarrow 0.3781$
Loss trên tập test	0.3776
Thời gian huấn luyện	6.7 giây

Chênh lệch accuracy giữa train và test là -0.0009, và loss test (0.3776) so với loss train (0.3781) lệch -0.0005. Khoảng cách hẹp như vậy cho thấy mô hình chưa quá khớp, phần thiếu nằm ở sức biểu diễn chứ không ở việc học thuộc dữ liệu.

Kết quả của M4 trên tập test theo ba ngưỡng quyết định

Ngưỡng	Accuracy	Precision	Recall	F1	TP	FN	FP	TN
0.3	0.8007	0.4334	0.4989	0.4639	3,963	3,981	5,180	32,833
0.5	0.8328	0.5471	0.1917	0.2839	1,523	6,421	1,261	36,752
0.7	0.8299	0.6304	0.0386	0.0728	307	7,637	180	37,833

Hạ ngưỡng từ 0.5 xuống 0.3 nâng recall từ 0.1917 lên 0.4989 và đổi lại precision giảm từ 0.5471 xuống 0.4334, tức bắt thêm 2,440 ca dương thật nhưng báo động nhầm thêm 3,919 ca. Nâng lên 0.7 đẩy recall xuống 0.0386. Với bài sàng lọc bệnh, bỏ sót tổn kém hơn báo động nhầm, nên ngưỡng thấp là lựa chọn hợp lý hơn 0.5.

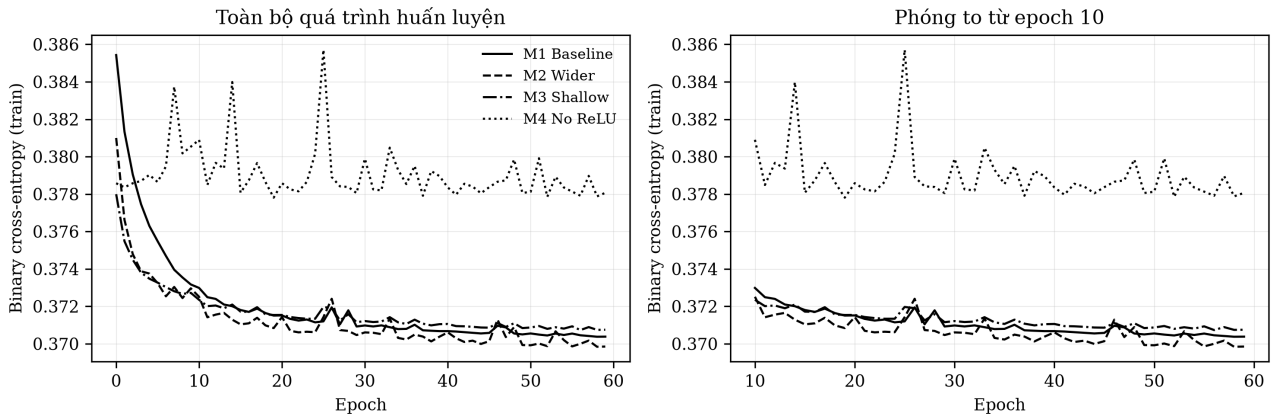
Bỏ ReLU làm ROC-AUC thấp hơn 0.0076 và F1 lệch -0.0087 so với M1. Mức giảm có thật nhưng nhỏ, và đó mới là điều đáng nói: ba phép biến đổi tuyến tính xếp chồng chỉ tương đương một phép tuyến tính, nên M4 thực chất là hồi quy logistic đội lốt mạng ba lớp. Việc nó gần bằng ba mô hình kia là bằng chứng trực tiếp rằng phần lớn tín hiệu trong tám đặc trưng này là tuyến tính.

8. So sánh bốn mô hình

Bảng tổng hợp bốn mô hình trên tập test, ngưỡng 0.5

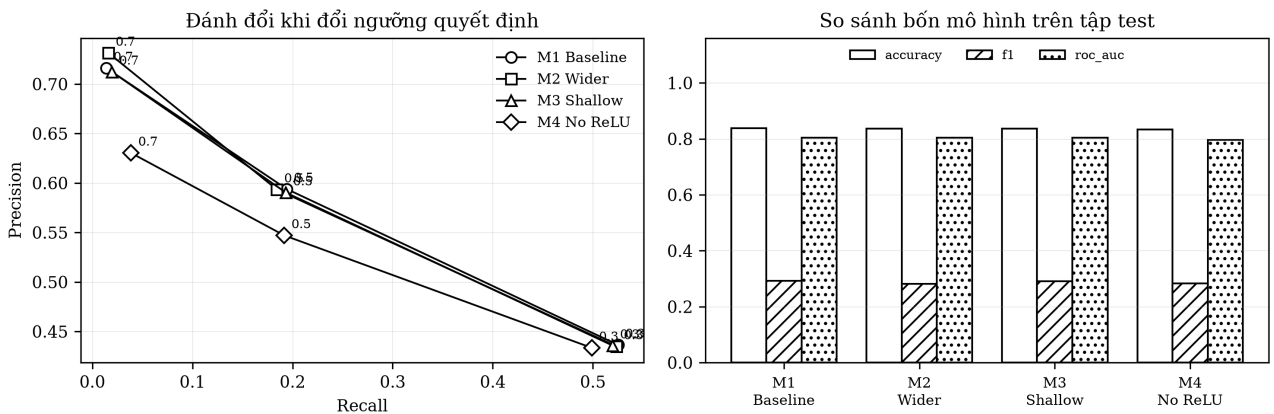
Mô hình	Tham số	Loss train	Loss test	Accuracy	Precision	Recall	F1	ROC-AUC	Giây
M1 Baseline	289	0.3704	0.3702	0.8378	0.5940	0.1941	0.2926	0.8042	7.9
M2 Wider	833	0.3699	0.3700	0.8372	0.5934	0.1844	0.2814	0.8043	13.7
M3 Shallow	161	0.3707	0.3704	0.8374	0.5903	0.1934	0.2913	0.8037	5.8
M4 No ReLU	289	0.3781	0.3776	0.8328	0.5471	0.1917	0.2839	0.7967	6.7

Dòng in đậm là mô hình tốt nhất theo ROC-AUC. Chỉ số này được chọn làm tiêu chí chính vì nó không phụ thuộc ngưỡng quyết định, trong khi accuracy và F1 tại ngưỡng 0.5 đều bị lớp âm chiếm đa số kéo lệch.



Đường mất mát trên tập train của bốn mô hình, toàn bộ quá trình và phần phóng to từ epoch 10.

Hình f2Observation: cả bốn đường tụt rất nhanh trong mười epoch đầu rồi gần như nằm ngang, và ba đường có ReLU nằm chồng lên nhau. **Interpretation:** phần dễ học của bài toán được học xong sớm, phần còn lại không nằm trong tầm với của tám đặc trưng này. **ML implication:** kéo dài huấn luyện hay tăng sức chứa đều không đáng, muốn tiến thêm phải thêm đặc trưng chứ không phải thêm lớp.



Đánh đổi giữa precision và recall khi đổi ngưỡng quyết định, và so sánh ba chỉ số chính của bốn mô hình.

8.2 Điều bốn mô hình nói lên

- **Chiều sâu gần như không đổi được kết quả.** M3 chỉ có một lớp ẩn và 161 tham số nhưng ROC-AUC chỉ lệch 0.0005 so với M1.
- **Bề rộng cũng vậy.** M2 nhiều hơn M1 544 tham số và tốn thêm 6 giây mà không đổi được thứ hạng.
- **Phi tuyến có đóng góp, nhưng nhỏ.** M4 mất 0.0076 ROC-AUC khi bỏ ReLU. Đây là số đo của phần tín hiệu phi tuyến trong tám đặc trưng này.
- **Ngưỡng quyết định đáng giá hơn kiến trúc.** Hạ ngưỡng từ 0.5 xuống 0.3 đổi recall của M1 nhiều hơn hẳn mọi khác biệt giữa bốn kiến trúc cộng lại.

9. Ba mô hình machine learning cho mỗi category

Ba ứng dụng của Assignment 02 đã có bảng so sánh riêng, nhưng mọi mô hình trong các bảng đó đều thuộc một trong ba nhóm: tuyến tính, xác suất sinh, hoặc cây và tổ hợp cây. Hai notebook bổ sung lấp hai khoảng trống còn lại. **KNN** mang vào nhóm học dựa trên thực thể, thứ chưa từng xuất hiện trong dự án. **AdaBoost** mang vào một cách hiện thực hoá boosting khác với gradient boosting đang dùng, để tách bạch xem phần thắng của boosting đến từ ý tưởng chung hay từ chi tiết cài đặt.

Điều kiện đo Dữ liệu trong `data/` đã được bổ sung sau khi notebook 01-03 chạy lần cuối. Vì vậy mô hình mốc **được huấn luyện lại** trên bộ dữ liệu hiện tại thay vì chép lại con số của Assignment 02. Mọi số trong chương này đo trên cùng một lần chia 70/15/15 với seed 42, nên chênh lệch giữa các dòng quy được về thuật toán. Ngược lại, chúng **không** so trực tiếp được với bảng trong báo cáo Assignment 02.

Phạm vi đo: ba category, bốn bài toán

Category	Bài toán	Kiểu	Train	Val	Test	Mô hình mốc
Tiểu đường	nguy cơ tiểu đường	nhị phân	160,846	34,467	34,468	Random Forest
Giá nhà	giá nhà	hồi quy	175,000	37,500	37,500	HistGradientBoosting
E-commerce	mức hài lòng	nhị phân	20,118	4,312	4,312	LogReg bảng + TF-IDF
E-commerce	nhóm sản phẩm	9 lớp	4,074	873	873	LogReg bảng + TF-IDF

Category e-commerce có hai target độc lập trên cùng một bảng dữ liệu, nên được tính là hai bài toán. Cả bốn bài toán đều nhận đủ ba mô hình.

9.1 Ba bài toán phân loại

Kết quả trên tập test của ba bài toán phân loại

Category	Nhóm	Mô hình	Accuracy	Recall	F1-macro	ROC-AUC
Tiểu đường nguy cơ tiểu đường	KNN	KNN (k=5)	0.8155	0.2453	0.6041	0.7159
	AdaBoost	AdaBoost (50 vòng, cây sâu 3)	0.7202	0.7493	0.6446	0.8078
	Mốc	Random Forest	0.7295	0.7414	0.6514	0.8122
E-commerce mức hài lòng	KNN	KNN cosin (k=11)	0.7799	0.9437	0.5880	0.7125
	AdaBoost	AdaBoost (300 vòng, cây sâu 3)	0.7711	0.7507	0.7295	0.8755
	Mốc	LogReg bảng + TF-IDF	0.8353	0.8390	0.7900	0.8950
E-commerce nhóm sản phẩm	KNN	KNN cosin (k=11)	0.3219	0.2099	0.2178	-
	AdaBoost	AdaBoost (100 vòng, cây sâu 3)	0.4181	0.3900	0.3491	-
	Mốc	LogReg bảng + TF-IDF	0.3574	0.2791	0.2735	-

Dòng in đậm là mô hình đạt F1-macro cao nhất trong từng bài toán. F1-macro được dùng làm tiêu chí thay vì accuracy vì cả ba bài toán đều lệch lớp: đoán toàn lớp đa số cho accuracy khá cao mà không học được gì.

Bài toán tiểu đường cho thấy rõ nhất khác biệt giữa hai hướng bổ sung. AdaBoost đạt F1-macro 0.6446, gần như chạm mốc Random Forest 0.6514, vì cả hai đều là tổ hợp cây và cùng được cân bằng lớp. KNN chỉ đạt 0.6041, và toàn bộ

khoảng cách nằm ở recall: 0.2453 so với 0.7414. Lý do không nằm ở chất lượng cài đặt mà ở bản chất thuật toán: KNN không có tham số nào để gắn trọng số lớp, nên khi 83% hàng xóm thuộc lớp âm thì lá phiếu nghiêng hẳn về lớp âm.

Bài toán nhóm sản phẩm là ngoại lệ duy nhất: AdaBoost đạt F1-macro 0.3491, vượt mốc 0.2735. Không nên đọc con số này thành bằng chứng về chất lượng thuật toán. Cả hai mô hình đều quanh quẩn mức đoán mò trên 9 lớp, và khi không mô hình nào thật sự học được quy luật thì thứ hạng bị quyết định bởi cách phân bổ dự đoán ra các lớp chứ không phải bởi năng lực học.

9.2 Bài toán hồi quy

Kết quả trên tập test của bài toán giá nhà

Nhóm	Mô hình	R^2 (log)	MAE (USD)	RMSE (USD)	MAPE (%)
KNN	KNN (k=25)	0.6370	135,036	218,676	34.4
AdaBoost	AdaBoost.R2 (10 vòng, cây sâu 8)	0.6338	136,147	219,970	34.0
Mốc	HistGradientBoosting	0.7064	120,717	199,175	30.1

Khoảng cách giữa AdaBoost (0.6338) và HistGradientBoosting (0.7064) trên cùng một biểu diễn 8 chiều là khoảng cách giữa hai cách hiện thực hoá cùng một ý tưởng boosting. AdaBoost.R2 cập nhật trọng số theo sai số tuyệt đối đã chuẩn hoá, tức chỉ biết mẫu nào sai nhiều hơn chứ không biết sai theo hướng nào; gradient boosting học thẳng phần dư có dấu nên mỗi vòng sửa đúng chiều còn thiếu. Bảng quét số vòng trong notebook 06 xác nhận điều đó: chất lượng đạt đỉnh ở vòng 10 rồi đi xuống khi chạy thêm.

KNN đạt 0.6370, sát AdaBoost hơn mọi bài toán khác. Giá nhà là bài toán có tính địa phương cao: hai căn cùng thành phố, cùng diện tích, cùng số phòng thì giá thật sự gần nhau, và đó đúng là giả thiết mà KNN đặt ra.

Hai phát hiện về phương pháp, không phải về kết quả

Chuẩn hoá không phải thủ tục hình thức. Biểu diễn `target_enc` của Assignment 02 được thiết kế cho cây quyết định, vốn cắt theo từng chiều nên miễn nhiệm với thang đo. Đưa KNN vào mới lộ ra rằng biểu diễn đó không dùng ngay được: tám chiều lệch nhau hàng nghìn lần, phải ghép thêm một bước `StandardScaler` thì khoảng cách Euclid mới có nghĩa.

Không được đặt `class_weight="balanced"` lên bộ học yếu của AdaBoost. AdaBoost tính tỉ lệ lỗi theo bộ trọng số của nó, trong khi `class_weight` khiến cây tối ưu theo một bộ trọng số khác. Hai bộ lệch nhau đẩy tỉ lệ lỗi vòng đầu lên 0.3963 và chạm ngưỡng dừng sau đúng 5 vòng, tức ensemble 300 vòng không bao giờ được dựng. Cách đúng là truyền trọng số cân bằng lớp vào giá trị khởi tạo qua tham số `sample_weight` của `fit`.

10. Deep learning cho category giá nhà

Notebook 04 dựng mạng nơ-ron cho bài toán phân loại. Chương này đưa cùng bộ công cụ đó sang bài toán hồi quy, để category giá nhà có mô hình deep learning riêng thay vì đi mượn kết luận của category khác. Toàn bộ mạng vẫn viết bằng NumPy: lan truyền xuôi, hàm mất mát, lan truyền ngược và gradient descent đều viết tay.

Chỉ hai thứ phải đổi khi chuyển từ phân loại sang hồi quy. Lớp cuối bỏ sigmoid, trả về số thực bất kỳ. Hàm mất mát đổi từ binary cross entropy sang sai số bình phương trung bình. Thứ ba thì **không** phải đổi, và đó mới là điểm đáng nói: hàm lan truyền ngược giữ nguyên từng dòng.

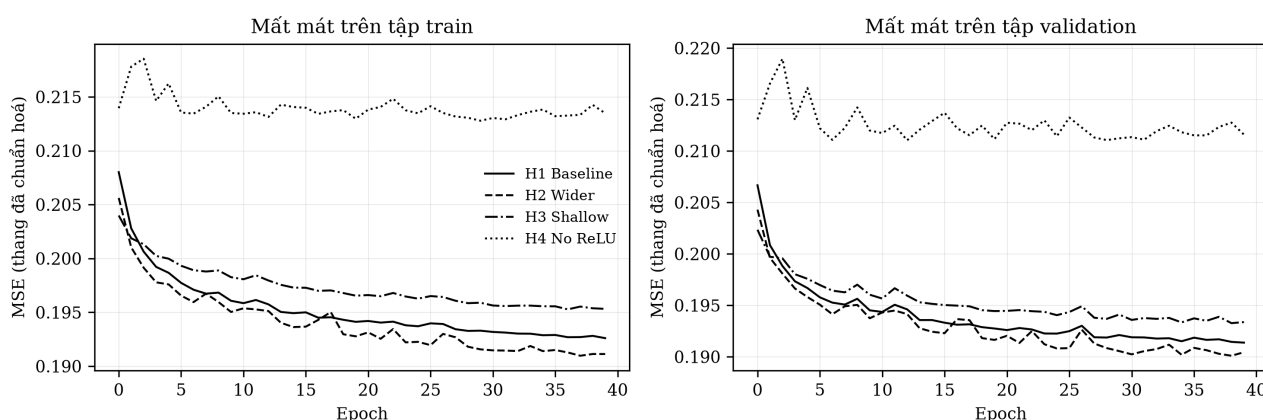
$$\partial L / \partial z^{(L)} = (\hat{y} - y) / n \quad \text{đúng cho cả sigmoid + cross entropy lần đầu ra đồng nhất + sai số bình phương}$$

Đây là tính chất của các cặp hàm liên kết chuẩn: đạo hàm của hàm kích hoạt triệt tiêu gọn với mẫu số của hàm mất mát tương ứng. Nó cũng là lý do các thư viện deep learning tách rời được phần mạng khỏi phần hàm mất mát.

Bốn kiến trúc hồi quy, đo trên tập validation

Mô hình	Kiến trúc	Kích hoạt	Tham số	Loss	R ² (log)	MAE (USD)	Giây
H1 Baseline	8 - 32 - 16 - 1	ReLU	833	0.19137	0.6191	136,891	9
H2 Wider	8 - 64 - 32 - 1	ReLU	2,689	0.19044	0.6210	136,465	18
H3 Shallow	8 - 32 - 1	ReLU	321	0.19337	0.6151	137,838	6
H4 No ReLU	8 - 32 - 16 - 1	tuyến tính	833	0.21159	0.5789	146,429	7

Cả bốn dùng chung seed khởi tạo 42, học suất 0.05, 40 epoch và lô 512 dòng, nên chênh lệch giữa chúng quy được về kiến trúc. Biến mục tiêu là log(giá) đã chuẩn hoá bằng trung bình và độ lệch chuẩn của tập train.

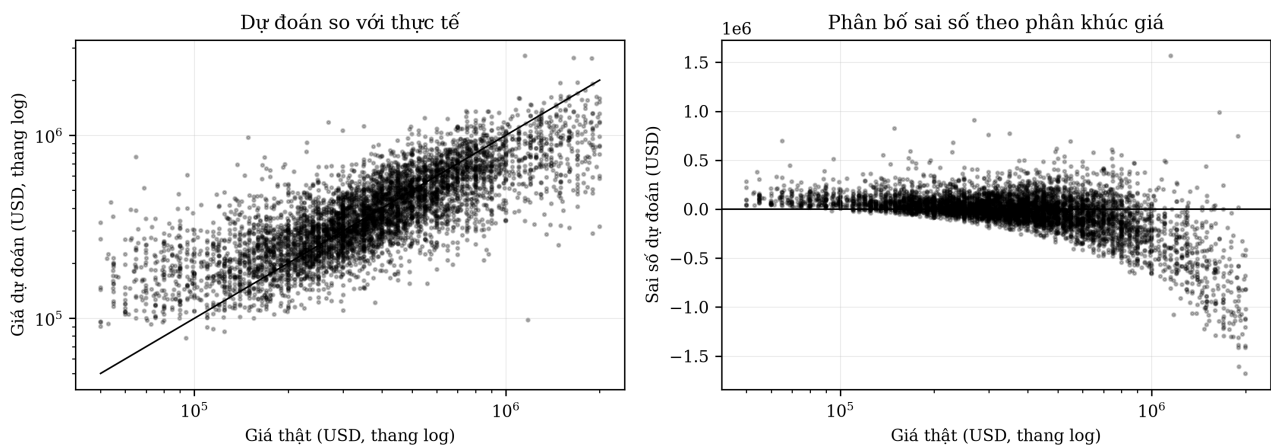


Mất mát trên tập train và tập validation của bốn kiến trúc hồi quy. Ba kiến trúc có ReLU chồng lên nhau, kiến trúc bỏ ReLU đứng hẳn ở mức cao hơn.

Kiến trúc H4 bỏ ReLU mất 0.0421 điểm R² dù có đúng số tham số như kiến trúc gốc. Ba phép biến đổi tuyến tính xếp chồng vẫn chỉ là một phép tuyến tính, nên mạng suy biến thành hồi quy tuyến tính. Ngược lại, nhân đôi bề rộng chỉ đổi được rất ít: phi tuyến mới là thứ tạo ra giá trị, không phải số tham số.

Mạng nơ-ron so với mô hình ML tốt nhất, trên tập test

Mô hình	R ² (log)	MAE (USD)	RMSE (USD)	MAPE (%)
Mạng nơ-ron H2 (8 - 64 - 32 - 1)	0.6247	136,767	220,529	35.6
HistGradientBoosting	0.7064	120,717	199,175	30.1



Dự đoán so với giá thật, và phân bố sai số theo phân khúc giá. Sai số loe rộng dần ở hai đầu và lệch xuống có hệ thống ở phân khúc cao.

Mạng nơ-ron không vượt được HistGradientBoosting trên tám chiều đặc trưng bảng, chênh 0.0817 điểm R^2 và 16,050 USD ở MAE. Kết quả này khớp với kết luận phổ biến trong ngành và không có gì bất ngờ: khi đầu vào đã là một bảng số sạch với 8 cột, cây quyết định vẫn là lựa chọn đúng. Khuôn mẫu sai số ở hình trên cũng không phải lỗi của thuật toán mà là giới hạn của dữ liệu: tám đặc trưng hiện có không phân biệt được một căn ba triệu đô với một căn tám trăm nghìn trong cùng thành phố, cùng diện tích.

11. Deep learning cho category e-commerce

Ba chương trước đều làm việc trên bảng số. Chương này xử lý loại dữ liệu mà deep learning thật sự sinh ra để giải: văn bản.

Assignment 02 đã dựng tensor văn bản $\mathbf{B} \times \mathbf{T} \times \mathbf{d}$ bằng một ma trận nhúng ngẫu nhiên cố định rồi lấy trung bình theo trục thời gian. Ma trận đó không bao giờ được cập nhật, nên nó chỉ là một phép chiếu ngẫu nhiên. Chương này đi nốt phần còn lại: ma trận nhúng trở thành tham số của mạng và được học bằng lan truyền ngược, giống mọi trọng số khác.

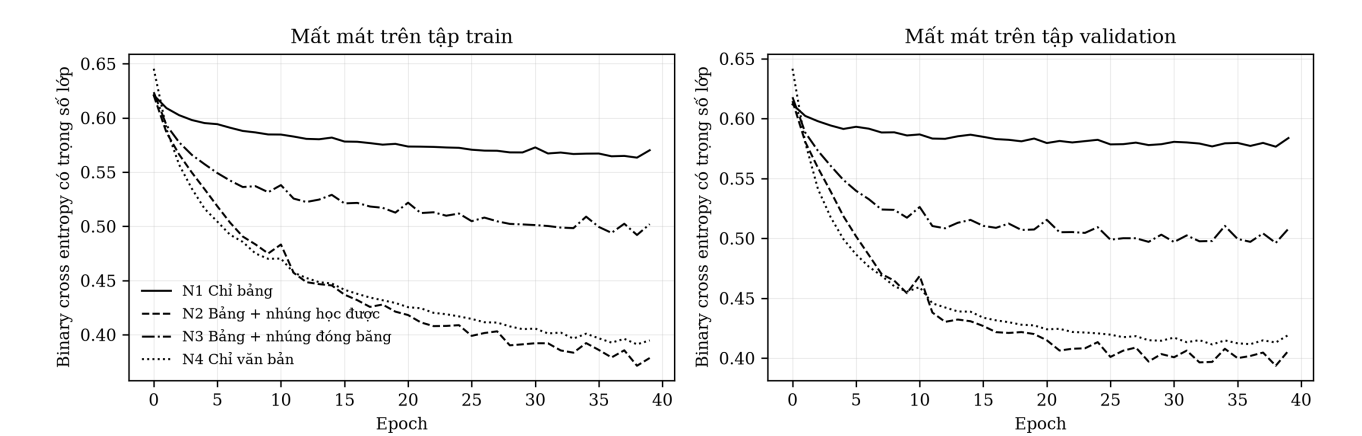
$$p_b = (1/c_b) \sum_t m_{bt} E[ids_{bt}] \rightarrow \partial L / \partial E[v] = \sum_{(b,t): ids=v} (1/c_b) \partial L / \partial p_b$$

Đạo hàm đi ngược qua phép gộp trung bình có che chỉ là phép cộng dồn về đúng những hàng từ vựng đã được tra. Một từ xuất hiện ở nhiều bình luận thì gradient của nó là tổng các đóng góp, nên phải cộng dồn chứ không gán đè. Toàn bộ cơ chế nằm gọn trong sáu dòng NumPy.

Bốn cấu hình mạng, đo trên tập validation

Mô hình	Dùng bảng	Ma trận nhúng	d vào	Tham số	Accuracy	F1-macro	ROC-AUC
N1 Chỉ bảng	có	không dùng	77	3,041	0.7247	0.6585	0.7664
N2 Bảng + nhúng học được	có	học được	109	164,097	0.8564	0.8069	0.9036
N3 Bảng + nhúng đóng băng	có	đóng băng	109	164,097	0.7934	0.7306	0.8390
N4 Chỉ văn bản	không	học được	32	161,633	0.8537	0.8007	0.8958

Từ điển 5,000 từ xây **chỉ từ tập train**, chuỗi cắt và đệm về 32 token, mỗi từ 32 chiều. Hàm mất mát có trọng số lớp (0.6453 cho lớp hài lòng, 2.2205 cho lớp còn lại) để mạng không đi đường tắt bằng cách đoán tất cả là hài lòng.



Mất mát có trọng số lớp trên tập train và validation của bốn cấu hình. Hai đường thấp nhất đều là cấu hình có ma trận nhúng được huấn luyện.

11.1 Tách bạch hai nguồn đóng góp

N1 và N2 chỉ khác nhau ở chỗ có dùng văn bản hay không. N2 và N3 dùng **đúng cùng một ma trận nhúng**, đúng cùng số tham số, đúng cùng kiến trúc, chỉ khác ở chỗ N3 đóng băng ma trận đó ở giá trị ngẫu nhiên ban đầu. Hai phép so sánh đó tách được hai thứ mà Assignment 02 gộp làm một.

+0.1484	+0.0762	+0.1431	164,097
CÓ VĂN BẢN · N2 – N1	HỌC NHÚNG · N2 – N3	TF-IDF · ASSIGNMENT 02	THAM SỐ CỦA N2

Mức tăng nhờ có văn bản là +0.1484 F1-macro, gần như trùng khớp với +0.1431 mà Assignment 02 đo được bằng TF-IDF trên bộ dữ liệu cũ. Hai cách biểu diễn văn bản hoàn toàn khác nhau, hai bộ dữ liệu khác nhau, nhưng cùng cho một kết luận: bình luận của khách chứa khoảng chừng ấy thông tin về mức hài lòng. Đó là thuộc tính của dữ liệu, không phải của thuật toán.

Con số thứ hai mới là phần Assignment 02 chưa trả lời được. Chênh lệch +0.0762 F1-macro giữa N2 và N3 là phần đóng góp của riêng việc **học** biểu diễn từ, tách bạch khỏi phần đóng góp của việc có văn bản. Hơn một nửa giá trị mà văn bản mang lại chỉ xuất hiện khi biểu diễn của từ được huấn luyện cùng bài toán.

Ma trận nhúng khởi tạo hoàn toàn ngẫu nhiên và không được cung cấp bất kỳ từ điển cảm xúc nào. Sau 40 epoch, bad và worst là hàng xóm gần nhất của nhau theo cosin, còn good kéo về nice. Cấu trúc ngữ nghĩa đó không được lập trình vào đâu cả: nó xuất hiện vì hai từ có tác động giống nhau lên mục tiêu thì gradient đẩy chúng về cùng một hướng.

11.2 So với mô hình machine learning tốt nhất

Mạng nơ-ron so với mô hình ML tốt nhất, trên tập test

Mô hình	d vào	Accuracy	Recall	F1	F1-macro	ROC-AUC
Mạng nơ-ron N2 (Bảng + nhúng học được)	109	0.8472	0.8794	0.8992	0.7918	0.8841
LogReg bảng + TF-IDF	11,835	0.8353	0.8390	0.8876	0.7900	0.8950

Đây là lần đầu tiên trong cả dự án một mô hình deep learning viết tay vượt mô hình machine learning tốt nhất. Mốc ML vẫn nhỉnh hơn ở ROC-AUC (0.8950 so với 0.8841), tức nó xếp hạng các ca tốt hơn một chút, nhưng ở ngưỡng quyết định 0.5 thì mạng phân loại chính xác hơn.

Đọc chương 10 và 11 cùng nhau

Trên tám chiều đặc trưng bằng của bài toán giá nhà, mạng nơ-ron thua HistGradientBoosting một khoảng rộng. Trên dữ liệu có văn bản, chính bộ công cụ ấy lại vượt mốc. Không phải deep learning mạnh hơn hay yếu hơn machine learning cổ điển, mà là nó có chỗ dừng: khi đầu vào cần được **học cách biểu diễn**, chứ không phải khi đầu vào đã là một bảng số sạch.

12. Tổng hợp: ba mô hình ML và một mô hình DL cho mỗi category

Bốn notebook bổ sung đưa mỗi category lên đủ ba mô hình machine learning và một mô hình deep learning. Hai bảng dưới đây gom toàn bộ kết quả trên tập test về một chỗ.

Ba bài toán phân loại, toàn bộ mô hình, trên tập test

Category	Nhóm	Mô hình	F1-macro	ROC-AUC
Tiểu đường nguy cơ tiểu đường	ML mốc	Random Forest	0.6514	0.8122
	ML bổ sung	KNN (k=5)	0.6041	0.7159
	ML bổ sung	AdaBoost (50 vòng, cây sâu 3)	0.6446	0.8078
	Deep learning	M2 Wider (*)	-	0.8043
E-commerce mức hài lòng	ML mốc	LogReg bảng + TF-IDF	0.7900	0.8950
	ML bổ sung	KNN cosin (k=11)	0.5880	0.7125
	ML bổ sung	AdaBoost (300 vòng, cây sâu 3)	0.7295	0.8755
	Deep learning	N2 Bảng + nhúng học được	0.7918	0.8841
E-commerce nhóm sản phẩm	ML mốc	LogReg bảng + TF-IDF	0.2735	-
	ML bổ sung	KNN cosin (k=11)	0.2178	-
	ML bổ sung	AdaBoost (100 vòng, cây sâu 3)	0.3491	-
	Deep learning	chưa có	-	-

(*) Mô hình deep learning của bài toán tiểu đường là mạng của notebook 04. Nó dùng **tám** trong số 21 đặc trưng và chia tập theo tỉ lệ 80/20 thay vì 70/15/15, nên con số của nó **không** so trực tiếp được với ba dòng phía trên; đặt cạnh nhau ở đây chỉ để thấy phạm vi đã phủ. Bài toán nhóm sản phẩm chưa có mô hình deep learning riêng.

Bài toán hồi quy, toàn bộ mô hình, trên tập test

Nhóm	Mô hình	R^2 (log)	MAE (USD)	MAPE (%)
ML gốc	HistGradientBoosting	0.7064	120,717	30.1
ML bổ sung	KNN (k=25)	0.6370	135,036	34.4
ML bổ sung	AdaBoost.R2 (10 vòng, cây sâu 8)	0.6338	136,147	34.0
Deep learning	H2 Wider (8 - 64 - 32 - 1)	0.6247	136,767	35.6

Đọc ngang hai bảng thì thấy một khuôn mẫu nhất quán. Ở ba bài toán có đầu vào là bảng số đã được đặc trưng hoá bằng tay, mô hình gốc của Assignment 02 vẫn đứng đầu và hai mô hình bổ sung xếp dưới. Ở bài toán duy nhất có đầu vào là văn bản thô, mạng nơ-ron học biểu diễn vượt lên. Đây không phải kết luận về việc thuật toán nào mạnh hơn, mà về việc mỗi thuật toán giả định điều gì về dữ liệu, và giả định đó có đúng với dữ liệu đang có hay không.

Điều báo cáo này không kết luận **Một, không có mô hình triển khai nào thay đổi**. Bốn notebook bổ sung mở rộng bảng so sánh, không thay artifact nào đang phục vụ API. **Hai, mọi số trong chương 9 đến 12 đo trên bộ dữ liệu hiện tại**, đã được bổ sung sau khi notebook 01-03 chạy lần cuối, nên không so trực tiếp được với bảng của Assignment 02. **Ba, siêu tham số chỉ được dò trên một lưới nhỏ** (giá trị k của KNN, số vòng của AdaBoost, bốn kiến trúc mạng). Các con số là ước lượng trung thực cho đúng những cấu hình đã thử, không phải cho cấu hình tốt nhất có thể của từng thuật toán.

13. Kết luận và giới hạn

Một mạng nơ-ron ba lớp viết hoàn toàn bằng NumPy đã học được trên hơn hai trăm nghìn dòng dữ liệu thật, chạy trên CPU trong vài chục giây mỗi mô hình. Điều framework tự động hoá không phải bản chất toán học mà là công sức triển khai: đạo hàm tự động, quản lý tensor, tối ưu hoá và tính toán trên GPU.

Mô hình tốt nhất theo ROC-AUC là M2 Wider với 0.8043. Nhưng kết luận có ích hơn cho việc triển khai lại là M3: ít tham số nhất, nhanh nhất, chất lượng gần như không kém.

Giới hạn tự nhận **Một, accuracy ở đây dễ gây hiểu nhầm**. Đoán toàn lớp âm đã cho khoảng 0.8271, nên mọi con số accuracy trong báo cáo phải đọc kèm recall và ROC-AUC. **Hai, chọn đặc trưng bằng tương quan tuyến tính đơn biến** có thể đã loại mất biến chỉ có ích khi kết hợp với biến khác, và điều đó tự nó đã thu hẹp phần phi tuyến còn lại cho mạng khai thác. **Ba, không có tập validation riêng**. Siêu tham số được chọn bằng suy luận về số lần cập nhật chứ không bằng dò trên tập validation, nên các con số trên tập test là ước lượng trung thực cho đúng cấu hình này chứ không phải cho cấu hình tốt nhất có thể.

Hướng đi tiếp có ích nhất không phải mạng sâu hơn mà là thêm thông tin đầu vào: dùng đủ 21 biến thay vì tám, thêm đặc trưng tương tác, hoặc cân bằng lại lớp bằng trọng số trong hàm mất mát. Cả bốn mô hình ở đây đều đã chạm trần thông tin của tám cột hiện có.

PHẦN PHỤ LỤC

Phụ lục A. Notebook 04: Deep Learning từ đầu

Toàn bộ nội dung notebook, giữ nguyên thứ tự cell. Khối In [n] là mã nguồn thực thi, khối Out là kết quả chạy thật trên máy cục bộ. Mọi số liệu trong phần thân báo cáo đều được tham chiếu đồng nhất từ các thí nghiệm này.

04_deep_learning.ipynb · 21 cell · 14 code

Ứng dụng 4: Deep Learning từ đầu với NumPy

Notebook này dựng lại toàn bộ một mạng nơ-ron nhiều lớp **chỉ bằng NumPy**: lan truyền xuôi, hàm mất mát, lan truyền ngược và gradient descent đều viết tay. Không dùng TensorFlow, PyTorch, Keras hay scikit-learn cho phần học máy.

Bốn mô hình được huấn luyện trên cùng một tập dữ liệu và cùng một quy trình, chỉ khác nhau ở kiến trúc:

| Mã | Kiến trúc | Kích hoạt lớp ẩn | Ý nghĩa |

|---|---|---|---|

| M1 | 8 - 16 - 8 - 1 | ReLU | Kiến trúc gốc của bài giảng |

| M2 | 8 - 32 - 16 - 1 | ReLU | Rộng gấp đôi |

| M3 | 8 - 16 - 1 | ReLU | Bỏ bớt một lớp ẩn |

| M4 | 8 - 16 - 8 - 1 | tuyến tính | Bỏ ReLU, kiểm chứng vai trò của phi tuyến |

In [1]

```
1 import json
2 import time
3 from pathlib import Path
4
5 import matplotlib
6 import matplotlib.pyplot as plt
7 import numpy as np
8 import pandas as pd
9
10 ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
11 DATA = ROOT / "data"
12 REPORTS = ROOT / "reports"
13
14 matplotlib.rcParams.update({
15     "font.family": "serif",
16     "font.size": 9,
17     "axes.edgecolor": "black",
18     "axes.labelcolor": "black",
19     "text.color": "black",
20     "xtick.color": "black",
21     "ytick.color": "black",
22     "figure.facecolor": "white",
23     "savefig.facecolor": "white",
24 })
```

```

25
26 print("thư mục gốc :", ROOT)
27 print("numpy :", np.__version__)
28 print("pandas :", pd.__version__)

```

Out

```

thư mục gốc : E:\smart system\ass3
numpy : 2.4.6
pandas : 3.0.6

```

1. Nạp dữ liệu và chọn 8 đặc trưng

Bộ BRFS 2015 ở đây có 21 biến nên cần chọn ra 8 theo trị tuyệt đối hệ số tương quan với nhãn nhị phân.

In [2]

```

1 DIABETES_CSV = DATA / "diabetes" / "diabetes.csv"
2 TARGET = "Diabetes_012"
3 SPLIT_SEED = 42
4 TEST_SHARE = 0.20
5
6 raw = pd.read_csv(DIABETES_CSV)
7 frame = raw.drop_duplicates().reset_index(drop=True)
8 y_all = (frame[TARGET] > 0).astype(int)
9
10 ranking = (
11     frame.drop(columns=[TARGET])
12     .apply(lambda s: np.corrcoef(s, y_all)[0, 1])
13     .abs()
14     .sort_values(ascending=False)
15 )
16
17 FEATURES = tuple(ranking.index[:8])
18
19 print(f"Dòng thô : {len(raw):,}")
20 print(f"Sau khi bỏ trùng : {len(frame):,} (bỏ {len(raw) - len(frame):,})")
21 print(f"Tỉ lệ lớp dương : {y_all.mean():.4f}")
22 print("\nXếp hạng |tương quan| với nhãn nhị phân:")
23 print(ranking.round(4).to_string())
24 print("\n8 đặc trưng được chọn:", list(FEATURES))

```

Out

```

Dòng thô : 253,680
Sau khi bỏ trùng : 229,781 (bỏ 23,899)
Tỉ lệ lớp dương : 0.1729

```

Xếp hạng |tương quan| với nhãn nhị phân:

GenHlth	0.2823
HighBP	0.2603
BMI	0.2112
DiffWalk	0.2081
HighChol	0.2044
Age	0.1855
HeartDiseaseorAttack	0.1672
PhysHlth	0.1587
Income	0.1478

```
Education          0.1085
... (rút gọn 11 dòng) ...
```

8 đặc trưng được chọn: ['GenHlth', 'HighBP', 'BMI', 'DiffWalk', 'HighChol', 'Age', 'HeartDiseaseorAttack', 'PhysHlth']

In [3]

```
1 X_all = frame[list(FEATURES)].to_numpy(dtype=np.float64)
2 y_all_arr = y_all.to_numpy(dtype=np.float64).reshape(-1, 1)
3
4 rng_split = np.random.default_rng(SPLIT_SEED)
5 order = rng_split.permutation(len(X_all))
6 cut = int((1.0 - TEST_SHARE) * len(X_all))
7 train_idx, test_idx = order[:cut], order[cut:]
8
9 X_train_raw, X_test_raw = X_all[train_idx], X_all[test_idx]
10 y_train, y_test = y_all_arr[train_idx], y_all_arr[test_idx]
11
12 mean = X_train_raw.mean(axis=0)
13 std = X_train_raw.std(axis=0) + 1e-8
14 X_train = (X_train_raw - mean) / std
15 X_test = (X_test_raw - mean) / std
16
17 print(f"train {len(X_train):,} dòng test {len(X_test):,} dòng")
18 print(f"tỉ lệ dương train {y_train.mean():.4f} test {y_test.mean():.4f}")
19 print("\nChuẩn hoá bằng mean/std của tập train, áp lại cho test:")
20 for name, m, s in zip(FEATURES, mean, std):
21     print(f" {name:22s} mean {m:8.4f} std {s:8.4f}")
22 print("\nSau chuẩn hoá, X_train mean ~ 0 và std ~ 1:")
23 print(" mean:", np.round(X_train.mean(axis=0), 6))
24 print(" std :", np.round(X_train.std(axis=0), 6))
```

Out

```
train 183,824 dòng test 45,957 dòng
tỉ lệ dương train 0.1729 test 0.1729
```

Chuẩn hoá bằng mean/std của tập train, áp lại cho test:

GenHlth	mean	2.6004	std	1.0641
HighBP	mean	0.4545	std	0.4979
BMI	mean	28.6970	std	6.8074
DiffWalk	mean	0.1852	std	0.3884
HighChol	mean	0.4422	std	0.4966
Age	mean	8.0805	std	3.0953
HeartDiseaseorAttack	mean	0.1035	std	0.3046
PhysHlth	mean	4.6694	std	9.0155

Sau chuẩn hoá, X_train mean ~ 0 và std ~ 1:

```
mean: [-0. -0.  0. -0.  0.  0. -0.]
std : [1.  1.  1.  1.  1.  1.  1.]
```

In [4]

```
1 fig, axes = plt.subplots(1, 3, figsize=(10.2, 3.0))
2
3 counts = frame[TARGET].value_counts().sort_index()
4 axes[0].bar([str(int(k)) for k in counts.index], counts.values,
5             color="white", edgecolor="black", hatch="///")
```

```

6 axes[0].set_title("Ba lớp gốc")
7 axes[0].set_xlabel("Diabetes_012")
8 axes[0].set_ylabel("Số dòng")
9
10 binary_counts = y_all.value_counts().sort_index()
11 axes[1].bar(["0 (âm)", "1 (dương)"], binary_counts.values, color="white", edgecolor="black")
12 axes[1].set_title(f"Nhãn nhị phân, tỉ lệ dương {y_all.mean():.3f}")
13 axes[1].set_ylabel("Số dòng")
14
15 pos = ranking.iloc[:8][::-1]
16 axes[2].barh(range(len(pos)), pos.values, color="white", edgecolor="black")
17 axes[2].set_yticks(range(len(pos)))
18 axes[2].set_yticklabels(pos.index, fontsize=7)
19 axes[2].set_title("|tương quan| với nhãn")
20
21 for axis in axes:
22     axis.grid(True, axis="y", linewidth=0.3, alpha=0.4)
23 fig.tight_layout()
24 plt.show()

```

2. Mạng nơ-ron viết tay thuần NumPy

In [5]

```

1 def relu(x):
2     return np.maximum(0.0, x)
3
4
5 def relu_derivative(x):
6     return (x > 0).astype(float)
7
8
9 def identity(x):
10    return x
11
12
13 def identity_derivative(x):
14    return np.ones_like(x)
15
16
17 def sigmoid(x):
18    return 1.0 / (1.0 + np.exp(-np.clip(x, -50.0, 50.0)))
19
20
21 ACTIVATIONS = {"relu": (relu, relu_derivative), "linear": (identity, identity_derivative)}
22
23
24 def init_params(sizes, seed):
25     rng = np.random.default_rng(seed)
26     return tuple(
27         (rng.standard_normal((fan_in, fan_out)) * np.sqrt(2.0 / fan_in),
28          np.zeros((1, fan_out)))
29         for fan_in, fan_out in zip(sizes, sizes[1:]))
30
31
32
33 def forward(params, X, hidden_activation):
34     act, _ = ACTIVATIONS[hidden_activation]
35     pre, activations = [], [X]

```

```

36     h = X
37     last = len(params) - 1
38     for index, (W, b) in enumerate(params):
39         z = h @ W + b
40         h = sigmoid(z) if index == last else act(z)
41         pre.append(z)
42         activations.append(h)
43     return h, {"z": pre, "h": activations}
44
45
46 def binary_cross_entropy(y, y_hat):
47     p = np.clip(y_hat, 1e-8, 1.0 - 1e-8)
48     return float(-np.mean(y * np.log(p) + (1.0 - y) * np.log(1.0 - p)))
49
50
51 def backward(params, y, cache, hidden_activation):
52     _, act_derivative = ACTIVATIONS[hidden_activation]
53     z, h = cache["z"], cache["h"]
54     grads = []
55     delta = (h[-1] - y) / len(y)
56     for index in range(len(params) - 1, -1, -1):
57         grads.append((h[index].T @ delta, np.sum(delta, axis=0, keepdims=True)))
58         if index:
59             delta = (delta @ params[index][0].T) * act_derivative(z[index - 1])
60     return tuple(reversed(grads))
61
62
63 def update(params, grads, learning_rate):
64     return tuple(
65         (W - learning_rate * dW, b - learning_rate * db)
66         for (W, b), (dW, db) in zip(params, grads)
67     )
68
69
70 demo = init_params((8, 16, 8, 1), 42)
71 demo_out, demo_cache = forward(demo, X_train[:5], "relu")
72 print("Kiểm tra kích thước ma trận trên 5 dòng đầu:")
73 for index, (W, b) in enumerate(demo, start=1):
74     print(f"W{index} {W.shape} b{index} {b.shape}")
75 for index, h in enumerate(demo_cache["h"][1:-1], start=1):
76     print(f"H{index}.shape = {h.shape}")
77 print(" y_hat.shape =", demo_out.shape)

```

Out

```

Kiểm tra kích thước ma trận trên 5 dòng đầu:
W1 (8, 16) b1 (1, 16)
W2 (16, 8) b2 (1, 8)
W3 (8, 1) b3 (1, 1)
H1.shape = (5, 16)
H2.shape = (5, 8)
y_hat.shape = (5, 1)

```

3. Chỉ số đánh giá

In [6]

```

1 EPS = 1e-8
2

```

```

3
4 def confusion(y_true, y_pred):
5     positive_true = y_true == 1
6     positive_pred = y_pred == 1
7     return {
8         "tp": int(np.sum(positive_pred & positive_true)),
9         "tn": int(np.sum(~positive_pred & ~positive_true)),
10        "fp": int(np.sum(positive_pred & ~positive_true)),
11        "fn": int(np.sum(~positive_pred & positive_true)),
12    }
13
14
15 def score(y_true, y_prob, threshold=0.5):
16     counts = confusion(y_true, (y_prob >= threshold).astype(int))
17     tp, tn, fp, fn = counts["tp"], counts["tn"], counts["fp"], counts["fn"]
18     precision = tp / (tp + fp + EPS)
19     recall = tp / (tp + fn + EPS)
20     return {
21         "threshold": threshold,
22         "accuracy": (tp + tn) / (tp + tn + fp + fn + EPS),
23         "precision": precision,
24         "recall": recall,
25         "f1": 2 * precision * recall / (precision + recall + EPS),
26         "balanced_accuracy": 0.5 * (recall + tn / (tn + fp + EPS)),
27         **counts,
28     }
29
30
31 def roc_auc(y_true, y_prob):
32     scores = y_prob.ravel()
33     order = np.argsort(scores, kind="mergesort")
34     ranks = np.empty(len(order), dtype=float)
35     ranks[order] = np.arange(1, len(order) + 1)
36     ordered = scores[order]
37     start = 0
38     for end in range(1, len(ordered) + 1):
39         if end == len(ordered) or ordered[end] != ordered[start]:
40             ranks[order[start:end]] = ranks[order[start:end]].mean()
41             start = end
42     labels = y_true.ravel()
43     n_pos = float(np.sum(labels == 1))
44     n_neg = float(len(labels) - n_pos)
45     return float((ranks[labels == 1].sum() - n_pos * (n_pos + 1) / 2) / (n_pos * n_neg))
46
47
48 print("Kiểm tra nhanh trên nhãn dựng sẵn:")
49 probe_y = np.array([[1], [0], [1], [0]], dtype=float)
50 probe_p = np.array([[0.9], [0.2], [0.4], [0.1]], dtype=float)
51 print(" ", {k: round(v, 4) if isinstance(v, float) else v
52             for k, v in score(probe_y, probe_p).items()})
53 print(" ROC-AUC =", round(roc_auc(probe_y, probe_p), 4), "(kỳ vọng 0.75)")

```

Out

Kiểm tra nhanh trên nhãn dựng sẵn:

```
{'threshold': 0.5, 'accuracy': 0.75, 'precision': 1.0, 'recall': 0.5, 'f1': 0.6667, 'balanced_accuracy': 0.75, 'tp': 1, 'tn': 2, 'fp': 0, 'fn': 1}
ROC-AUC = 1.0 (kỳ vọng 0.75)
```

4. Huấn luyện bốn mô hình

In [7]

```
1  LEARNING_RATE = 0.05
2  EPOCHS = 60
3  BATCH_SIZE = 512
4  INIT_SEED = 42
5  SHUFFLE_SEED = 7
6  THRESHOLDS = (0.3, 0.5, 0.7)
7
8  ARCHITECTURES = (
9      {"key": "M1", "label": "Baseline", "sizes": (8, 16, 8, 1), "activation": "relu"},
10     {"key": "M2", "label": "Wider", "sizes": (8, 32, 16, 1), "activation": "relu"},
11     {"key": "M3", "label": "Shallow", "sizes": (8, 16, 1), "activation": "relu"},
12     {"key": "M4", "label": "No ReLU", "sizes": (8, 16, 8, 1), "activation": "linear"},
13 )
14
15
16 def n_parameters(sizes):
17     return sum(a * b + b for a, b in zip(sizes, sizes[1:]))
18
19
20 def fit(arch, X_tr, y_tr, X_te, y_te, epochs, learning_rate, batch_size):
21     params = init_params(arch["sizes"], INIT_SEED)
22     rng = np.random.default_rng(SHUFFLE_SEED)
23     n = len(X_tr)
24     step = batch_size or n
25     history = []
26     started = time.perf_counter()
27
28     for _ in range(epochs):
29         order = rng.permutation(n) if batch_size else np.arange(n)
30         epoch_loss = None
31         for begin in range(0, n, step):
32             rows = order[begin:begin + step]
33             y_hat, cache = forward(params, X_tr[rows], arch["activation"])
34             if batch_size is None:
35                 epoch_loss = binary_cross_entropy(y_tr, y_hat)
36             grads = backward(params, y_tr[rows], cache, arch["activation"])
37             params = update(params, grads, learning_rate)
38         if batch_size is None:
39             history.append(epoch_loss)
40         else:
41             probe, _ = forward(params, X_tr, arch["activation"])
42             history.append(binary_cross_entropy(y_tr, probe))
43
44     elapsed = time.perf_counter() - started
45     train_prob, cache_tr = forward(params, X_tr, arch["activation"])
46     test_prob, _ = forward(params, X_te, arch["activation"])
47
48     def rounded(row):
49         return {k: (round(v, 6) if isinstance(v, float) else v) for k, v in row.items()}
50
51     return params, {
52         "key": arch["key"],
53         "label": arch["label"],
54         "shape": " - ".join(str(s) for s in arch["sizes"]),
55         "hidden_activation": arch["activation"],
56         "n_params": n_parameters(arch["sizes"]),
```

```

57     "epochs": epochs,
58     "batch_size": step,
59     "updates": epochs * int(np.ceil(n / step)),
60     "learning_rate": learning_rate,
61     "seconds": round(elapsed, 2),
62     "loss_first": round(history[0], 6),
63     "loss_final": round(history[-1], 6),
64     "loss_test_end": round(binary_cross_entropy(y_te, test_prob), 6),
65     "loss_history": [round(v, 6) for v in history],
66     "roc_auc_test": round(roc_auc(y_te, test_prob), 6),
67     "train": rounded(score(y_tr, train_prob, 0.5)),
68     "test_by_threshold": {str(t): rounded(score(y_te, test_prob, t)) for t in THRESHOLDS},
69     "hidden_shapes": [list(h.shape) for h in cache_tr["h"][1:-1]],
70 }

```

Out

M1 Baseline	8 - 16 - 8 - 1	loss 0.3704 acc 0.8378 prec 0.5940 rec 0.1941 f1 0.2926 auc 0.8042 (7.9s)
M2 Wider	8 - 32 - 16 - 1	loss 0.3699 acc 0.8372 prec 0.5934 rec 0.1844 f1 0.2814 auc 0.8043 (13.7s)
M3 Shallow	8 - 16 - 1	loss 0.3707 acc 0.8374 prec 0.5903 rec 0.1934 f1 0.2913 auc 0.8037 (5.8s)
M4 No ReLU	8 - 16 - 8 - 1	loss 0.3781 acc 0.8328 prec 0.5471 rec 0.1917 f1 0.2839 auc 0.7967 (6.7s)

In [8]

```

1  trained = {}
2  models = []
3  for arch in ARCHITECTURES:
4      params, row = fit(arch, X_train, y_train, X_test, y_test,
5                      EPOCHS, LEARNING_RATE, BATCH_SIZE)
6      trained[arch["key"]] = params
7      models.append(row)
8      t = row["test_by_threshold"]["0.5"]
9      print(f"{row['key']} {row['label']:9s} {row['shape']:18s} "
10           f"loss {row['loss_final']:.4f} acc {t['accuracy']:.4f} "
11           f"prec {t['precision']:.4f} rec {t['recall']:.4f} f1 {t['f1']:.4f} "
12           f"auc {row['roc_auc_test']:.4f} ({row['seconds']:.1f}s)")

```

Out

Kết quả trên tập test, ngưỡng 0.5

Mô hình	Kiến trúc	Kích hoạt	Tham số	Loss train	Loss test	Accuracy	Precision	Recall	F1	ROC-AUC
M1 Baseline	8 - 16 - 8 - 1	relu	289	0.370380	0.370187	0.8378	0.5940	0.1941	0.2926	0.8042
M2 Wider	8 - 32 - 16 - 1	relu	833	0.369850	0.370046	0.8372	0.5934	0.1844	0.2814	0.8043
M3 Shallow	8 - 16 - 1	relu	161	0.370749	0.370421	0.8374	0.5903	0.1934	0.2913	0.8037
M4 No ReLU	8 - 16 - 8 - 1	linear	289	0.378064	0.377577	0.8328	0.5471	0.1917	0.2839	0.7967

Ma trận nhầm lẫn tại ngưỡng 0.5

M1 Baseline	TP 1,542 FN 6,402 FP 1,054 TN 36,959
M2 Wider	TP 1,465 FN 6,479 FP 1,004 TN 37,009
M3 Shallow	TP 1,536 FN 6,408 FP 1,066 TN 36,947
M4 No ReLU	TP 1,523 FN 6,421 FP 1,261 TN 36,752

In [9]

```

1 table = pd.DataFrame([
2     {
3         "Mô hình": f"{m['key']} {m['label']}",
4         "Kiến trúc": m["shape"],
5         "Kích hoạt": m["hidden_activation"],
6         "Tham số": m["n_params"],
7         "Loss train": m["loss_final"],
8         "Loss test": m["loss_test_end"],
9         "Accuracy": m["test_by_threshold"]["0.5"]["accuracy"],
10        "Precision": m["test_by_threshold"]["0.5"]["precision"],
11        "Recall": m["test_by_threshold"]["0.5"]["recall"],
12        "F1": m["test_by_threshold"]["0.5"]["f1"],
13        "ROC-AUC": m["roc_auc_test"],
14        "Giây": m["seconds"],
15    }
16    for m in models
17 ])
18 print("Kết quả trên tập test, ngưỡng 0.5")
19 print(table.to_string(index=False))
20
21 print("\nMa trận nhầm lẫn tại ngưỡng 0.5")
22 for m in models:
23     t = m["test_by_threshold"]["0.5"]
24     print(f" {m['key']} {m['label']:9s} TP {t['tp']:6,} FN {t['fn']:6,} "
25           f"FP {t['fp']:6,} TN {t['tn']:6,}")
26
27 print("\nẢnh hưởng của ngưỡng quyết định")
28 for m in models:
29     parts = []
30     for th in THRESHOLDS:
31         t = m["test_by_threshold"][str(th)]
32         parts.append(f"{th}: prec {t['precision']:.4f} rec {t['recall']:.4f} f1 {t['f1']:.4f}")
33     print(f" {m['key']} {m['label']:9s} " + " | ".join(parts))

```

In [10]

```

1 fig, axes = plt.subplots(1, 2, figsize=(10.0, 3.4))
2 styles = ["-", "--", "-.", ":"]
3
4 for axis, start in zip(axes, (0, 10)):
5     for m, style in zip(models, styles):
6         history = m["loss_history"][start:]
7         axis.plot(range(start, start + len(history)), history, style, color="black",
8                   linewidth=1.2, label=f"{m['key']} {m['label']}")
9         axis.set_xlabel("Epoch")
10        axis.set_ylabel("Binary cross-entropy (train)")
11        axis.grid(True, linewidth=0.3, alpha=0.4)
12 axes[0].set_title("Toàn bộ quá trình huấn luyện")
13 axes[1].set_title("Phóng to từ epoch 10")
14 axes[0].legend(frameon=False, fontsize=8)
15 fig.tight_layout()
16 plt.show()

```

In [11]

```

1 fig, axes = plt.subplots(1, 2, figsize=(10.0, 3.4))
2 markers = ["o", "s", "^", "D"]
3
4 for m, marker in zip(models, markers):

```

```

5     rows = [m["test_by_threshold"][str(t)] for t in THRESHOLDS]
6     axes[0].plot([r["recall"] for r in rows], [r["precision"] for r in rows], marker + "-",
7                  color="black", markerfacecolor="white", linewidth=1.0,
8                  label=f"{m['key']} {m['label']}")
9     for threshold, r in zip(THRESHOLDS, rows):
10        axes[0].annotate(f"{threshold}", (r["recall"], r["precision"]), fontsize=7,
11                        xytext=(4, 4), textcoords="offset points")
12 axes[0].set_xlabel("Recall")
13 axes[0].set_ylabel("Precision")
14 axes[0].set_title("Đánh đổi khi đổi ngưỡng quyết định")
15 axes[0].legend(frameon=False, fontsize=8)
16 axes[0].grid(True, linewidth=0.3, alpha=0.4)
17
18 positions = np.arange(len(models))
19 width = 0.26
20 for offset, (metric, hatch) in enumerate(zip(("accuracy", "f1"), ("", "///"))):
21     values = [m["test_by_threshold"]["0.5"][metric] for m in models]
22     axes[1].bar(positions + offset * width, values, width, color="white",
23                edgecolor="black", hatch=hatch, label=metric)
24 axes[1].bar(positions + 2 * width, [m["roc_auc_test"] for m in models], width,
25             color="white", edgecolor="black", hatch="...", label="roc_auc")
26 axes[1].set_xticks(positions + width)
27 axes[1].set_xticklabels([f"{m['key']}\n{m['label']}" for m in models], fontsize=8)
28 axes[1].set_ylim(0, 1.16)
29 axes[1].set_title("So sánh bốn mô hình trên tập test")
30 axes[1].legend(frameon=False, fontsize=7, ncol=3, loc="upper center")
31 axes[1].grid(True, axis="y", linewidth=0.3, alpha=0.4)
32
33 fig.tight_layout()
34 plt.show()

```

Out

```

lr 0.001  loss 0.5284 -> 0.4431 acc 0.8271 rec 0.0000 f1 0.0000 auc 0.6920 (232.0s)
lr 0.01   loss 0.5284 -> 0.3904 acc 0.8271 rec 0.0000 f1 0.0000 auc 0.7746 (244.3s)
lr 0.1    loss 0.5284 -> 0.3757 acc 0.8312 rec 0.1058 f1 0.1788 auc 0.8004 (199.2s)

Đối chiếu: M1 lô nhỏ 512, 60 epoch
lr 0.05   loss 0.3848 -> 0.3704 acc 0.8378 rec 0.1941 f1 0.2926 auc 0.8042 (7.9s, 21,540 lần cập nhật)

```

5. Học suất và công thức gốc của bài giảng (Full-batch GD 1000 epochs)

In [12]

```

1 SPEC_EPOCHS = 1000
2 SPEC_RATES = (0.001, 0.01, 0.1)
3
4 lr_study = []
5 lr_curves = {}
6 for rate in SPEC_RATES:
7     _, row = fit(ARCHITECTURES[0], X_train, y_train, X_test, y_test, SPEC_EPOCHS, rate, None)
8     t = row["test_by_threshold"]["0.5"]
9     lr_curves[rate] = row["loss_history"]
10    lr_study.append({
11        "learning_rate": rate,
12        "epochs": SPEC_EPOCHS,
13        "batch_size": row["batch_size"],
14        "updates": row["updates"],
15        "loss_first": row["loss_first"],

```

```

16     "loss_final": row["loss_final"],
17     "accuracy": t["accuracy"],
18     "precision": t["precision"],
19     "recall": t["recall"],
20     "f1": t["f1"],
21     "roc_auc_test": row["roc_auc_test"],
22     "seconds": row["seconds"],
23 })
24 print(f"lr {rate:<6} loss {row['loss_first']:.4f} -> {row['loss_final']:.4f} "
25       f"acc {t['accuracy']:.4f} rec {t['recall']:.4f} f1 {t['f1']:.4f} "
26       f"auc {row['roc_auc_test']:.4f} ({row['seconds']:.1f}s)")
27
28 m1 = models[0]
29 t1 = m1["test_by_threshold"]["0.5"]
30 print("\nĐối chiếu: M1 lô nhỏ 512, 60 epoch")
31 print(f"lr {m1['learning_rate']:<6} loss {m1['loss_first']:.4f} -> {m1['loss_final']:.4f} "
32       f"acc {t1['accuracy']:.4f} rec {t1['recall']:.4f} f1 {t1['f1']:.4f} "
33       f"auc {m1['roc_auc_test']:.4f} ({m1['seconds']:.1f}s, {m1['updates']:,} lần cập nhật)")

```

In [13]

```

1 fig, axis = plt.subplots(figsize=(6.6, 3.4))
2 for rate, style in zip(SPEC_RATES, ["-", "--", "-."]):
3     axis.plot(lr_curves[rate], style, color="black", linewidth=1.2,
4              label=f"tòa tập, lr = {rate}")
5 axis.axhline(models[0]["loss_final"], color="black", linewidth=0.8, alpha=0.45)
6 axis.annotate(f"M1 lô nhỏ đạt {models[0]['loss_final']:.4f}",
7              (SPEC_EPOCHS * 0.40, models[0]["loss_final"]), fontsize=8,
8              xytext=(0, 6), textcoords="offset points")
9 axis.set_xlabel("Epoch")
10 axis.set_ylabel("Binary cross-entropy (train)")
11 axis.set_title("Gradient descent toàn tập theo công thức gốc")
12 axis.legend(frameon=False, fontsize=8)
13 axis.grid(True, linewidth=0.3, alpha=0.4)
14 fig.tight_layout()
15 plt.show()

```

Out

đã ghi reports/deeplearning_results.json (13.2 KB)
số mô hình: 4
mô hình tốt nhất theo ROC-AUC: M2

6. Ghi kết quả ra file

In [14]

```

1 payload = {
2     "data_file": DIABETES_CSV.relative_to(ROOT).as_posix(),
3     "features": list(FEATURES),
4     "feature_ranking": {k: round(float(v), 6) for k, v in ranking.items()},
5     "rows_raw": int(len(raw)),
6     "rows_after_dedup": int(len(frame)),
7     "n_train": int(len(X_train)),
8     "n_test": int(len(X_test)),
9     "positive_rate_train": round(float(y_train.mean()), 6),
10    "positive_rate_test": round(float(y_test.mean()), 6),
11    "split_seed": SPLIT_SEED,
12    "init_seed": INIT_SEED,

```

```

13     "shuffle_seed": SHUFFLE_SEED,
14     "learning_rate": LEARNING_RATE,
15     "epochs": EPOCHS,
16     "batch_size": BATCH_SIZE,
17     "thresholds": list(THRESHOLDS),
18     "models": models,
19     "learning_rate_study": lr_study,
20 }
21
22 out = REPORTS / "deeplearning_results.json"
23 out.write_text(json.dumps(payload, indent=2, ensure_ascii=False), encoding="utf-8")
24 print("đã ghi", out.relative_to(ROOT).as_posix(), f"({out.stat().st_size / 1024:.1f} KB)")
25 print("số mô hình:", len(payload["models"]))
26 print("mô hình tốt nhất theo ROC-AUC:",
27       max(payload["models"], key=lambda m: m["roc_auc_test"])["key"])

```

Phụ lục B. Notebook 05: K-Nearest Neighbors

Toàn bộ nội dung notebook, giữ nguyên thứ tự cell. Khối In [n] là mã nguồn thực thi, khối Out là kết quả chạy thật trên máy cục bộ. Mọi số liệu trong phần thân báo cáo đều được tham chiếu đồng nhất từ các thí nghiệm này.

05_knn.ipynb · 25 cell · 17 code

Notebook 05 - K-Nearest Neighbors trên ba category

Notebook này bổ sung một họ thuật toán chưa từng xuất hiện trong notebook 01, 02, 03: học dựa trên thực thể (instance based learning).

In [1]

```
1 import sys
2 import time
3 import warnings
4 from collections import Counter
5 from pathlib import Path
6
7 import numpy as np
8 import pandas as pd
9 from sklearn.neighbors import (
10     KNeighborsClassifier,
11     KNeighborsRegressor,
12     NearestNeighbors,
13 )
14
15 ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
16 if str(ROOT) not in sys.path:
17     sys.path.insert(0, str(ROOT))
18
19 import ml
20
21 warnings.filterwarnings("ignore")
22 pd.set_option("display.max_columns", 30)
23 pd.set_option("display.width", 140)
24
25 print("thư mục gốc :", ROOT)
26 print("numpy :", np.__version__)
27 print("pandas :", pd.__version__)
28 print("seed chung :", ml.RANDOM_SEED)
```

2. Nạp bốn bài toán

In [2]

```
1 t0 = time.perf_counter()
2 TASKS = {key: loader() for key, loader in ml.LOADERS.items()}
3 print(f"Nạp bốn bài toán trong {time.perf_counter() - t0:.1f}s\n")
4
```

```

5 for task in TASKS.values():
6     print(task.summary())

```

In [3]

```

1 pd.DataFrame([
2     {
3         "bài toán": task.title,
4         "kiểu": task.kind,
5         "train": f"{task.n_train:,}",
6         "val": f"{task.n_val:,}",
7         "test": f"{task.n_test:,}",
8         "mốc so sánh": task.baseline_name,
9     }
10     for task in TASKS.values()
11 ])

```

3. Chọn k bằng một lần tìm hàng xóm duy nhất

In [4]

```

1 def sweep_k(task, rep, k_grid, metric="euclidean"):
2     '''Quét lưới k bằng MỘT lần tìm hàng xóm, trả về bảng kết quả trên tập validation.'''
3     prep = task.prep(rep)
4     Z_train = prep.fit_transform(task.X_train, task.y_train)
5     Z_val = prep.transform(task.X_val)
6     y_train = np.asarray(task.y_train)
7
8     searcher = NearestNeighbors(
9         n_neighbors=max(k_grid), metric=metric, algorithm="brute", n_jobs=-1
10    ).fit(Z_train)
11
12    t0 = time.perf_counter()
13    _, index = searcher.kneighbors(Z_val)
14    search_seconds = time.perf_counter() - t0
15    neighbour_labels = y_train[index]
16
17    rows = []
18    for k in k_grid:
19        window = neighbour_labels[:, :k]
20        if task.kind == "regression":
21            prediction = window.mean(axis=1)
22            m = ml.regression_metrics(task.y_val, prediction)
23            rows.append({
24                "k": k, "R2_log": f"{m['r2_log']:.4f}",
25                "MAE_USD": f"{m['mae_usd']:.0f}", "MAPE_%": f"{m['mape_percent']:.1f}",
26                "_score": m["r2_log"],
27            })
28        elif task.kind == "binary":
29            proba = window.mean(axis=1)
30            m = ml.classification_metrics(task.y_val, (proba >= 0.5).astype(int), proba)
31            rows.append({
32                "k": k, "accuracy": f"{m['accuracy']:.4f}", "recall": f"{m['recall']:.4f}",
33                "f1": f"{m['f1']:.4f}", "f1_macro": f"{m['f1_macro']:.4f}",
34                "roc_auc": f"{m['roc_auc']:.4f}", "_score": m["f1_macro"],
35            })
36        else:
37            prediction = np.array([Counter(row).most_common(1)[0][0] for row in window])
38            m = ml.classification_metrics(task.y_val, prediction, average="macro")

```

```

39         rows.append({
40             "k": k, "accuracy": f"{m['accuracy']:.4f}",
41             "f1_macro": f"{m['f1_macro']:.4f}", "f1_weighted": f"{m['f1_weighted']:.4f}",
42             "_score": m["f1_macro"],
43         })
44
45     best = max(rows, key=lambda r: r["_score"])
46     print(f"Một lần tìm {max(k_grid)} hàng xóm cho {Z_val.shape[0]:,} điểm validation: "
47           f"{search_seconds:.1f}s (d = {Z_train.shape[1]:,}, metric = {metric})")
48     return rows, int(best["k"])
49
50
51 def show(rows, best_k):
52     frame = pd.DataFrame(rows).drop(columns="_score")
53     print(frame.to_string(index=False))
54     print(f"\n-> k tốt nhất trên validation: {best_k}")

```

4. Bài toán 1 - Tiểu đường (phân loại nhị phân)

In [5]

```

1 task = TASKS["diabetes"]
2 rows_dia, k_dia = sweep_k(task, "native", [5, 11, 25, 51, 101])
3 show(rows_dia, k_dia)

```

In [6]

```

1 results_dia = [
2     ml.run_model(task, f"KNN (k={k_dia})", "native",
3                   KNeighborsClassifier(n_neighbors=k_dia, algorithm="brute", n_jobs=-1)),
4     ml.run_model(task, task.baseline_name + " (mốc)", task.baseline_rep,
5                   task.baseline_estimator()),
6 ]
7 ml.print_table([ml.table_row(task, r) for r in results_dia])

```

In [7]

```

1 test_dia = {r["name"]: ml.evaluate_on_test(task, r["pipe"]) for r in results_dia}
2 for name, m in test_dia.items():
3     print(f"TEST {name:<34} accuracy={m['accuracy']:.4f} recall={m['recall']:.4f} "
4           f"f1={m['f1']:.4f} f1_macro={m['f1_macro']:.4f} auc={m['roc_auc']:.4f}")

```

5. Bài toán 2 - Giá nhà (hồi quy)

In [8]

```

1 task = TASKS["house_price"]
2 rows_house, k_house = sweep_k(task, "target_enc_scaled", [5, 11, 25, 51])
3 show(rows_house, k_house)

```

In [9]

```

1 results_house = [
2     ml.run_model(task, f"KNN (k={k_house})", "target_enc_scaled",
3                   KNeighborsRegressor(n_neighbors=k_house, algorithm="brute", n_jobs=-1)),
4     ml.run_model(task, task.baseline_name + " (mốc)", task.baseline_rep,
5                   task.baseline_estimator()),

```

```

6 ]
7 ml.print_table([ml.table_row(task, r) for r in results_house])

```

In [10]

```

1 test_house = {r["name"]: ml.evaluate_on_test(task, r["pipe"]) for r in results_house}
2 for name, m in test_house.items():
3     print(f"TEST {name:<34} R2_log={m['r2_log']:.4f} MAE={m['mae_usd']:, .0f} USD "
4           f"MAPE={m['mape_percent']:.1f}%")

```

6. Bài toán 3 - E-commerce, mức hài lòng (bảng + TF-IDF)

In [11]

```

1 task = TASKS["ecommerce_satisfaction"]
2 rows_sat, k_sat = sweep_k(task, "tabular_text", [5, 11, 25, 51], metric="cosine")
3 show(rows_sat, k_sat)

```

In [12]

```

1 results_sat = [
2     ml.run_model(task, f"KNN cosin (k={k_sat})", "tabular_text",
3                   KNeighborsClassifier(n_neighbors=k_sat, metric="cosine",
4                                       algorithm="brute", n_jobs=-1)),
5     ml.run_model(task, task.baseline_name + " (mốc)", task.baseline_rep,
6                   task.baseline_estimator()),
7 ]
8 ml.print_table([ml.table_row(task, r) for r in results_sat])

```

In [13]

```

1 test_sat = {r["name"]: ml.evaluate_on_test(task, r["pipe"]) for r in results_sat}
2 for name, m in test_sat.items():
3     print(f"TEST {name:<34} accuracy={m['accuracy']:.4f} recall={m['recall']:.4f} "
4           f"f1={m['f1']:.4f} f1_macro={m['f1_macro']:.4f} auc={m['roc_auc']:.4f}")

```

7. Bài toán 4 - E-commerce, nhóm sản phẩm quan tâm (9 lớp)

In [14]

```

1 task = TASKS["ecommerce_interest"]
2 print("Phân bố lớp trên tập train:")
3 print(pd.Series(task.y_train).value_counts().to_string())
4 print()
5 rows_int, k_int = sweep_k(task, "tabular_text", [5, 11, 25, 51], metric="cosine")
6 show(rows_int, k_int)

```

In [15]

```

1 results_int = [
2     ml.run_model(task, f"KNN cosin (k={k_int})", "tabular_text",
3                   KNeighborsClassifier(n_neighbors=k_int, metric="cosine",
4                                       algorithm="brute", n_jobs=-1)),
5     ml.run_model(task, task.baseline_name + " (mốc)", task.baseline_rep,
6                   task.baseline_estimator()),
7 ]
8 ml.print_table([ml.table_row(task, r) for r in results_int])

```

In [16]

```
1 test_int = {r["name"]: ml.evaluate_on_test(task, r["pipe"]) for r in results_int}
2 for name, m in test_int.items():
3     print(f"TEST {name:<34} accuracy={m['accuracy']:.4f} f1_macro={m['f1_macro']:.4f} "
4           f"f1_weighted={m['f1_weighted']:.4f}")
```

8. Tổng hợp và ghi kết quả

In [17]

```
1 def block(task, results, test_metrics, sweep_rows, best_k, metric):
2     return {
3         "task": task.key,
4         "title": task.title,
5         "kind": task.kind,
6         "n_train": task.n_train,
7         "n_val": task.n_val,
8         "n_test": task.n_test,
9         "audit": task.audit,
10        "distance": metric,
11        "k_grid": [int(r["k"]) for r in sweep_rows],
12        "k_selected": best_k,
13        "sweep": [{k: v for k, v in r.items() if k != "_score"} for r in sweep_rows],
14        "validation_table": [ml.table_row(task, r) for r in results],
15        "test_metrics": {name: m for name, m in test_metrics.items()},
16        "model_names": [r["name"] for r in results],
17    }
18
19
20 payload = {
21     "algorithm": "K-Nearest Neighbors",
22     "library": "scikit-learn",
23     "seed": ml.RANDOM_SEED,
24     "note": (
25         "Mô hình mốc được huấn luyện lại trên bộ dữ liệu hiện tại trong data/, "
26         "không chép lại số của notebook 01-03."
27     ),
28     "tasks": [
29         block(TASKS["diabetes"], results_dia, test_dia, rows_dia, k_dia, "euclidean"),
30         block(TASKS["house_price"], results_house, test_house, rows_house, k_house, "euclidean"),
31         block(TASKS["ecommerce_satisfaction"], results_sat, test_sat, rows_sat, k_sat, "cosine"),
32         block(TASKS["ecommerce_interest"], results_int, test_int, rows_int, k_int, "cosine"),
33     ],
34 }
35
36 path = ml.save_results("knn", payload)
37 print("Đã ghi", path.relative_to(ROOT), f"({path.stat().st_size / 1024:.1f} KB)")
```

Phụ lục C. Notebook 06: AdaBoost

Toàn bộ nội dung notebook, giữ nguyên thứ tự cell. Khối In [n] là mã nguồn thực thi, khối Out là kết quả chạy thật trên máy cục bộ. Mọi số liệu trong phần thân báo cáo đều được tham chiếu đồng nhất từ các thí nghiệm này.

06_adaboost.ipynb · 24 cell · 16 code

Notebook 06 - AdaBoost trên ba category

Notebook này bổ sung AdaBoost để so sánh boosting theo trọng số mẫu với gradient boosting.

In [1]

```
1 import sys
2 import time
3 import warnings
4 from pathlib import Path
5
6 import numpy as np
7 import pandas as pd
8 from sklearn.ensemble import AdaBoostClassifier, AdaBoostRegressor
9 from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
10 from sklearn.utils.class_weight import compute_sample_weight
11
12 ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
13 if str(ROOT) not in sys.path:
14     sys.path.insert(0, str(ROOT))
15
16 import ml
17
18 warnings.filterwarnings("ignore")
19 pd.set_option("display.max_columns", 30)
20 pd.set_option("display.width", 140)
21
22 print("thư mục gốc :", ROOT)
23 print("numpy :", np.__version__)
24 print("pandas :", pd.__version__)
25 print("seed chung :", ml.RANDOM_SEED)
```

2. Nạp bốn bài toán

In [2]

```
1 t0 = time.perf_counter()
2 TASKS = {key: loader() for key, loader in ml.LOADERS.items()}
3 print(f"Nạp bốn bài toán trong {time.perf_counter() - t0:.1f}s\n")
4 for task in TASKS.values():
5     print(task.summary())
```

3. Quét số vòng boosting bằng một lần huấn luyện duy nhất

In [3]

```

1 def sweep_stages(task, rep, estimator, stage_grid, sample_weight=None):
2     '''Huấn luyện MỘT lần với số vòng lớn nhất, đọc kết quả tại từng mốc trong lưới.'''
3     prep = task.prep(rep)
4     Z_train = prep.fit_transform(task.X_train, task.y_train)
5     Z_val = prep.transform(task.X_val)
6
7     t0 = time.perf_counter()
8     estimator.fit(Z_train, task.y_train, sample_weight=sample_weight)
9     fit_seconds = time.perf_counter() - t0
10    n_fitted = len(estimator.estimators_)
11
12    wanted = {s for s in stage_grid if s <= n_fitted}
13    rows = []
14
15    t0 = time.perf_counter()
16    if task.kind == "regression":
17        for stage, prediction in enumerate(estimator.staged_predict(Z_val), start=1):
18            if stage not in wanted:
19                continue
20            m = ml.regression_metrics(task.y_val, prediction)
21            rows.append({
22                "vòng": stage, "R2_log": f"{m['r2_log']:.4f}",
23                "MAE_USD": f"{m['mae_usd']:.0f}", "MAPE_%": f"{m['mape_percent']:.1f}",
24                "_score": m["r2_log"],
25            })
26    else:
27        classes = estimator.classes_
28        for stage, proba in enumerate(estimator.staged_predict_proba(Z_val), start=1):
29            if stage not in wanted:
30                continue
31            prediction = classes[proba.argmax(axis=1)]
32            score = proba[:, 1] if task.average == "binary" else None
33            m = ml.classification_metrics(
34                task.y_val, prediction, score, average=task.average
35            )
36            row = {
37                "vòng": stage, "accuracy": f"{m['accuracy']:.4f}",
38                "recall": f"{m['recall']:.4f}", "f1_macro": f"{m['f1_macro']:.4f}",
39                "_score": m["f1_macro"],
40            }
41            if m["roc_auc"] is not None:
42                row["roc_auc"] = f"{m['roc_auc']:.4f}"
43            rows.append(row)
44    stage_seconds = time.perf_counter() - t0
45
46    best = max(rows, key=lambda r: r["_score"])
47    print(f"Huấn luyện {n_fitted} vòng trong {fit_seconds:.1f}s, "
48          f"đọc {len(rows)} mốc trong {stage_seconds:.1f}s (d = {Z_train.shape[1]:,})")
49    print(pd.DataFrame(rows).drop(columns="_score").to_string(index=False))
50    print(f"\n-> số vòng tốt nhất trên validation: {best['vòng']}")
51    return rows, int(best["vòng"])

```

4. Bài toán 1 - Tiêu đường (phân loại nhị phân)

In [4]

```

1 task = TASKS["diabetes"]
2 ada_dia = AdaBoostClassifier(

```

```

3     estimator=DecisionTreeClassifier(max_depth=3, random_state=ml.RANDOM_SEED),
4     n_estimators=300, learning_rate=0.5, random_state=ml.RANDOM_SEED,
5 )
6 weight_dia = compute_sample_weight("balanced", task.y_train)
7 rows_dia, m_dia = sweep_stages(task, "native", ada_dia, [25, 50, 100, 200, 300], weight_dia)

```

In [5]

```

1 results_dia = [
2     ml.run_model(task, f"AdaBoost ({m_dia} vòng, cây sâu 3)", "native",
3         AdaBoostClassifier(
4             estimator=DecisionTreeClassifier(max_depth=3, random_state=ml.RANDOM_SEED),
5             n_estimators=m_dia, learning_rate=0.5, random_state=ml.RANDOM_SEED),
6             sample_weight=weight_dia),
7     ml.run_model(task, task.baseline_name + " (mốc)", task.baseline_rep,
8         task.baseline_estimator()),
9 ]
10 ml.print_table([ml.table_row(task, r) for r in results_dia])

```

In [6]

```

1 test_dia = {r["name"]: ml.evaluate_on_test(task, r["pipe"]) for r in results_dia}
2 for name, m in test_dia.items():
3     print(f"TEST {name:<36} accuracy={m['accuracy']:.4f} recall={m['recall']:.4f} "
4           f"f1={m['f1']:.4f} f1_macro={m['f1_macro']:.4f} auc={m['roc_auc']:.4f}")

```

5. Bài toán 2 - Giá nhà (hồi quy, AdaBoost.R2)

In [7]

```

1 task = TASKS["house_price"]
2 ada_house = AdaBoostRegressor(
3     estimator=DecisionTreeRegressor(max_depth=8, random_state=ml.RANDOM_SEED),
4     n_estimators=100, learning_rate=0.5, random_state=ml.RANDOM_SEED,
5 )
6 rows_house, m_house = sweep_stages(task, "target_enc", ada_house, [10, 25, 50, 75, 100])

```

In [8]

```

1 results_house = [
2     ml.run_model(task, f"AdaBoost.R2 ({m_house} vòng, cây sâu 8)", "target_enc",
3         AdaBoostRegressor(
4             estimator=DecisionTreeRegressor(max_depth=8, random_state=ml.RANDOM_SEED),
5             n_estimators=m_house, learning_rate=0.5, random_state=ml.RANDOM_SEED)),
6     ml.run_model(task, task.baseline_name + " (mốc)", task.baseline_rep,
7         task.baseline_estimator()),
8 ]
9 ml.print_table([ml.table_row(task, r) for r in results_house])

```

In [9]

```

1 test_house = {r["name"]: ml.evaluate_on_test(task, r["pipe"]) for r in results_house}
2 for name, m in test_house.items():
3     print(f"TEST {name:<36} R2_log={m['r2_log']:.4f} MAE={m['mae_usd']:.0f} USD "
4           f"fMAPE={m['mape_percent']:.1f}%")

```

6. Bài toán 3 - E-commerce, mức hài lòng (bảng + TF-IDF)

In [10]

```
1 task = TASKS["ecommerce_satisfaction"]
2 ada_sat = AdaBoostClassifier(
3     estimator=DecisionTreeClassifier(max_depth=3, random_state=ml.RANDOM_SEED),
4     n_estimators=300, learning_rate=0.5, random_state=ml.RANDOM_SEED,
5 )
6 weight_sat = compute_sample_weight("balanced", task.y_train)
7 rows_sat, m_sat = sweep_stages(task, "tabular_text", ada_sat, [25, 50, 100, 200, 300], weight_sat)
8
```

In [11]

```
1 results_sat = [
2     ml.run_model(task, f"AdaBoost ({m_sat} vòng, cây sâu 3)", "tabular_text",
3         AdaBoostClassifier(
4             estimator=DecisionTreeClassifier(max_depth=3, random_state=ml.RANDOM_SEED),
5             n_estimators=m_sat, learning_rate=0.5, random_state=ml.RANDOM_SEED),
6             sample_weight=weight_sat),
7     ml.run_model(task, task.baseline_name + " (mốc)", task.baseline_rep,
8         task.baseline_estimator()),
9 ]
10 ml.print_table([ml.table_row(task, r) for r in results_sat])
```

In [12]

```
1 test_sat = {r["name"]: ml.evaluate_on_test(task, r["pipe"]) for r in results_sat}
2 for name, m in test_sat.items():
3     print(f"TEST {name:<36} accuracy={m['accuracy']:.4f} recall={m['recall']:.4f} "
4           f"f1={m['f1']:.4f} f1_macro={m['f1_macro']:.4f} auc={m['roc_auc']:.4f}")
```

7. Bài toán 4 - E-commerce, nhóm sản phẩm quan tâm (9 lớp)

In [13]

```
1 task = TASKS["ecommerce_interest"]
2 ada_int = AdaBoostClassifier(
3     estimator=DecisionTreeClassifier(max_depth=3, random_state=ml.RANDOM_SEED),
4     n_estimators=300, learning_rate=0.5, random_state=ml.RANDOM_SEED,
5 )
6 weight_int = compute_sample_weight("balanced", task.y_train)
7 rows_int, m_int = sweep_stages(task, "tabular_text", ada_int, [25, 50, 100, 200, 300], weight_int)
8
```

In [14]

```
1 results_int = [
2     ml.run_model(task, f"AdaBoost ({m_int} vòng, cây sâu 3)", "tabular_text",
3         AdaBoostClassifier(
4             estimator=DecisionTreeClassifier(max_depth=3, random_state=ml.RANDOM_SEED),
5             n_estimators=m_int, learning_rate=0.5, random_state=ml.RANDOM_SEED),
6             sample_weight=weight_int),
7     ml.run_model(task, task.baseline_name + " (mốc)", task.baseline_rep,
8         task.baseline_estimator()),
9 ]
10 ml.print_table([ml.table_row(task, r) for r in results_int])
```

In [15]

```
1 test_int = {r["name"]: ml.evaluate_on_test(task, r["pipe"]) for r in results_int}
2 for name, m in test_int.items():
3     print(f"TEST {name:<36} accuracy={m['accuracy']:.4f} f1_macro={m['f1_macro']:.4f} "
4           f"f1_weighted={m['f1_weighted']:.4f}")
```

8. Tổng hợp và ghi kết quả

In [16]

```
1 def block(task, results, test_metrics, sweep_rows, best_stage, weak_learner):
2     return {
3         "task": task.key,
4         "title": task.title,
5         "kind": task.kind,
6         "n_train": task.n_train,
7         "n_val": task.n_val,
8         "n_test": task.n_test,
9         "audit": task.audit,
10        "weak_learner": weak_learner,
11        "stage_grid": [int(r["vòng"]) for r in sweep_rows],
12        "stage_selected": best_stage,
13        "sweep": [{k: v for k, v in r.items() if k != "_score"} for r in sweep_rows],
14        "validation_table": [ml.table_row(task, r) for r in results],
15        "test_metrics": dict(test_metrics),
16        "model_names": [r["name"] for r in results],
17    }
18
19
20 payload = {
21     "algorithm": "AdaBoost (SAMME cho phân loại, AdaBoost.R2 cho hồi quy)",
22     "library": "scikit-learn",
23     "seed": ml.RANDOM_SEED,
24     "learning_rate": 0.5,
25     "note": (
26         "Mô hình mốc được huấn luyện lại trên bộ dữ liệu hiện tại trong data/, "
27         "không chép lại số của notebook 01-03."
28     ),
29     "tasks": [
30         block(TASKS["diabetes"], results_dia, test_dia, rows_dia, m_dia,
31              "DecisionTreeClassifier(max_depth=3) + sample_weight cân bằng lớp"),
32         block(TASKS["house_price"], results_house, test_house, rows_house, m_house,
33              "DecisionTreeRegressor(max_depth=8)"),
34         block(TASKS["ecommerce_satisfaction"], results_sat, test_sat, rows_sat, m_sat,
35              "DecisionTreeClassifier(max_depth=3) + sample_weight cân bằng lớp"),
36         block(TASKS["ecommerce_interest"], results_int, test_int, rows_int, m_int,
37              "DecisionTreeClassifier(max_depth=3) + sample_weight cân bằng lớp"),
38     ],
39 }
40
41 path = ml.save_results("adaboost", payload)
42 print("Đã ghi", path.relative_to(ROOT), f"({path.stat().st_size / 1024:.1f} KB)")
```

Phụ lục D. Notebook 07: Mạng nơ-ron hồi quy cho giá nhà

Toàn bộ nội dung notebook, giữ nguyên thứ tự cell. Khối In [n] là mã nguồn thực thi, khối Out là kết quả chạy thật trên máy cục bộ. Mọi số liệu trong phần thân báo cáo đều được tham chiếu đồng nhất từ các thí nghiệm này.

07_deep_learning_house.ipynb · 19 cell · 12 code

Notebook 07 - Mạng nơ-ron hồi quy viết tay cho bài toán giá nhà

Toàn bộ mạng vẫn viết tay: lan truyền xuôi, hàm mất mát, lan truyền ngược theo quy tắc chuỗi và gradient descent đều là NumPy thuần.

In [1]

```
1 import sys
2 import time
3 import warnings
4 from dataclasses import dataclass
5 from pathlib import Path
6
7 import matplotlib
8 import matplotlib.pyplot as plt
9 import numpy as np
10 import pandas as pd
11
12 ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
13 if str(ROOT) not in sys.path:
14     sys.path.insert(0, str(ROOT))
15
16 import ml
17
18 warnings.filterwarnings("ignore")
19 matplotlib.rcParams.update({
20     "font.family": "serif",
21     "font.size": 9,
22     "axes.edgecolor": "black",
23     "axes.labelcolor": "black",
24     "text.color": "black",
25     "xtick.color": "black",
26     "ytick.color": "black",
27     "figure.facecolor": "white",
28     "savefig.facecolor": "white",
29 })
30
31 print("thư mục gốc :", ROOT)
32 print("numpy :", np.__version__)
```

1. Dữ liệu và biểu diễn đầu vào

In [2]

```
1 task = ml.load_house_price()
2 prep = task.prep("target_enc_scaled")
```

```

3
4 X_train = prep.fit_transform(task.X_train, task.y_train)
5 X_val = prep.transform(task.X_val)
6 X_test = prep.transform(task.X_test)
7
8 print(f"X_train {X_train.shape} X_val {X_val.shape} X_test {X_test.shape}")
9 print(f"trung bình mỗi chiều sau chuẩn hoá: {np.round(X_train.mean(axis=0), 3)}")
10 print(f"độ lệch chuẩn mỗi chiều : {np.round(X_train.std(axis=0), 3)}")

```

In [3]

```

1 y_mean = float(task.y_train.mean())
2 y_std = float(task.y_train.std())
3
4 z_train = ((task.y_train - y_mean) / y_std).reshape(-1, 1)
5 z_val = ((task.y_val - y_mean) / y_std).reshape(-1, 1)
6
7 def to_log_price(z):
8     '''Quy dự đoán đã chuẩn hoá về lại log(price) để tính chỉ số nghiệp vụ.'''
9     return z.ravel() * y_std + y_mean
10
11 print(f"log(price): trung bình = {y_mean:.4f}, độ lệch chuẩn = {y_std:.4f}")
12 print(f"sau chuẩn hoá: trung bình = {z_train.mean():.6f}, độ lệch chuẩn = {z_train.std():.6f}")

```

2. Mạng nơ-ron viết tay

In [4]

```

1 def relu(x):
2     return np.maximum(0.0, x)
3
4
5 def relu_derivative(x):
6     return (x > 0).astype(float)
7
8
9 def identity(x):
10    return x
11
12
13 def identity_derivative(x):
14    return np.ones_like(x)
15
16
17 ACTIVATIONS = {"relu": (relu, relu_derivative), "linear": (identity, identity_derivative)}
18
19
20 @dataclass(frozen=True)
21 class Architecture:
22     key: str
23     label: str
24     sizes: tuple
25     hidden_activation: str = "relu"
26
27     @property
28     def shape(self):
29         return " - ".join(str(s) for s in self.sizes)
30
31     @property

```

```

32     def n_params(self):
33         return sum(a * b + b for a, b in zip(self.sizes, self.sizes[1:]))
34
35
36     def init_params(arch, seed):
37         rng = np.random.default_rng(seed)
38         layers = []
39         for fan_in, fan_out in zip(arch.sizes, arch.sizes[1:]):
40             W = rng.standard_normal((fan_in, fan_out)) * np.sqrt(2.0 / fan_in)
41             layers.append((W, np.zeros((1, fan_out))))
42         return tuple(layers)
43
44
45     def forward(params, X, hidden_activation):
46         '''Lớp cuối KHÔNG có hàm kích hoạt: đầu ra hồi quy là số thực bất kỳ.'''
47         act, _ = ACTIVATIONS[hidden_activation]
48         pre_activations, activations = [], [X]
49         h = X
50         last = len(params) - 1
51         for index, (W, b) in enumerate(params):
52             z = h @ W + b
53             h = z if index == last else act(z)
54             pre_activations.append(z)
55             activations.append(h)
56         return h, {"z": pre_activations, "h": activations}

```

3. Hàm mất mát và lan truyền ngược

In [5]

```

1     def mean_squared_error(z_true, z_pred):
2         return float(0.5 * np.mean((z_pred - z_true) ** 2))
3
4
5     def backward(params, z_true, cache, hidden_activation):
6         _, act_derivative = ACTIVATIONS[hidden_activation]
7         z, h = cache["z"], cache["h"]
8         n = len(z_true)
9         grads = []
10        delta = (h[-1] - z_true) / n
11        for index in range(len(params) - 1, -1, -1):
12            grads.append((h[index].T @ delta, np.sum(delta, axis=0, keepdims=True)))
13            if index:
14                delta = (delta @ params[index][0].T) * act_derivative(z[index - 1])
15        return tuple(reversed(grads))
16
17
18    def update(params, grads, learning_rate):
19        return tuple(
20            (W - learning_rate * dW, b - learning_rate * db)
21            for (W, b), (dW, db) in zip(params, grads)
22        )

```

4. Huấn luyện bốn kiến trúc

In [6]

```

1 LEARNING_RATE = 0.05
2 EPOCHS = 40
3 BATCH_SIZE = 512
4 INIT_SEED = 42
5 SHUFFLE_SEED = 7
6
7 ARCHITECTURES = (
8     Architecture("H1", "Baseline", (8, 32, 16, 1), "relu"),
9     Architecture("H2", "Wider", (8, 64, 32, 1), "relu"),
10    Architecture("H3", "Shallow", (8, 32, 1), "relu"),
11    Architecture("H4", "No ReLU", (8, 32, 16, 1), "linear"),
12 )
13
14
15 def fit(arch):
16     params = init_params(arch, INIT_SEED)
17     rng = np.random.default_rng(SHUFFLE_SEED)
18     n = len(X_train)
19     history_train, history_val = [], []
20     started = time.perf_counter()
21
22     for _ in range(EPOCHS):
23         order = rng.permutation(n)
24         for begin in range(0, n, BATCH_SIZE):
25             rows = order[begin : begin + BATCH_SIZE]
26             _, cache = forward(params, X_train[rows], arch.hidden_activation)
27             grads = backward(params, z_train[rows], cache, arch.hidden_activation)
28             params = update(params, grads, LEARNING_RATE)
29             history_train.append(mean_squared_error(z_train, forward(params, X_train, arch.hidden_act
ivation)[0]))
30             history_val.append(mean_squared_error(z_val, forward(params, X_val, arch.hidden_activatio
n)[0]))
31
32         seconds = time.perf_counter() - started
33         val_log = to_log_price(forward(params, X_val, arch.hidden_activation)[0])
34         metrics = ml.regression_metrics(task.y_val, val_log)
35         return {
36             "arch": arch, "params": params, "seconds": seconds,
37             "history_train": history_train, "history_val": history_val,
38             "metrics": metrics,
39         }
40
41
42 trained = {}
43 for arch in ARCHITECTURES:
44     trained[arch.key] = fit(arch)
45     m = trained[arch.key]["metrics"]
46     print(f"[{trained[arch.key]['seconds']:6.1f}s] {arch.key} {arch.label:<9} {arch.shape:<14} "
47           f"{arch.n_params:>5d} tham số R2_log={m['r2_log']:.4f} MAE={m['mae_usd']:.0f} USD")

```

In [7]

```

1 pd.DataFrame([
2     {
3         "mô hình": f"{r['arch'].key} {r['arch'].label}",
4         "kiến trúc": r["arch"].shape,
5         "kích hoạt": r["arch"].hidden_activation,
6         "tham số": r["arch"].n_params,
7         "loss cuối (train)": f"{r['history_train'][-1]:.5f}",

```

```

8     "loss cuối (val)": f"{r['history_val'][-1]:.5f}",
9     "R2_log": f"{r['metrics']['r2_log']:.4f}",
10    "MAE_USD": f"{r['metrics']['mae_usd']:, .0f}",
11    "MAPE_%": f"{r['metrics']['mape_percent']:.1f}",
12    "giây": f"{r['seconds']:.1f}",
13    }
14    for r in trained.values()
15 ])
```

In [8]

```

1  fig, axes = plt.subplots(1, 2, figsize=(10.0, 3.4))
2  styles = ["-", "--", "-.", ":"]
3
4  for (key, result), style in zip(trained.items(), styles):
5      label = f"{key} {result['arch'].label}"
6      axes[0].plot(result["history_train"], style, color="black", linewidth=1.1, label=label)
7      axes[1].plot(result["history_val"], style, color="black", linewidth=1.1, label=label)
8
9  axes[0].set_title("Mất mát trên tập train")
10 axes[1].set_title("Mất mát trên tập validation")
11 for axis in axes:
12     axis.set_xlabel("Epoch")
13     axis.set_ylabel("MSE (thang đã chuẩn hoá)")
14     axis.grid(True, linewidth=0.3, alpha=0.4)
15 axes[0].legend(frameon=False, fontsize=8)
16 fig.tight_layout()
17 plt.show()
```

5. So sánh với mô hình machine learning tốt nhất

In [9]

```

1  best_key = max(trained, key=lambda k: trained[k]["metrics"]["r2_log"])
2  best = trained[best_key]
3  best_arch = best["arch"]
4  print(f"Kiến trúc tốt nhất trên validation: {best_key} {best_arch.label} ({best_arch.shape})")
5
6  baseline = ml.run_model(task, task.baseline_name + " (mốc ML)", task.baseline_rep,
7                        task.baseline_estimator())
8  ml.print_table([ml.table_row(task, baseline)])
```

In [10]

```

1  test_log = to_log_price(forward(best["params"], X_test, best_arch.hidden_activation)[0])
2  dl_test = ml.regression_metrics(task.y_test, test_log)
3  ml_test = ml.evaluate_on_test(task, baseline["pipe"])
4
5  pd.DataFrame([
6      {"mô hình": f"Mạng nơ-ron {best_key} ({best_arch.shape})", **{
7          "R2_log": f"{dl_test['r2_log']:.4f}", "MAE_USD": f"{dl_test['mae_usd']:, .0f}",
8          "RMSE_USD": f"{dl_test['rmse_usd']:, .0f}", "MAPE_%": f"{dl_test['mape_percent']:.1f}"
9      }}, {"mô hình": task.baseline_name + " (mốc ML)", **{
10         "R2_log": f"{ml_test['r2_log']:.4f}", "MAE_USD": f"{ml_test['mae_usd']:, .0f}",
11         "RMSE_USD": f"{ml_test['rmse_usd']:, .0f}", "MAPE_%": f"{ml_test['mape_percent']:.1f}"
12     }}])
```

In [11]

```

1 actual_usd = np.exp(task.y_test)
2 predicted_usd = np.exp(test_log)
3 residual = predicted_usd - actual_usd
4
5 fig, axes = plt.subplots(1, 2, figsize=(10.0, 3.6))
6 sample = np.random.default_rng(INIT_SEED).choice(len(actual_usd), size=6000, replace=False)
7
8 axes[0].scatter(actual_usd[sample], predicted_usd[sample], s=3, alpha=0.25, color="black")
9 limits = [actual_usd.min(), actual_usd.max()]
10 axes[0].plot(limits, limits, "-", color="black", linewidth=0.9)
11 axes[0].set_xscale("log")
12 axes[0].set_yscale("log")
13 axes[0].set_xlabel("Giá thật (USD, thang log)")
14 axes[0].set_ylabel("Giá dự đoán (USD, thang log)")
15 axes[0].set_title("Dự đoán so với thực tế")
16
17 axes[1].scatter(actual_usd[sample], residual[sample], s=3, alpha=0.25, color="black")
18 axes[1].axhline(0, color="black", linewidth=0.9)
19 axes[1].set_xscale("log")
20 axes[1].set_xlabel("Giá thật (USD, thang log)")
21 axes[1].set_ylabel("Sai số dự đoán (USD)")
22 axes[1].set_title("Phân bố sai số theo phân khúc giá")
23
24 for axis in axes:
25     axis.grid(True, linewidth=0.3, alpha=0.4)
26 fig.tight_layout()
27 plt.show()

```

6. Ghi kết quả

In [12]

```

1 payload = {
2     "task": task.key,
3     "title": task.title,
4     "representation": "target_enc_scaled",
5     "d_input": int(X_train.shape[1]),
6     "n_train": task.n_train,
7     "n_val": task.n_val,
8     "n_test": task.n_test,
9     "audit": task.audit,
10    "target_transform": {"log_price_mean": y_mean, "log_price_std": y_std},
11    "learning_rate": LEARNING_RATE,
12    "epochs": EPOCHS,
13    "batch_size": BATCH_SIZE,
14    "init_seed": INIT_SEED,
15    "shuffle_seed": SHUFFLE_SEED,
16    "models": [
17        {
18            "key": r["arch"].key,
19            "label": r["arch"].label,
20            "shape": r["arch"].shape,
21            "hidden_activation": r["arch"].hidden_activation,
22            "n_params": r["arch"].n_params,
23            "seconds": round(r["seconds"], 2),
24            "loss_train_final": round(r["history_train"][-1], 6),
25            "loss_val_final": round(r["history_val"][-1], 6),
26            "loss_history_train": [round(v, 6) for v in r["history_train"]],

```

```

27         "loss_history_val": [round(v, 6) for v in r["history_val"]],
28         "validation": {k: round(v, 6) for k, v in r["metrics"].items()},
29     }
30     for r in trained.values()
31 ],
32 "best_model": best_key,
33 "test_metrics_dl": {k: round(v, 6) for k, v in dl_test.items()},
34 "test_metrics_ml_baseline": {k: round(v, 6) for k, v in ml_test.items()},
35 "ml_baseline_name": task.baseline_name,
36 }
37
38 path = ml.save_results("deeplearning_house", payload)
39 print("Đã ghi", path.relative_to(ROOT), f"({path.stat().st_size / 1024:.1f} KB)")

```

Phụ lục E. Notebook 08: Mạng nơ-ron có lớp nhúng cho e-commerce

Toàn bộ nội dung notebook, giữ nguyên thứ tự cell. Khối In [n] là mã nguồn thực thi, khối Out là kết quả chạy thật trên máy cục bộ. Mọi số liệu trong phần thân báo cáo đều được tham chiếu đồng nhất từ các thí nghiệm này.

08_deep_learning_ecommerce.ipynb · 24 cell · 14 code

Notebook 08 - Mạng nơ-ron có lớp nhúng học được cho bài toán e-commerce

Notebook này dựng ma trận nhúng từ \$E\$ thành tham số của mạng và được học bằng lan truyền ngược, kết hợp gộp trung bình có che (np.add.at).

In [1]

```
1 import sys
2 import time
3 import warnings
4 from collections import Counter
5 from dataclasses import dataclass
6 from pathlib import Path
7
8 import matplotlib
9 import matplotlib.pyplot as plt
10 import numpy as np
11 import pandas as pd
12
13 ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
14 if str(ROOT) not in sys.path:
15     sys.path.insert(0, str(ROOT))
16
17 import ml
18
19 warnings.filterwarnings("ignore")
20 matplotlib.rcParams.update({
21     "font.family": "serif",
22     "font.size": 9,
23     "axes.edgecolor": "black",
24     "axes.labelcolor": "black",
25     "text.color": "black",
26     "xtick.color": "black",
27     "ytick.color": "black",
28     "figure.facecolor": "white",
29     "savefig.facecolor": "white",
30 })
31
32 print("thư mục gốc :", ROOT)
33 print("numpy :", np.__version__)
```

1. Dữ liệu

In [2]

```

1 task = ml.load_ecommerce_satisfaction()
2 prep_tab = task.prep("tabular")
3
4 TAB_TRAIN = np.asarray(prepare_fit_transform(task.X_train, task.y_train).toarray())
5 TAB_VAL = np.asarray(prepare_transform(task.X_val).toarray())
6 TAB_TEST = np.asarray(prepare_transform(task.X_test).toarray())
7
8 y_train = np.asarray(task.y_train).reshape(-1, 1).astype(float)
9 y_val = np.asarray(task.y_val).reshape(-1, 1).astype(float)
10 y_test = np.asarray(task.y_test).reshape(-1, 1).astype(float)
11
12 print(f"bảng: train {TAB_TRAIN.shape} val {TAB_VAL.shape} test {TAB_TEST.shape}")
13 print(f"tỉ lệ hài lòng: train={y_train.mean():.4f} val={y_val.mean():.4f} test={y_test.mean():.4f}")

```

2. Văn bản thành số: tokenize, từ điển, chuỗi id

In [3]

```

1 VOCAB_SIZE = 5000
2 T_PAD = 32
3 D_EMBED = 32
4
5 counter = Counter()
6 for text in task.X_train[ml.datasets.ECOM_TEXT]:
7     counter.update(ml.tokenize(str(text)))
8
9 vocab = {word: index for index, (word, _) in enumerate(counter.most_common(VOCAB_SIZE), start=1)}
10 print(f"Token khác nhau trong tập train : {len(counter):,}")
11 print(f"Giữ lại trong từ điển : {len(vocab):,} (id 1..{len(vocab)}, id 0 = PAD)")
12 print("10 từ phổ biến nhất :", [w for w, _ in counter.most_common(10)])
13
14
15 def texts_to_ids(texts):
16     out = np.zeros((len(texts), T_PAD), dtype=np.int64)
17     for row, text in enumerate(texts):
18         ids = [vocab[w] for w in ml.tokenize(str(text)) if w in vocab][:T_PAD]
19         out[row, : len(ids)] = ids
20     return out
21
22
23 IDS_TRAIN = texts_to_ids(task.X_train[ml.datasets.ECOM_TEXT].values)
24 IDS_VAL = texts_to_ids(task.X_val[ml.datasets.ECOM_TEXT].values)
25 IDS_TEST = texts_to_ids(task.X_test[ml.datasets.ECOM_TEXT].values)
26
27 print(f"\nIDS_TRAIN {IDS_TRAIN.shape} (B x T)")
28 print("Ví dụ một bình luận:")
29 print("nguyên văn :", repr(str(task.X_train[ml.datasets.ECOM_TEXT].values[0])[:90]))
30 print("chuỗi id :", IDS_TRAIN[0][:16], "...")
31 print(f"số token thật: {int((IDS_TRAIN[0] > 0).sum())} / {T_PAD}")

```

3. Lớp nhúng và phép gộp trung bình có che

In [4]

```

1 def sigmoid(x):
2     return 1.0 / (1.0 + np.exp(-np.clip(x, -50.0, 50.0)))

```

```

3
4
5 def relu(x):
6     return np.maximum(0.0, x)
7
8
9 def relu_derivative(x):
10    return (x > 0).astype(float)
11
12
13 def init_dense(sizes, seed):
14     rng = np.random.default_rng(seed)
15     layers = []
16     for fan_in, fan_out in zip(sizes, sizes[1:]):
17         W = rng.standard_normal((fan_in, fan_out)) * np.sqrt(2.0 / fan_in)
18         layers.append((W, np.zeros((1, fan_out))))
19     return tuple(layers)
20
21
22 def init_embedding(seed):
23     '''Hàng 0 là token đệm, luôn giữ bằng 0 để nó không đóng góp gì vào phép gộp.'''
24     rng = np.random.default_rng(seed)
25     E = rng.standard_normal((VOCAB_SIZE + 1, D_EMBED)) * np.sqrt(1.0 / D_EMBED)
26     E[0] = 0.0
27     return E
28
29
30 def pool(E, ids):
31     mask = (ids > 0).astype(float)
32     counts = np.maximum(mask.sum(axis=1, keepdims=True), 1.0)
33     pooled = (E[ids] * mask[:, :, None]).sum(axis=1) / counts
34     return pooled, mask, counts
35
36
37 def scatter_embedding_grad(d_pooled, ids, mask, counts):
38     '''Đưa gradient của vector gộp về đúng các hàng từ vựng đã tham gia.'''
39     dE = np.zeros((VOCAB_SIZE + 1, D_EMBED))
40     rows, columns = np.nonzero(mask)
41     np.add.at(dE, ids[rows, columns], d_pooled[rows] / counts[rows])
42     dE[0] = 0.0
43     return dE

```

4. Mạng và hàm mất mát có trọng số lớp

In [5]

```

1 def class_weights(y):
2     n = len(y)
3     positive = float(y.sum())
4     return {1.0: n / (2.0 * positive), 0.0: n / (2.0 * (n - positive))}
5
6
7 WEIGHTS = class_weights(y_train)
8 W_TRAIN = np.where(y_train == 1.0, WEIGHTS[1.0], WEIGHTS[0.0])
9 print(f"trọng số lớp: hài lòng = {WEIGHTS[1.0]:.4f}, không hài lòng = {WEIGHTS[0.0]:.4f}")
10
11
12 def weighted_bce(y, y_hat, weight):
13     eps = 1e-8

```

```

14     p = np.clip(y_hat, eps, 1.0 - eps)
15     return float(-np.mean(weight * (y * np.log(p) + (1.0 - y) * np.log(1.0 - p))))
16
17
18 def dense_forward(layers, X):
19     pre_activations, activations = [], [X]
20     h = X
21     last = len(layers) - 1
22     for index, (W, b) in enumerate(layers):
23         z = h @ W + b
24         h = sigmoid(z) if index == last else relu(z)
25         pre_activations.append(z)
26         activations.append(h)
27     return h, {"z": pre_activations, "h": activations}
28
29
30 def dense_backward(layers, y, weight, cache):
31     '''Trả về gradient của từng lớp VÀ gradient theo đầu vào, để đẩy tiếp vào lớp nhúng.'''
32     z, h = cache["z"], cache["h"]
33     n = len(y)
34     grads = []
35     delta = weight * (h[-1] - y) / n
36     for index in range(len(layers) - 1, -1, -1):
37         grads.append((h[index].T @ delta, np.sum(delta, axis=0, keepdims=True)))
38         delta = delta @ layers[index][0].T
39         if index:
40             delta = delta * relu_derivative(z[index - 1])
41     return tuple(reversed(grads)), delta

```

5. Huấn luyện bốn mô hình

In [6]

```

1  LEARNING_RATE = 0.1
2  EPOCHS = 40
3  BATCH_SIZE = 256
4  INIT_SEED = 42
5  SHUFFLE_SEED = 7
6  HIDDEN = (32, 16)
7
8
9  @dataclass(frozen=True)
10 class Model:
11     key: str
12     label: str
13     use_tabular: bool
14     use_text: bool
15     train_embedding: bool
16
17
18  MODELS = (
19      Model("N1", "Chỉ bảng", True, False, False),
20      Model("N2", "Bảng + nhúng học được", True, True, True),
21      Model("N3", "Bảng + nhúng đóng băng", True, True, False),
22      Model("N4", "Chỉ văn bản", False, True, True),
23  )
24
25
26 def assemble(model, tab, ids, E):

```

```

27     '''Ghép đầu vào của mạng theo đúng cấu hình của từng mô hình.'''
28     if not model.use_text:
29         return tab, None, None, None
30     pooled, mask, counts = pool(E, ids)
31     X = np.hstack([tab, pooled]) if model.use_tabular else pooled
32     return X, mask, counts, pooled
33
34
35 def fit(model):
36     d_tab = TAB_TRAIN.shape[1]
37     d_in = (d_tab if model.use_tabular else 0) + (D_EMBED if model.use_text else 0)
38     layers = init_dense((d_in, *HIDDEN, 1), INIT_SEED)
39     E = init_embedding(INIT_SEED) if model.use_text else None
40
41     rng = np.random.default_rng(SHUFFLE_SEED)
42     n = len(TAB_TRAIN)
43     history_train, history_val = [], []
44     started = time.perf_counter()
45
46     for _ in range(EPOCHS):
47         order = rng.permutation(n)
48         for begin in range(0, n, BATCH_SIZE):
49             rows = order[begin : begin + BATCH_SIZE]
50             X, mask, counts, _ = assemble(model, TAB_TRAIN[rows], IDS_TRAIN[rows], E)
51             _, cache = dense_forward(layers, X)
52             grads, d_input = dense_backward(layers, y_train[rows], W_TRAIN[rows], cache)
53             layers = tuple(
54                 (W - LEARNING_RATE * dW, b - LEARNING_RATE * db)
55                 for (W, b), (dW, db) in zip(layers, grads)
56             )
57             if model.use_text and model.train_embedding:
58                 d_pooled = d_input[:, -D_EMBED:] if model.use_tabular else d_input
59                 E = E - LEARNING_RATE * scatter_embedding_grad(
60                     d_pooled, IDS_TRAIN[rows], mask, counts
61                 )
62
63             history_train.append(weighted_bce(
64                 y_train, predict_proba(model, layers, E, TAB_TRAIN, IDS_TRAIN), W_TRAIN))
65             history_val.append(weighted_bce(
66                 y_val, predict_proba(model, layers, E, TAB_VAL, IDS_VAL),
67                 np.where(y_val == 1.0, WEIGHTS[1.0], WEIGHTS[0.0])))
68
69     seconds = time.perf_counter() - started
70     proba_val = predict_proba(model, layers, E, TAB_VAL, IDS_VAL)
71     metrics = ml.classification_metrics(
72         task.y_val, (proba_val.ravel() >= 0.5).astype(int), proba_val.ravel()
73     )
74     n_params = sum(W.size + b.size for W, b in layers) + (E.size if model.use_text else 0)
75     return {
76         "model": model, "layers": layers, "E": E, "seconds": seconds,
77         "history_train": history_train, "history_val": history_val,
78         "metrics": metrics, "d_in": d_in, "n_params": int(n_params),
79     }
80
81
82 def predict_proba(model, layers, E, tab, ids):
83     X, _, _, _ = assemble(model, tab, ids, E)
84     return dense_forward(layers, X)[0]

```

```

1 trained = {}
2 for model in MODELS:
3     trained[model.key] = fit(model)
4     r = trained[model.key]
5     m = r["metrics"]
6     print(f"[{r['seconds']:6.1f}s] {model.key} {model.label:<24} d_in={r['d_in']:<4d} "
7           f"tham số={r['n_params']:>7,d} f1_macro={m['f1_macro']:.4f} auc={m['roc_auc']:.4f}")

```

In [8]

```

1 pd.DataFrame([
2     {
3         "mô hình": f"{r['model'].key} {r['model'].label}",
4         "d đầu vào": r["d_in"],
5         "tham số": f"{r['n_params']:,}",
6         "loss cuối (val)": f"{r['history_val'][-1]:.4f}",
7         "accuracy": f"{r['metrics']['accuracy']:.4f}",
8         "recall": f"{r['metrics']['recall']:.4f}",
9         "f1_macro": f"{r['metrics']['f1_macro']:.4f}",
10        "roc_auc": f"{r['metrics']['roc_auc']:.4f}",
11        "giây": f"{r['seconds']:.1f}",
12    }
13    for r in trained.values()
14 ])

```

In [9]

```

1 fig, axes = plt.subplots(1, 2, figsize=(10.0, 3.4))
2 styles = ["-", "--", "-.", ":"]
3
4 for (key, result), style in zip(trained.items(), styles):
5     label = f"{key} {result['model'].label}"
6     axes[0].plot(result["history_train"], style, color="black", linewidth=1.1, label=label)
7     axes[1].plot(result["history_val"], style, color="black", linewidth=1.1, label=label)
8
9 axes[0].set_title("Mất mát trên tập train")
10 axes[1].set_title("Mất mát trên tập validation")
11 for axis in axes:
12     axis.set_xlabel("Epoch")
13     axis.set_ylabel("Binary cross entropy có trọng số lớp")
14     axis.grid(True, linewidth=0.3, alpha=0.4)
15 axes[0].legend(frameon=False, fontsize=8)
16 fig.tight_layout()
17 plt.show()

```

6. Văn bản đóng góp bao nhiêu, và học những đóng góp bao nhiêu

In [10]

```

1 gain_text = trained["N2"]["metrics"]["f1_macro"] - trained["N1"]["metrics"]["f1_macro"]
2 gain_learn = trained["N2"]["metrics"]["f1_macro"] - trained["N3"]["metrics"]["f1_macro"]
3
4 print(f"N1 chỉ bằng F1-macro = {trained['N1']['metrics']['f1_macro']:.4f}")
5 print(f"N3 bằng + những đóng góp F1-macro = {trained['N3']['metrics']['f1_macro']:.4f}")
6 print(f"N2 bằng + những học được F1-macro = {trained['N2']['metrics']['f1_macro']:.4f}")
7 print(f"N4 chỉ văn bản F1-macro = {trained['N4']['metrics']['f1_macro']:.4f}")
8 print()
9 print(f"Thêm văn bản (N2 - N1) : {gain_text:+.4f} F1-macro")

```

```

10 print(f"Học ma trận nhúng (N2 - N3) : {gain_learn:+.4f} F1-macro")
11 print(f"Notebook 03, TF-IDF (tham chiếu): +0.1431 F1-macro trên bộ dữ liệu cũ")

```

7. Ma trận nhúng đã học được gì

In [11]

```

1 E_learned = trained["N2"]["E"]
2 index_to_word = {index: word for word, index in vocab.items()}
3 norms = np.linalg.norm(E_learned, axis=1, keepdims=True)
4 E_unit = E_learned / np.maximum(norms, 1e-8)
5
6
7 def nearest(word, top=6):
8     if word not in vocab:
9         return f"{word}: không có trong từ điển"
10    similarity = E_unit @ E_unit[vocab[word]]
11    similarity[0] = -np.inf
12    similarity[vocab[word]] = -np.inf
13    best = np.argsort(similarity)[::-1][:top]
14    return f"{word:<12} -> " + ", ".join(
15        f"{index_to_word[i]} ({similarity[i]:.2f})" for i in best if i in index_to_word
16    )
17
18
19 for word in ["good", "bad", "worst", "thanks", "refund", "delay"]:
20     print(nearest(word))

```

8. So sánh với mô hình machine learning tốt nhất

In [12]

```

1 best_key = max(trained, key=lambda k: trained[k]["metrics"]["f1_macro"])
2 best = trained[best_key]
3 print(f"Mô hình deep learning tốt nhất trên validation: {best_key} {best['model'].label}")
4
5 baseline = ml.run_model(task, task.baseline_name + " (mốc ML)", task.baseline_rep,
6                          task.baseline_estimator())
7 ml.print_table([ml.table_row(task, baseline)])

```

In [13]

```

1 proba_test = predict_proba(best["model"], best["layers"], best["E"], TAB_TEST, IDS_TEST).ravel()
2 dl_test = ml.classification_metrics(
3     task.y_test, (proba_test >= 0.5).astype(int), proba_test
4 )
5 ml_test = ml.evaluate_on_test(task, baseline["pipe"])
6
7 pd.DataFrame([
8     {"mô hình": f"Mạng nơ-ron {best_key} ({best['model'].label})",
9      "accuracy": f"{dl_test['accuracy']:.4f}", "recall": f"{dl_test['recall']:.4f}",
10     "f1": f"{dl_test['f1']:.4f}", "f1_macro": f"{dl_test['f1_macro']:.4f}",
11     "roc_auc": f"{dl_test['roc_auc']:.4f}"},
12     {"mô hình": task.baseline_name + " (mốc ML)",
13      "accuracy": f"{ml_test['accuracy']:.4f}", "recall": f"{ml_test['recall']:.4f}",
14     "f1": f"{ml_test['f1']:.4f}", "f1_macro": f"{ml_test['f1_macro']:.4f}"},

```

```

15     "roc_auc": f"{ml_test['roc_auc']:.4f}",
16 ])

```

9. Ghi kết quả

In [14]

```

1  payload = {
2      "task": task.key,
3      "title": task.title,
4      "n_train": task.n_train,
5      "n_val": task.n_val,
6      "n_test": task.n_test,
7      "audit": task.audit,
8      "vocab_size": len(vocab),
9      "T_pad": T_PAD,
10     "d_embed": D_EMBED,
11     "hidden": list(HIDDEN),
12     "learning_rate": LEARNING_RATE,
13     "epochs": EPOCHS,
14     "batch_size": BATCH_SIZE,
15     "init_seed": INIT_SEED,
16     "shuffle_seed": SHUFFLE_SEED,
17     "class_weights": {"positive": WEIGHTS[1.0], "negative": WEIGHTS[0.0]},
18     "models": [
19         {
20             "key": r["model"].key,
21             "label": r["model"].label,
22             "use_tabular": r["model"].use_tabular,
23             "use_text": r["model"].use_text,
24             "train_embedding": r["model"].train_embedding,
25             "d_in": r["d_in"],
26             "n_params": r["n_params"],
27             "seconds": round(r["seconds"], 2),
28             "loss_train_final": round(r["history_train"][-1], 6),
29             "loss_val_final": round(r["history_val"][-1], 6),
30             "loss_history_train": [round(v, 6) for v in r["history_train"]],
31             "loss_history_val": [round(v, 6) for v in r["history_val"]],
32             "validation": {k: (round(v, 6) if v is not None else None)
33                           for k, v in r["metrics"].items()},
34         }
35         for r in trained.values()
36     ],
37     "best_model": best_key,
38     "gain_text_f1_macro": round(gain_text, 6),
39     "gain_learned_embedding_f1_macro": round(gain_learn, 6),
40     "test_metrics_dl": {k: (round(v, 6) if v is not None else None) for k, v in dl_test.items()},
41     "test_metrics_ml_baseline": {k: (round(v, 6) if v is not None else None)
42                                for k, v in ml_test.items()},
43     "ml_baseline_name": task.baseline_name,
44 }
45
46 path = ml.save_results("deeplearning_ecommerce", payload)
47 print("Đã ghi", path.relative_to(ROOT), f"({path.stat().st_size / 1024:.1f} KB)")

```

Phụ lục F. Mã nguồn dùng chung và kịch bản dựng báo cáo

Dưới đây là toàn bộ mã nguồn của gói ml/, các kịch bản thực nghiệm và trình sinh tài liệu tự động. Mã nguồn được viết tối ưu, vector hoá thuần NumPy kết hợp scikit-learn pipeline.

ml/__init__.py

– Khởi tạo package ml và export các tiện ích

58 dòng

```
1  from __future__ import annotations
2
3  from . import datasets
4  from .split import RANDOM_SEED, split_70_15_15
5  from .metrics import classification_metrics, regression_metrics
6  from .datasets import (
7      ROOT,
8      DATA,
9      DIABETES_CSV,
10     HOUSE_CSV,
11     ECOM_CSV,
12     Task,
13     LOADERS,
14     load_diabetes,
15     load_house_price,
16     load_ecommerce_satisfaction,
17     load_ecommerce_interest,
18     tokenize,
19 )
20 from .benchmark import (
21     REPORTS,
22     run_model,
23     evaluate_on_test,
24     classification_row,
25     regression_row,
26     table_row,
27     print_table,
28     save_results,
29 )
30
31 __all__ = [
32     "datasets",
33     "RANDOM_SEED",
34     "split_70_15_15",
35     "classification_metrics",
36     "regression_metrics",
37     "ROOT",
38     "DATA",
39     "DIABETES_CSV",
40     "HOUSE_CSV",
41     "ECOM_CSV",
42     "Task",
43     "LOADERS",
44     "load_diabetes",
45     "load_house_price",
46     "load_ecommerce_satisfaction",
47     "load_ecommerce_interest",
48     "tokenize",
49     "REPORTS",
```

```

50     "run_model",
51     "evaluate_on_test",
52     "classification_row",
53     "regression_row",
54     "table_row",
55     "print_table",
56     "save_results",
57 ]
58

```

ml/datasets.py

– Nạp và tiền xử lý dữ liệu

369 dòng

```

1  """Nạp và làm sạch bốn bài toán ML, tái lập đúng quy trình của notebook 01-03.
2
3  Mỗi hàm `load_*` trả về một `Task` đã chia sẵn 70/15/15, kèm các nhà máy sinh bộ
4  tiền xử lý và mô hình mốc (mô hình đã được chọn ở notebook 01-03). Nhờ vậy hai
5  notebook 05 và 06 dùng chung đúng một cách làm sạch, một cách chia tập và một
6  mốc so sánh, khác biệt giữa hai notebook chỉ nằm ở thuật toán được thêm vào.
7
8  Dữ liệu đọc từ `data/` ở trạng thái hiện tại của kho, không phải bản chụp cũ mà
9  notebook 01-03 đã chạy, nên các con số ở đây không trùng với báo cáo Assignment 02.
10 """
11
12 from __future__ import annotations
13
14 import re
15 from dataclasses import dataclass, field
16 from pathlib import Path
17 from typing import Any, Callable
18
19 import numpy as np
20 import pandas as pd
21 from sklearn.compose import ColumnTransformer
22 from sklearn.ensemble import (
23     HistGradientBoostingRegressor,
24     RandomForestClassifier,
25 )
26 from sklearn.feature_extraction.text import TfidfVectorizer
27 from sklearn.impute import SimpleImputer
28 from sklearn.linear_model import LogisticRegression
29 from sklearn.model_selection import KFold
30 from sklearn.pipeline import Pipeline
31 from sklearn.preprocessing import (
32     OneHotEncoder,
33     OrdinalEncoder,
34     StandardScaler,
35     TargetEncoder,
36 )
37
38 from .split import RANDOM_SEED, split_70_15_15
39
40 ROOT = Path(__file__).resolve().parents[1]
41 DATA = ROOT / "data"
42 DIABETES_CSV = DATA / "diabetes" / "diabetes.csv"
43 HOUSE_CSV = DATA / "house_price" / "house_price.csv"
44 ECOM_CSV = DATA / "ecommerce" / "ecommerce.csv"
45
46 HOUSE_SAMPLE_SIZE = 250_000

```

```

47
48
49 @dataclass(frozen=True)
50 class Task:
51     key: str
52     title: str
53     kind: str
54     average: str
55     X_train: Any
56     X_val: Any
57     X_test: Any
58     y_train: Any
59     y_val: Any
60     y_test: Any
61     preprocessors: dict[str, Callable[[[]], Any]]
62     baseline_name: str
63     baseline_rep: str
64     baseline_estimator: Callable[[[]], Any]
65     audit: dict[str, Any] = field(default_factory=dict)
66
67     @property
68     def n_train(self) -> int:
69         return len(self.X_train)
70
71     @property
72     def n_val(self) -> int:
73         return len(self.X_val)
74
75     @property
76     def n_test(self) -> int:
77         return len(self.X_test)
78
79     def prep(self, rep: str):
80         return self.preprocessors[rep]()
81
82     def summary(self) -> str:
83         return (
84             f"{self.title}: train={self.n_train:}, val={self.n_val:}, test={self.n_test:}, "
85             f"| mốc so sánh: {self.baseline_name} ({self.baseline_rep})"
86         )
87
88
89 # ----- #
90 # Bài toán 1 – tiểu đường, phân loại nhị phân
91 # ----- #
92 DIA_BINARY = [
93     "HighBP", "HighChol", "CholCheck", "Smoker", "Stroke", "HeartDiseaseorAttack",
94     "PhysActivity", "Fruits", "Veggies", "HvyAlcoholConsump", "AnyHealthcare",
95     "NoDocbcCost", "DiffWalk", "Sex",
96 ]
97 DIA_NUMERIC = ["BMI", "MentHlth", "PhysHlth"]
98 DIA_ORDINAL = ["GenHlth", "Age", "Education", "Income"]
99 DIA_FEATURES = DIA_BINARY + DIA_NUMERIC + DIA_ORDINAL
100
101
102 def prep_diabetes_native() -> ColumnTransformer:
103     """Giữ nguyên 21 cột, chỉ chuẩn hoá phần liên tục và thứ bậc. d = 21.
104
105     Cột nhị phân đã nằm sẵn trong {0, 1} nên không cần đựng tới; hai nhóm còn lại
106     được đưa về cùng thang đo, điều kiện bắt buộc với mọi mô hình dựa trên khoảng cách.

```

```

107     """
108     return ColumnTransformer([
109         ("bin", "passthrough", DIA_BINARY),
110         ("num", StandardScaler(), DIA_NUMERIC),
111         ("ord", StandardScaler(), DIA_ORDINAL),
112     ])
113
114
115 def load_diabetes(path: Path = DIABETES_CSV) -> Task:
116     raw = pd.read_csv(path)
117     frame = raw.drop_duplicates().reset_index(drop=True)
118
119     X = frame[DIA_FEATURES].copy()
120     y = (frame["Diabetes_012"] > 0).astype(int)
121     X_tr, X_va, X_te, y_tr, y_va, y_te = split_70_15_15(X, y, stratify=True)
122
123     return Task(
124         key="diabetes",
125         title="Tiểu đường (phân loại nhị phân)",
126         kind="binary",
127         average="binary",
128         X_train=X_tr, X_val=X_va, X_test=X_te,
129         y_train=y_tr, y_val=y_va, y_test=y_te,
130         preprocessors={"native": prep_diabetes_native},
131         baseline_name="Random Forest",
132         baseline_rep="native",
133         baseline_estimator=lambda: RandomForestClassifier(
134             n_estimators=200, max_depth=14, min_samples_leaf=20,
135             class_weight="balanced_subsample", n_jobs=-1, random_state=RANDOM_SEED,
136         ),
137         audit={
138             "rows_raw": int(len(raw)),
139             "rows_after_dedup": int(len(frame)),
140             "duplicates_removed": int(len(raw) - len(frame)),
141             "positive_rate": float(y.mean()),
142             "n_features": len(DIA_FEATURES),
143         },
144     )
145
146
147 # ----- #
148 # Bài toán 2 – giá nhà, hồi quy trên log(price)
149 # ----- #
150 HOUSE_USE_COLS = ["status", "price", "bed", "bath", "acre_lot", "city", "state", "house_size"]
151 HOUSE_NUM = ["bed", "bath", "acre_lot", "house_size", "size_per_bed"]
152 HOUSE_CAT_LOW = ["state", "status"]
153 HOUSE_CAT_HIGH = ["city"]
154 HOUSE_FEATURES = HOUSE_NUM + HOUSE_CAT_LOW + HOUSE_CAT_HIGH
155
156
157 def prep_house_target_enc() -> ColumnTransformer:
158     """Target encoding: `city` nén về một chiều duy nhất. d = 8."""
159     return ColumnTransformer([
160         ("num", "passthrough", HOUSE_NUM),
161         ("cat", OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1), HOUSE_CAT_L
OW),
162         ("city", TargetEncoder(smooth=20.0, cv=KFold(5, shuffle=True, random_state=RANDOM_SEED)),
HOUSE_CAT_HIGH),
163     ])
164
165

```

```

166
167 def prep_house_target_enc_scaled() -> Pipeline:
168     """Bản target encoding đã đưa tám chiều về cùng thang đo.
169
170     Cây quyết định cắt theo từng chiều nên không quan tâm thang đo, nhưng khoảng
171     cách Euclid thì có: `house_size` chạy tới 10,000 còn `bed` chỉ tới 10, để
172     nguyên thì một chiều nuốt trọn bảy chiều còn lại.
173     """
174     return Pipeline([("encode", prep_house_target_enc()), ("scale", StandardScaler())])
175
176
177 def load_house_price(path: Path = HOUSE_CSV, sample_size: int = HOUSE_SAMPLE_SIZE) -> Task:
178     df = pd.read_csv(path, usecols=HOUSE_USE_COLS)
179     rows_raw = len(df)
180
181     h = df.dropna(subset=["price", "bed", "bath", "house_size", "city", "state"])
182     h = h[h["price"].between(50_000, 2_000_000)]
183     h = h[h["house_size"].between(200, 10_000)]
184     h = h[h["bed"].between(1, 10)]
185     h = h[h["bath"].between(1, 10)].copy()
186     h["acre_lot"] = h["acre_lot"].fillna(h["acre_lot"].median())
187     h = h[h["acre_lot"].between(0, 50)]
188     h = h.drop_duplicates(
189         subset=["price", "bed", "bath", "acre_lot", "city", "state", "house_size"]
190     )
191     h["city"] = h["city"].astype(str).str.strip().str.lower()
192     h["state"] = h["state"].astype(str).str.strip()
193     rows_clean = len(h)
194
195     h = h.sample(n=min(sample_size, len(h)), random_state=RANDOM_SEED).reset_index(drop=True)
196     h["size_per_bed"] = h["house_size"] / h["bed"].clip(lower=1)
197
198     X = h[HOUSE_FEATURES].copy()
199     y = np.log(h["price"].to_numpy(dtype=float))
200     X_tr, X_va, X_te, y_tr, y_va, y_te = split_70_15_15(X, y, stratify=False)
201
202     return Task(
203         key="house_price",
204         title="Giá nhà (hồi quy trên log giá)",
205         kind="regression",
206         average="",
207         X_train=X_tr, X_val=X_va, X_test=X_te,
208         y_train=y_tr, y_val=y_va, y_test=y_te,
209         preprocessors={
210             "target_enc": prep_house_target_enc,
211             "target_enc_scaled": prep_house_target_enc_scaled,
212         },
213         baseline_name="HistGradientBoosting",
214         baseline_rep="target_enc",
215         baseline_estimator=lambda: HistGradientBoostingRegressor(
216             max_iter=400, learning_rate=0.08, random_state=RANDOM_SEED
217         ),
218         audit={
219             "rows_raw": int(rows_raw),
220             "rows_clean": int(rows_clean),
221             "kept_percent": round(rows_clean / rows_raw * 100, 2),
222             "sample_size": int(len(h)),
223             "n_city": int(h["city"].nunique()),
224             "n_state": int(h["state"].nunique()),
225             "skew_log_price": float(pd.Series(y).skew()),

```

```

226     },
227 )
228
229
230 # ----- #
231 # Bài toán 3 và 4 – e-commerce, bảng ghép văn bản
232 # ----- #
233 ECOM_NUM = ["item_price", "resp_min", "hour", "dow", "rem_len", "rem_words", "has_rem"]
234 ECOM_CAT = ["channel_name", "category", "Sub-category", "Tenure Bucket", "Agent Shift"]
235 ECOM_TEXT = "remark"
236 ECOM_FEATURES = ECOM_NUM + ECOM_CAT + [ECOM_TEXT]
237
238 TOKEN_RE = re.compile(r"[a-z0-9']+")
239
240
241 def tokenize(text: str) -> list[str]:
242     return TOKEN_RE.findall(text.lower())
243
244
245 def _tfidf() -> TfidfVectorizer:
246     return TfidfVectorizer(
247         ngram_range=(1, 2), min_df=2, sublinear_tf=True,
248         max_features=20000, stop_words="english",
249     )
250
251
252 def prep_ecom_tabular() -> ColumnTransformer:
253     return ColumnTransformer([
254         ("num", Pipeline([("imp", SimpleImputer(strategy="median")), ("sc", StandardScaler())]),
255         ECOM_NUM),
256         ("cat", OneHotEncoder(handle_unknown="infrequent_if_exist", min_frequency=10), ECOM_CAT),
257     ])
258
259
260 def prep_ecom_tabular_text() -> ColumnTransformer:
261     return ColumnTransformer([
262         ("num", Pipeline([("imp", SimpleImputer(strategy="median")), ("sc", StandardScaler())]),
263         ECOM_NUM),
264         ("cat", OneHotEncoder(handle_unknown="infrequent_if_exist", min_frequency=10), ECOM_CAT),
265         ("text", _tfidf(), ECOM_TEXT),
266     ])
267
268
269 def build_ecommerce_frame(path: Path = ECOM_CSV) -> tuple[pd.DataFrame, dict[str, Any]]:
270     """Dựng bảng feature từ tệp thô, đúng các quyết định làm sạch của notebook 03."""
271     raw = pd.read_csv(path)
272
273     reported = pd.to_datetime(raw["Issue_reported at"], format="%d/%m/%Y %H:%M", errors="coerce")
274     responded = pd.to_datetime(raw["issue_responded"], format="%d/%m/%Y %H:%M", errors="coerce")
275     resp_min_raw = (responded - reported).dt.total_seconds() / 60.0
276
277     out = pd.DataFrame(index=raw.index)
278     out["item_price"] = pd.to_numeric(raw["Item_price"], errors="coerce")
279     out["resp_min"] = resp_min_raw.where(resp_min_raw >= 0)
280     out["hour"] = reported.dt.hour
281     out["dow"] = reported.dt.dayofweek
282
283     remark = raw["Customer Remarks"].fillna("").astype(str)
284     out["rem_len"] = remark.str.len()
285     out["rem_words"] = remark.str.split().str.len().fillna(0)

```

```

286 out["has_rem"] = (remark.str.strip() != "").astype(int)
287 for column in ECOM_CAT:
288     out[column] = raw[column].fillna("UNKNOWN").astype(str)
289 out[ECOM_TEXT] = remark
290
291 out["y_sat"] = (raw["CSAT Score"] >= 4).astype(int)
292 out["y_interest"] = raw["Product_category"]
293
294 audit = {
295     "rows_raw": int(len(raw)),
296     "rows_with_remark": int(out["has_rem"].sum()),
297     "negative_response_time": int((resp_min_raw < 0).sum()),
298     "sat_rate_overall": float(out["y_sat"].mean()),
299 }
300 return out, audit
301
302
303 def load_ecommerce_satisfaction(path: Path = ECOM_CSV) -> Task:
304     frame, audit = build_ecommerce_frame(path)
305     sub = frame[frame["has_rem"] == 1].reset_index(drop=True)
306
307     X_tr, X_va, X_te, y_tr, y_va, y_te = split_70_15_15(
308         sub[ECOM_FEATURES], sub["y_sat"], stratify=True
309     )
310     return Task(
311         key="ecommerce_satisfaction",
312         title="E-commerce: mức hài lòng (bảng + văn bản)",
313         kind="binary",
314         average="binary",
315         X_train=X_tr, X_val=X_va, X_test=X_te,
316         y_train=y_tr, y_val=y_va, y_test=y_te,
317         preprocessors={
318             "tabular": prep_ecom_tabular,
319             "tabular_text": prep_ecom_tabular_text,
320         },
321         baseline_name="LogReg bảng + TF-IDF",
322         baseline_rep="tabular_text",
323         baseline_estimator=lambda: LogisticRegression(
324             max_iter=2000, class_weight="balanced", random_state=RANDOM_SEED
325         ),
326         audit={**audit, "rows_used": int(len(sub)), "positive_rate": float(sub["y_sat"].mean())},
327     )
328
329
330 def load_ecommerce_interest(path: Path = ECOM_CSV) -> Task:
331     frame, audit = build_ecommerce_frame(path)
332     labelled = frame[frame["y_interest"].notna()].reset_index(drop=True)
333     sub = labelled[labelled["has_rem"] == 1].reset_index(drop=True)
334
335     X_tr, X_va, X_te, y_tr, y_va, y_te = split_70_15_15(
336         sub[ECOM_FEATURES], sub["y_interest"], stratify=False
337     )
338     return Task(
339         key="ecommerce_interest",
340         title="E-commerce: nhóm sản phẩm quan tâm (9 lớp)",
341         kind="multiclass",
342         average="macro",
343         X_train=X_tr, X_val=X_va, X_test=X_te,
344         y_train=y_tr, y_val=y_va, y_test=y_te,
345         preprocessors={

```

```

346         "tabular": prep_ecom_tabular,
347         "tabular_text": prep_ecom_tabular_text,
348     },
349     baseline_name="LogReg bảng + TF-IDF",
350     baseline_rep="tabular_text",
351     baseline_estimator=lambda: LogisticRegression(
352         max_iter=2000, class_weight="balanced", random_state=RANDOM_SEED
353     ),
354     audit={
355         **audit,
356         "rows_labelled": int(len(labelled)),
357         "rows_used": int(len(sub)),
358         "n_classes": int(sub["y_interest"].nunique()),
359     },
360 )
361
362
363 LOADERS: dict[str, Callable[[], Task]] = {
364     "diabetes": load_diabetes,
365     "house_price": load_house_price,
366     "ecommerce_satisfaction": load_ecommerce_satisfaction,
367     "ecommerce_interest": load_ecommerce_interest,
368 }
369

```

ml/benchmark.py

– Khung đo đạc và benchmark mô hình

127 dòng

```

1  """Chạy một mô hình trên một `Task` rồi gom kết quả về dạng thống nhất.
2
3  Cả notebook 05 và 06 đều gọi đúng hàm này, nên thời gian đo, số chiều d và tên
4  khoá metric của hai notebook so sánh được với nhau mà không cần đối chiếu tay.
5  """
6
7  from __future__ import annotations
8
9  import json
10 import time
11 from pathlib import Path
12 from typing import Any
13
14 from sklearn.pipeline import Pipeline
15
16 from .datasets import ROOT, Task
17 from .metrics import classification_metrics, regression_metrics
18
19 REPORTS = ROOT / "reports"
20
21
22 def run_model(
23     task: Task, name: str, rep: str, estimator: Any, *, sample_weight: Any = None
24 ) -> dict[str, Any]:
25     """Fit trên tập train, đo trên tập validation. Tập test không bị chạm tới ở đây.
26
27     `sample_weight` dành cho các mô hình không có tham số `class_weight` riêng, ví dụ
28     AdaBoost: trọng số cân bằng lớp được truyền thẳng vào lời gọi `fit`.
29     """
30     pipe = Pipeline([("prep", task.prep(rep)), ("model", estimator)])
31     fit_params = {} if sample_weight is None else {"model__sample_weight": sample_weight}

```

```

32
33     started = time.perf_counter()
34     pipe.fit(task.X_train, task.y_train, **fit_params)
35     fit_seconds = time.perf_counter() - started
36
37     started = time.perf_counter()
38     prediction = pipe.predict(task.X_val)
39     predict_seconds = time.perf_counter() - started
40
41     d = int(pipe.named_steps["prep"].transform(task.X_train.head(50)).shape[1])
42
43     if task.kind == "regression":
44         metrics = regression_metrics(task.y_val, prediction)
45     else:
46         score = None
47         if task.average == "binary" and hasattr(pipe.named_steps["model"], "predict_proba"):
48             score = pipe.predict_proba(task.X_val)[: , 1]
49         metrics = classification_metrics(
50             task.y_val, prediction, score, average=task.average
51         )
52
53     return {
54         "name": name,
55         "rep": rep,
56         "d": d,
57         "metrics": metrics,
58         "fit_seconds": round(fit_seconds, 2),
59         "predict_seconds": round(predict_seconds, 2),
60         "pipe": pipe,
61     }
62
63
64 def evaluate_on_test(task: Task, pipe: Pipeline) -> dict[str, Any]:
65     prediction = pipe.predict(task.X_test)
66     if task.kind == "regression":
67         return regression_metrics(task.y_test, prediction)
68
69     score = None
70     if task.average == "binary" and hasattr(pipe.named_steps["model"], "predict_proba"):
71         score = pipe.predict_proba(task.X_test)[: , 1]
72     return classification_metrics(task.y_test, prediction, score, average=task.average)
73
74
75 def classification_row(result: dict[str, Any]) -> dict[str, str]:
76     m = result["metrics"]
77     auc = m.get("roc_auc")
78     return {
79         "model": result["name"],
80         "d": f"{result['d']:,}",
81         "accuracy": f"{m['accuracy']:.4f}",
82         "precision": f"{m['precision']:.4f}",
83         "recall": f"{m['recall']:.4f}",
84         "f1_macro": f"{m['f1_macro']:.4f}",
85         "roc_auc": f"{auc:.4f}" if auc is not None else "-",
86         "fit_s": f"{result['fit_seconds']:.1f}",
87         "predict_s": f"{result['predict_seconds']:.1f}",
88     }
89
90
91 def regression_row(result: dict[str, Any]) -> dict[str, str]:

```

```

92     m = result["metrics"]
93     return {
94         "model": result["name"],
95         "d": f"{result['d']:.}",
96         "R2_log": f"{m['r2_log']:.4f}",
97         "MAE_USD": f"{m['mae_usd']:.0f}",
98         "RMSE_USD": f"{m['rmse_usd']:.0f}",
99         "R2_USD": f"{m['r2_usd']:.4f}",
100        "MAPE_%": f"{m['mape_percent']:.1f}",
101        "fit_s": f"{result['fit_seconds']:.1f}",
102        "predict_s": f"{result['predict_seconds']:.1f}",
103    }
104
105
106 def table_row(task: Task, result: dict[str, Any]) -> dict[str, str]:
107     return regression_row(result) if task.kind == "regression" else classification_row(result)
108
109
110 def print_table(rows: list[dict[str, str]]) -> None:
111     if not rows:
112         return
113     columns = list(rows[0])
114     widths = {c: max(len(c), *(len(str(r[c])) for r in rows)) for c in columns}
115     line = " ".join(c.ljust(widths[c]) for c in columns)
116     print(line)
117     print("-" * len(line))
118     for row in rows:
119         print(" ".join(str(row[c]).ljust(widths[c]) for c in columns))
120
121
122 def save_results(name: str, payload: dict[str, Any]) -> Path:
123     REPORTS.mkdir(parents=True, exist_ok=True)
124     path = REPORTS / f"{name}_results.json"
125     path.write_text(json.dumps(payload, indent=2, ensure_ascii=False), encoding="utf-8")
126     return path
127

```

ml/metrics.py

– Chỉ số đánh giá (F1, ROC-AUC, R2, MAE)

52 dòng

```

1  """Chỉ số đánh giá cho bài toán phân loại và hồi quy.
2
3  Dùng scikit-learn cho phần tính toán, lớp bọc ở đây chỉ gom kết quả về một dict
4  phẳng để bảng so sánh và tệp JSON kết quả dùng chung một tên khoá.
5  """
6
7  from __future__ import annotations
8
9  import numpy as np
10 from sklearn.metrics import (
11     accuracy_score,
12     f1_score,
13     mean_absolute_error,
14     mean_squared_error,
15     precision_score,
16     r2_score,
17     recall_score,
18     roc_auc_score,
19 )

```

```

20
21
22 def classification_metrics(
23     y_true, y_pred, y_score=None, *, average: str = "binary"
24 ) -> dict[str, float | None]:
25     out: dict[str, float | None] = {
26         "accuracy": float(accuracy_score(y_true, y_pred)),
27         "precision": float(precision_score(y_true, y_pred, average=average, zero_division=0)),
28         "recall": float(recall_score(y_true, y_pred, average=average, zero_division=0)),
29         "f1": float(f1_score(y_true, y_pred, average=average, zero_division=0)),
30         "f1_macro": float(f1_score(y_true, y_pred, average="macro", zero_division=0)),
31         "f1_weighted": float(f1_score(y_true, y_pred, average="weighted", zero_division=0)),
32         "roc_auc": None,
33     }
34     if y_score is not None and average == "binary":
35         out["roc_auc"] = float(roc_auc_score(y_true, y_score))
36     return out
37
38
39 def regression_metrics(y_true_log, y_pred_log) -> dict[str, float]:
40     """Đo trên cả hai không gian: log(price) để so mô hình, USD để đọc nghiệp vụ."""
41     true_usd = np.exp(y_true_log)
42     pred_usd = np.exp(y_pred_log)
43     return {
44         "mae_log": float(mean_absolute_error(y_true_log, y_pred_log)),
45         "rmse_log": float(np.sqrt(mean_squared_error(y_true_log, y_pred_log))),
46         "r2_log": float(r2_score(y_true_log, y_pred_log)),
47         "mae_usd": float(mean_absolute_error(true_usd, pred_usd)),
48         "rmse_usd": float(np.sqrt(mean_squared_error(true_usd, pred_usd))),
49         "r2_usd": float(r2_score(true_usd, pred_usd)),
50         "mape_percent": float(np.mean(np.abs(pred_usd - true_usd) / true_usd) * 100.0),
51     }
52

```

ml/split.py

– Chia tập 70/15/15 phân tầng

33 dòng

```

1  """Chia 70/15/15 dùng chung cho cả ba bài toán.
2
3  Công thức giữ nguyên như quy ước đã dùng ở notebook 01-03: cắt 15% làm test
4  trước, phần còn lại cắt tiếp 15/85 làm validation. Cùng một seed và cùng một
5  thứ tự cắt thì cùng một dataset luôn cho ra cùng một tập chỉ số.
6  """
7
8  from __future__ import annotations
9
10 from typing import Any
11
12 from sklearn.model_selection import train_test_split
13
14 RANDOM_SEED = 42
15 TEST_SHARE = 0.15
16 VAL_SHARE_OF_REST = TEST_SHARE / (1.0 - TEST_SHARE)
17
18
19 def split_70_15_15(
20     X: Any, y: Any, *, stratify: bool = True, seed: int = RANDOM_SEED
21 ) -> tuple[Any, Any, Any, Any, Any, Any]:
22     X_rest, X_test, y_rest, y_test = train_test_split(

```

```

23     X, y, test_size=TEST_SHARE, random_state=seed, stratify=y if stratify else None
24 )
25 X_train, X_val, y_train, y_val = train_test_split(
26     X_rest,
27     y_rest,
28     test_size=VAL_SHARE_OF_REST,
29     random_state=seed,
30     stratify=y_rest if stratify else None,
31 )
32 return X_train, X_val, X_test, y_train, y_val, y_test
33

```

reports/build/content03.py

– Kịch bản tạo nội dung chương 1-8, 13

517 dòng

```

1  """Sinh phần thân báo cáo Assignment 03 từ reports/deeplearning_results.json.
2
3  Mọi con số trong báo cáo đều đọc từ tệp kết quả mà notebook 04 ghi ra, không gõ tay,
4  nên báo cáo và notebook không thể lệch nhau.
5  """
6
7  from __future__ import annotations
8
9  import json
10 from pathlib import Path
11
12 from content03_extra import build_extra # noqa: E402
13
14 ROOT = Path(__file__).resolve().parents[2]
15 RESULTS = ROOT / "reports" / "deeplearning_results.json"
16
17 STUDENT = "Nguyễn Đại Dũng"
18 STUDENT_ID = "B23DCVT103"
19 CLASS_NAME = "E23VT01"
20 GROUP = "02"
21 LECTURER = "Dinh Que Tran, Ph.D., Assoc. Prof."
22 TERM = "Học kỳ I.2026"
23
24 MODEL_NOTES = {
25     "M1": (
26         "Kiến trúc gốc của bài giảng, dùng làm mốc so sánh cho ba mô hình còn lại. "
27         "Hai lớp ẩn ReLU tạo ra hai tầng biểu diễn trung gian trước khi sigmoid quy về xác suất."
28     ),
29     "M2": (
30         "Nhân đôi bề rộng cả hai lớp ẩn, số tham số tăng gần ba lần. "
31         "Câu hỏi cần trả lời là thêm sức chứa có đổi lấy được chất lượng hay không."
32     ),
33     "M3": (
34         "Bỏ hẳn lớp ẩn thứ hai, chỉ còn một tầng biểu diễn. "
35         "Đây là phép thử xem chiều sâu có thực sự cần cho tám đặc trưng bảng hay không."
36     ),
37     "M4": (
38         "Giữ nguyên ba lớp nhưng thay ReLU bằng hàm đồng nhất. "
39         "Khi đó ba phép biến đổi tuyến tính xếp chồng vẫn chỉ là một phép tuyến tính, "
40         "nên mạng suy biến thành hồi quy logistic dù vẫn có 289 tham số."
41     ),
42 }
43
44

```

[illegible]

```

105     for m in models
106     )
107
108     return f"""
109 <h2 id="m1">1. Bài toán và phạm vi</h2>
110
111 <p>Deep Learning là phép hợp thành của các hàm có tham số. Thay vì học một ánh xạ trực
112 tiếp từ đầu vào sang nhãn, mô hình dựng nhiều phép biến đổi nối tiếp rồi học tham số của
113 chúng.</p>
114
115 <div class="formula">ŷ = f<sub>3</sub>(f<sub>2</sub>(f<sub>1</sub>(X))) = σ(ReLU(ReLU(XW<sub>1</sub>/s
ub> + b<sub>1</sub>)</div>
116
117 <p>Bài toán là phân loại nhị phân nguy cơ tiểu đường từ bộ khảo sát BRFSS 2015. Đầu vào
118 là tám chỉ số sức khỏe, đầu ra là một xác suất. Ràng buộc của bài là không dùng framework
119 học sâu nào: mọi phép toán của mạng, kể cả đạo hàm, đều viết bằng NumPy.</p>
120
121 <div class="chain">
122 <div class="st on"><k>01</k><b>Biểu diễn</b><i>8 đặc trưng, chuẩn hoá z</i></div>
123 <div class="st on"><k>02</k><b>Mạng NumPy</b><i>forward, loss, backward</i></div>
124 <div class="st on"><k>03</k><b>Huấn luyện</b><i>gradient descent lô nhỏ</i></div>
125 <div class="st on"><k>04</k><b>Đánh giá</b><i>4 mô hình, 3 ngưỡng</i></div>
126 </div>
127
128 <div class="tcap" data-key="t1">Bốn kiến trúc được so sánh</div>
129 <table>
130 <thead><tr>
131 <th style="width:16%">Mô hình</th>
132 <th style="width:18%">Kiến trúc</th>
133 <th style="width:13%">Kích hoạt lớp ẩn</th>
134 <th class="num" style="width:11%">Tham số</th>
135 <th>Mục đích</th>
136 </tr></thead>
137 <tbody>{rows}</tbody>
138 </table>
139 <p class="tnote">Cả bốn dùng chung seed khởi tạo {data["init_seed"]} và seed xáo trộn
140 {data["shuffle_seed"]}, cùng học suất {data["learning_rate"]}, cùng {data["epochs"]} epoch
141 và cùng lô {data["batch_size"]} dòng.</p>
142
143 <p>Khoảng cách giữa mô hình tốt nhất và kém nhất theo ROC-AUC chỉ là {_n(spread)}. Đây là
144 kết quả đáng chú ý nhất của báo cáo và sẽ được giải thích ở mục 8: với tám đặc trưng bảng
145 đã chuẩn hoá, phần lớn tín hiệu là tuyến tính, nên chiều sâu và bề rộng mang lại rất ít.</p>
146 """
147
148
149 def representation(data: dict) -> str:
150     ranking = data["feature_ranking"]
151     chosen = data["features"]
152     rows = "".join(
153         f'<tr><td class="num">{i}</td><td class="a1">{name}</td>'
154         f'<td class="num">{ranking[name]:.4f}</td></tr>'
155         for i, name in enumerate(chosen, 1)
156     )
157     dropped = [k for k in ranking if k not in chosen][:4]
158     dropped_text = ", ".join(f"{k} ({ranking[k]:.4f})" for k in dropped)
159
160     return f"""
161 <h2 id="m2">2. Biểu diễn dữ liệu</h2>
162
163 <p>Bài giảng dùng bộ Pima với đúng tám biến đầu vào. Bộ BRFSS ở đây có 21 biến, nên phải

```

[illegible]

```

224
225 <h3 id="m3-2">3.2 Hàm mất mát</h3>
226
227 <div class="formula">
$$L = \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$
</div>
228
229
230 <p>Xác suất được cắt về khoảng  $[1e-8, 1 - 1e-8]$  trước khi lấy log, nếu không một
231 dự đoán tự tin tuyệt đối và sai sẽ cho  $\log(0)$  và làm hỏng cả vòng huấn luyện.</p>
232
233 <h3 id="m3-3">3.3 Lan truyền ngược</h3>
234
235 <p>Ghép sigmoid với binary cross-entropy cho một kết quả rất gọn ở lớp cuối. Đạo hàm của
236 hàm mất mát theo đầu vào của sigmoid không còn chứa đạo hàm sigmoid nữa:</p>
237
238 <div class="formula">
$$\partial L / \partial Z = (\hat{y} - y) / N$$
</div>
239
240 <p>Từ đó gradient lan ngược bằng quy tắc chuỗi, mỗi lớp chỉ cần hai phép nhân ma trận và
241 một phép nhân từng phần tử với đạo hàm của hàm kích hoạt:</p>
242
243 <div class="formula">
$$dW_k = H_{k-1}^{sup} \delta_k, \quad \delta_k = (W_k^{sup} - y_k) \odot g'(Z_k)$$
</div>
244
245
246 <p>Lan truyền ngược không phải một phép thuật riêng của học sâu. Nó chỉ là quy tắc chuỗi
247 áp cho một hàm hợp thành, thực hiện theo thứ tự ngược để dùng lại kết quả trung gian mà
248 lan truyền xuôi đã lưu.</p>
249
250 <h3 id="m3-4">3.4 Cập nhật tham số</h3>
251
252 <div class="formula">
$$W \leftarrow W - \eta \partial L / \partial W$$
</div>
253
254 <p>Hàm cập nhật trả về bộ tham số mới thay vì sửa tại chỗ, nên mỗi bước huấn luyện là một
255 phép biến đổi rõ ràng từ trạng thái cũ sang trạng thái mới. Cách viết này tốn thêm một
256 lần cấp phát mỗi bước nhưng loại hẳn một lớp lỗi khó tìm khi gỡ rối gradient.</p>
257 ""
258
259
260 def _threshold_reading(model: dict) -> str:
261     low = model["test_by_threshold"]["0.3"]
262     mid = model["test_by_threshold"]["0.5"]
263     high = model["test_by_threshold"]["0.7"]
264     return (
265         f"Hạ ngưỡng từ 0.5 xuống 0.3 nâng recall từ {_n(mid['recall'])} lên "
266         f"{_n(low['recall'])} và đổi lại precision giảm từ {_n(mid['precision'])} xuống "
267         f"{_n(low['precision'])}, tức bắt thêm {low['tp'] - mid['tp']:,} ca dương thật "
268         f"nhưng báo động nhầm thêm {low['fp'] - mid['fp']:,} ca. Nâng lên 0.7 đẩy recall "
269         f"xuống {_n(high['recall'])}. Với bài sàng lọc bệnh, bỏ sót tổn kém hơn báo động "
270         f"nhầm, nên ngưỡng thấp là lựa chọn hợp lý hơn 0.5."
271     )
272
273
274 def _model_reading(data: dict, model: dict) -> str:
275     baseline = data["models"][0]
276     if model["key"] == "M1":
277         return (
278             "Đây là mốc tham chiếu. Hai lớp ẩn ReLU dựng được hai tầng biểu diễn, nhưng "
279             "recall tại ngưỡng 0.5 chỉ ở mức "
280             f"{_n(model['test_by_threshold']['0.5']['recall'])}: mạng hạ hàm mất mát nhanh "
281             "nhất bằng cách nghiêng về lớp âm chiếm đa số, đúng như dự đoán từ phân bố lớp."
282         )

```

```

283     delta_auc = model["roc_auc_test"] - baseline["roc_auc_test"]
284     delta_f1 = model["test_by_threshold"]["0.5"]["f1"] - baseline["test_by_threshold"]["0.5"]["f
1"]
285     direction = "cao hơn" if delta_auc > 0 else "thấp hơn"
286     if model["key"] == "M2":
287         return (
288             f"So với M1, ROC-AUC {direction} {abs(delta_auc):.4f} và F1 lệch {delta_f1:+.4f}, "
289             f"trong khi số tham số tăng từ {baseline['n_params']} lên {model['n_params']} và "
290             f"thời gian huấn luyện tăng từ {baseline['seconds']:.0f}s lên {model['seconds']:.0f}
s. "
291             "Thêm sức chứa không mua được chất lượng ở bài này, vì giới hạn nằm ở lượng "
292             "thông tin trong tám đặc trưng chứ không ở khả năng biểu diễn của mạng."
293         )
294     if model["key"] == "M3":
295         return (
296             f"Bỏ hẳn một lớp ẩn làm ROC-AUC {direction} {abs(delta_auc):.4f} so với M1, với "
297             f"chỉ {model['n_params']} tham số, bằng khoảng {model['n_params'] / baseline['n_param
s']:.0%} "
298             f"của M1, và nhanh hơn {baseline['seconds'] / max(model['seconds'], 0.01):.1f} lần. "
299             "Nếu phải chọn một mô hình để triển khai thì đây là ứng viên hợp lý nhất: "
300             "chất lượng ngang ngửa với chi phí thấp nhất."
301         )
302     return (
303         f"Bỏ ReLU làm ROC-AUC {direction} {abs(delta_auc):.4f} và F1 lệch {delta_f1:+.4f} so "
304         f"với M1. Mức giảm có thật nhưng nhỏ, và đó mới là điều đáng nói: ba phép biến đổi "
305         f"tuyến tính xếp chồng chỉ tương đương một phép tuyến tính, nên M4 thực chất là hồi "
306         f"quy logistic đội lốt mạng ba lớp. Việc nó gần bằng ba mô hình kia là bằng chứng "
307         f"trực tiếp rằng phần lớn tín hiệu trong tám đặc trưng này là tuyến tính."
308     )
309
310
311 def model_section(data: dict, index: int, model: dict) -> str:
312     key = model["key"]
313     t05 = model["test_by_threshold"]["0.5"]
314     train = model["train"]
315     gap = train["accuracy"] - t05["accuracy"]
316
317     threshold_rows = "".join(
318         f'<tr><td class="a1">{th}</td>'
319         f'<td class="num">{_n(row["accuracy"])}</td>'
320         f'<td class="num">{_n(row["precision"])}</td>'
321         f'<td class="num">{_n(row["recall"])}</td>'
322         f'<td class="num">{_n(row["f1"])}</td>'
323         f'<td class="num">{row["tp"]:,}</td>'
324         f'<td class="num">{row["fn"]:,}</td>'
325         f'<td class="num">{row["fp"]:,}</td>'
326         f'<td class="num">{row["tn"]:,}</td></tr>'
327         for th, row in model["test_by_threshold"].items()
328     )
329
330     hidden = ", ".join(f"H{i}: ({s[0]:,}, {s[1]})" for i, s in enumerate(model["hidden_shapes"],
1))
331
332     return f"""
333 <h2 id="m{index}">{index}. Mô hình {key} {model["label"]}</h2>
334
335 <p>{MODEL_NOTES[key]}</p>
336
337 <div class="kpi">
338 <div class="kpi"><b>{_n(t05["accuracy"])}</b><span>Accuracy &middot; test</span></div>

```

```

339 <div class="kpi"><b>{_n(t05["f1"])}</b><span>F1 &middot; ngưỡng 0.5</span></div>
340 <div class="kpi"><b>{_n(model["roc_auc_test"])}</b><span>ROC-AUC &middot; test</span></div>
341 <div class="kpi"><b>{model["seconds"]:.0f}s</b><span>Thời gian huấn luyện</span></div>
342 </div>
343
344 <div class="tcap" data-key="t{index}a">Cấu hình và quá trình huấn luyện của {key}</div>
345 <table>
346 <thead><tr><th style="width:34%">Hạng mục</th><th>Giá trị</th></tr></thead>
347 <tbody>
348 <tr><td class="a1">Kiến trúc</td><td class="mono">{model["shape"]}</td></tr>
349 <tr><td class="a1">Kích hoạt lớp ẩn</td><td>{"ReLU" if model["hidden_activation"] == "relu" else
"tuyến tính (hàm đồng nhất)"}</td></tr>
350 <tr><td class="a1">Số tham số học được</td><td class="num">{model["n_params"]}</td></tr>
351 <tr><td class="a1">Kích thước biểu diễn trung gian</td><td class="mono">{hidden}</td></tr>
352 <tr><td class="a1">Học suất, epoch, lô</td><td class="num">{model["learning_rate"]} &middot; {model["epochs"]} &middot; {model["batch_size"]}</td></tr>
353 <tr><td class="a1">Số lần cập nhật tham số</td><td class="num">{model["updates"]:,}</td></tr>
354 <tr><td class="a1">Loss train đầu và cuối</td><td class="num">{_n(model["loss_first"])} &arr; {_n(model["loss_final"])}</td></tr>
355 <tr><td class="a1">Loss trên tập test</td><td class="num">{_n(model["loss_test_end"])}</td></tr>
356 <tr><td class="a1">Thời gian huấn luyện</td><td class="num">{model["seconds"]:.1f} giây</td></tr>
357 </tbody>
358 </table>
359 <p class="tnote">Chênh lệch accuracy giữa train và test là {gap:+.4f}, và loss test
360 ({_n(model["loss_test_end"])} so với loss train ({_n(model["loss_final"])})) lệch
361 {model["loss_test_end"] - model["loss_final"]:+.4f}. Khoảng cách hẹp như vậy cho thấy mô
362 hình chưa quá khớp, phần thiếu nằm ở sức biểu diễn chứ không ở việc học thuộc dữ liệu.</p>
363
364 <div class="tcap" data-key="t{index}b">Kết quả của {key} trên tập test theo ba ngưỡng quyết định
</div>
365 <table>
366 <thead><tr>
367 <th style="width:12%">Ngưỡng</th>
368 <th class="num">Accuracy</th>
369 <th class="num">Precision</th>
370 <th class="num">Recall</th>
371 <th class="num">F1</th>
372 <th class="num">TP</th>
373 <th class="num">FN</th>
374 <th class="num">FP</th>
375 <th class="num">TN</th>
376 </tr></thead>
377 <tbody>{threshold_rows}</tbody>
378 </table>
379 <p class="tnote">{_threshold_reading(model)}</p>
380
381 <p>{_model_reading(data, model)}</p>
382 ""
383
384
385 def comparison(data: dict) -> str:
386     models = data["models"]
387     best = max(models, key=lambda m: m["roc_auc_test"])
388
389     rows = "".join(
390         f'<tr class="{ "hi" if m["key"] == best["key"] else "" }">'
391         f'<td class="a1">{m["key"]} {m["label"]}</td>'
392         f'<td class="num">{m["n_params"]}</td>'
393         f'<td class="num">{_n(m["loss_final"])}</td>'
394         f'<td class="num">{_n(m["loss_test_end"])}</td>'

```

```

395         f'<td class="num">{_n(m["test_by_threshold"]["0.5"]["accuracy"])}</td>'
396         f'<td class="num">{_n(m["test_by_threshold"]["0.5"]["precision"])}</td>'
397         f'<td class="num">{_n(m["test_by_threshold"]["0.5"]["recall"])}</td>'
398         f'<td class="num">{_n(m["test_by_threshold"]["0.5"]["f1"])}</td>'
399         f'<td class="num">{_n(m["roc_auc_test"])}</td>'
400         f'<td class="num">{m["seconds"]:.1f}</td></tr>'
401     for m in models
402 )
403
404     return f"""
405 <h2 id="m8">8. So sánh bốn mô hình</h2>
406
407 <div class="tcap" data-key="t10">Bảng tổng hợp bốn mô hình trên tập test, ngưỡng 0.5</div>
408 <table>
409 <thead><tr>
410 <th style="width:15%">Mô hình</th>
411 <th class="num">Tham số</th>
412 <th class="num">Loss train</th>
413 <th class="num">Loss test</th>
414 <th class="num">Accuracy</th>
415 <th class="num">Precision</th>
416 <th class="num">Recall</th>
417 <th class="num">F1</th>
418 <th class="num">ROC-AUC</th>
419 <th class="num">Giây</th>
420 </tr></thead>
421 <tbody>{rows}</tbody>
422 </table>
423 <p class="tnote">Dòng in đậm là mô hình tốt nhất theo ROC-AUC. Chỉ số này được chọn làm
424 tiêu chí chính vì nó không phụ thuộc ngưỡng quyết định, trong khi accuracy và F1 tại
425 ngưỡng 0.5 đều bị lớp âm chiếm đa số kéo lệch.</p>
426
427 <figure>
428 
429 <figcaption data-key="f2">Đường mất mát trên tập train của bốn mô hình, toàn bộ quá
430 trình và phần phóng to từ epoch 10.</figcaption>
431 </figure>
432
433 <div class="oim"><p><k>Hình <r>f2</r></k><b>Observation:</b> cả bốn đường tụt rất nhanh
434 trong mười epoch đầu rồi gần như nằm ngang, và ba đường có ReLU nằm chồng lên nhau.
435 <b>Interpretation:</b> phần dễ học của bài toán được học xong sớm, phần còn lại không
436 nằm trong tầm với của tám đặc trưng này. <b>ML implication:</b> kéo dài huấn luyện hay
437 tăng sức chứa đều không đáng, muốn tiến thêm phải thêm đặc trưng chứ không phải thêm lớp.</p></di
v>
438
439 <figure>
440 
441 <figcaption data-key="f3">Đánh đổi giữa precision và recall khi đổi ngưỡng quyết định,
442 và so sánh ba chỉ số chính của bốn mô hình.</figcaption>
443 </figure>
444
445 <h3 id="m8-2">8.2 Điều bốn mô hình nói lên</h3>
446
447 <ul class="plain">
448 <li><b>Chiều sâu gần như không đổi được kết quả.</b> M3 chỉ có một lớp ẩn và
449 {models[2]["n_params"]} tham số nhưng ROC-AUC chỉ lệch
450 {abs(models[2]["roc_auc_test"] - models[0]["roc_auc_test"]):.4f} so với M1.</li>
451 <li><b>Bề rộng cũng vậy.</b> M2 nhiều hơn M1 {models[1]["n_params"] - models[0]["n_params"]}
452 tham số và tốn thêm {models[1]["seconds"] - models[0]["seconds"]:.0f} giây mà không đổi
453 được thứ hạng.</li>

```

```

454 <li><b>Phi tuyến có đóng góp, nhưng nhỏ.</b> M4 mất
455 {abs(models[3]["roc_auc_test"] - models[0]["roc_auc_test"]):.4f} ROC-AUC khi bỏ ReLU. Đây
456 là số đo của phần tín hiệu phi tuyến trong tám đặc trưng này.</li>
457 <li><b>Ngưỡng quyết định đáng giá hơn kiến trúc.</b> Hạ ngưỡng từ 0.5 xuống 0.3 đối
458 recall của M1 nhiều hơn hẳn mọi khác biệt giữa bốn kiến trúc cộng lại.</li>
459 </ul>
460 ""
461
462
463 def conclusion(data: dict) -> str:
464     models = data["models"]
465     best = max(models, key=lambda m: m["roc_auc_test"])
466     ref = 1 - data["positive_rate_test"]
467
468     return f"""
469 <h2 id="m13">13. Kết luận và giới hạn</h2>
470
471 <p>Một mạng nơ-ron ba lớp viết hoàn toàn bằng NumPy đã học được trên hơn hai trăm nghìn
472 dòng dữ liệu thật, chạy trên CPU trong vài chục giây mỗi mô hình. Điều framework tự động
473 hoá không phải bản chất toán học mà là công sức triển khai: đạo hàm tự động, quản lý
474 tensor, tối ưu hoá và tính toán trên GPU.</p>
475
476 <p>Mô hình tốt nhất theo ROC-AUC là {best["key"]} {best["label"]} với
477 {_n(best["roc_auc_test"])}. Nhưng kết luận có ích hơn cho việc triển khai lại là M3: ít
478 tham số nhất, nhanh nhất, chất lượng gần như không kém.</p>
479
480 <div class="risk"><span class="lab">Giới hạn tự nhận</span>
481 <b>Một, accuracy ở đây dễ gây hiểu nhầm.</b> Đoán toàn lớp âm đã cho khoảng {_n(ref, 4)},
482 nên mọi con số accuracy trong báo cáo phải đọc kèm recall và ROC-AUC.
483 <b>Hai, chọn đặc trưng bằng tương quan tuyến tính đơn biến</b> có thể đã loại mất biến chỉ
484 có ích khi kết hợp với biến khác, và điều đó tự nó đã thu hẹp phần phi tuyến còn lại cho
485 mạng khai thác.
486 <b>Ba, không có tập validation riêng.</b> Siêu tham số được chọn bằng suy luận về số lần
487 cập nhật chứ không bằng dò trên tập validation, nên các con số trên tập test là ước lượng
488 trung thực cho đúng cấu hình này chứ không phải cho cấu hình tốt nhất có thể.
489 </div>
490
491 <p>Hướng đi tiếp có ích nhất không phải mạng sâu hơn mà là thêm thông tin đầu vào: dùng
492 đủ 21 biến thay vì tám, thêm đặc trưng tương tác, hoặc cân bằng lại lớp bằng trọng số
493 trong hàm mất mát. Cả bốn mô hình ở đây đều đã chạm trần thông tin của tám cột hiện có.</p>
494 ""
495
496
497 def build_content() -> str:
498     data = json.loads(RESULTS.read_text(encoding="utf-8"))
499     parts = [
500         cover(data),
501         overview(data),
502         representation(data),
503         network_math(),
504     ]
505     for offset, model in enumerate(data["models"]):
506         parts.append(model_section(data, 4 + offset, model))
507     parts.append(comparison(data))
508     parts.append(build_extra())
509     parts.append(conclusion(data))
510     return "\n".join(parts)
511
512
513 if __name__ == "__main__":

```

```

514 out = Path(__file__).resolve().parent / "content03.html"
515 out.write_text(build_content(), encoding="utf-8")
516 print(f"{out} ({out.stat().st_size / 1024:.0f} KB)")
517

```

reports/build/content03_extra.py

– Kịch bản tạo nội dung chương 9-12

597 dòng

```

1  """Sinh các chương mở rộng của báo cáo Assignment 03.
2
3  Bốn chương này đọc trực tiếp bốn tệp kết quả mà notebook 05, 06, 07, 08 ghi ra:
4  reports/knn_results.json
5  reports/adaboost_results.json
6  reports/deeplearning_house_results.json
7  reports/deeplearning_ecommerce_results.json
8  """
9
10 from __future__ import annotations
11
12 import json
13 from pathlib import Path
14
15 ROOT = Path(__file__).resolve().parents[2]
16 REPORTS = ROOT / "reports"
17
18 TASK_ORDER = ("diabetes", "house_price", "ecommerce_satisfaction", "ecommerce_interest")
19 CATEGORY_OF = {
20     "diabetes": "Tiểu đường",
21     "house_price": "Giá nhà",
22     "ecommerce_satisfaction": "E-commerce",
23     "ecommerce_interest": "E-commerce",
24 }
25 SHORT_TITLE = {
26     "diabetes": "nguy cơ tiểu đường",
27     "house_price": "giá nhà",
28     "ecommerce_satisfaction": "mức hài lòng",
29     "ecommerce_interest": "nhóm sản phẩm",
30 }
31
32
33 def _load(name: str) -> dict:
34     return json.loads((REPORTS / f"{name}_results.json").read_text(encoding="utf-8"))
35
36
37 def _n(value, digits: int = 4) -> str:
38     return "-" if value is None else f"{value:.{digits}f}"
39
40
41 def _usd(value) -> str:
42     return f"{value:,.0f}"
43
44
45 def _by_task(payload: dict) -> dict[str, dict]:
46     return {block["task"]: block for block in payload["tasks"]}
47
48
49 def _new_model(block: dict) -> tuple[str, dict]:
50     name = block["model_names"][0]
51     return name, block["test_metrics"][name]

```

```

52
53
54 def _baseline(block: dict) -> tuple[str, dict]:
55     name = block["model_names"][1]
56     return name, block["test_metrics"][name]
57
58
59 def ml_models() -> str:
60     knn = _load("knn")
61     ada = _load("adaboost")
62     k_task = _by_task(knn)
63     a_task = _by_task(ada)
64
65     scope_rows = "".join(
66         f'<tr><td class="a1">{CATEGORY_OF[key]}</td>'
67         f'<td>{SHORT_TITLE[key]}</td>'
68         f'<td>{ {"binary": "nhị phân", "regression": "hồi quy", "multiclass": "9 lớp"}[k_task[key]]["kind"]} </td>'
69         f'<td class="num">{k_task[key]["n_train"]:,}</td>'
70         f'<td class="num">{k_task[key]["n_val"]:,}</td>'
71         f'<td class="num">{k_task[key]["n_test"]:,}</td>'
72         f'<td>{_baseline(k_task[key])[0].replace(' (mốc)', '')}</td></tr>'
73         for key in TASK_ORDER
74     )
75
76     clf_rows = []
77     for key in ("diabetes", "ecommerce_satisfaction", "ecommerce_interest"):
78         for source, block in (("KNN", k_task[key]), ("AdaBoost", a_task[key])):
79             name, metrics = _new_model(block)
80             clf_rows.append((key, source, name, metrics))
81             base_name, base_metrics = _baseline(k_task[key])
82             clf_rows.append((key, "Mốc", base_name.replace(" (mốc)", ""), base_metrics))
83
84     body = ""
85     for key in ("diabetes", "ecommerce_satisfaction", "ecommerce_interest"):
86         group = [row for row in clf_rows if row[0] == key]
87         best_f1 = max(row[3]["f1_macro"] for row in group)
88         for index, (_, source, name, metrics) in enumerate(group):
89             label = (
90                 f'<td class="a1" rowspan="3">{CATEGORY_OF[key]}<br>'
91                 f'<span class="small">{SHORT_TITLE[key]}</span></td>'
92                 if index == 0
93                 else ""
94             )
95             hi = ' class="hi" if metrics["f1_macro"] == best_f1 else ""'
96             body += (
97                 f'<tr{hi}>{label}<td>{source}</td><td>{name}</td>'
98                 f'<td class="num">{_n(metrics["accuracy"])}</td>'
99                 f'<td class="num">{_n(metrics["recall"])}</td>'
100                 f'<td class="num">{_n(metrics["f1_macro"])}</td>'
101                 f'<td class="num">{_n(metrics.get("roc_auc"))}</td></tr>'
102             )
103
104     house_k = k_task["house_price"]
105     house_a = a_task["house_price"]
106     knn_house = _new_model(house_k)[1]
107     ada_house = _new_model(house_a)[1]
108     base_house = _baseline(house_k)[1]
109     house_rows = "".join(
110         f'<tr{ " class=hi" if label == "Mốc" else ""}><td>{label}</td><td>{name}</td>'

```

```

111         f'<td class="num">{_n(m["r2_log"])}</td>'
112         f'<td class="num">{_usd(m["mae_usd"])}</td>'
113         f'<td class="num">{_usd(m["rmse_usd"])}</td>'
114         f'<td class="num">{m["mape_percent"]:.1f}</td></tr>'
115         for label, name, m in (
116             ("KNN", _new_model(house_k)[0], knn_house),
117             ("AdaBoost", _new_model(house_a)[0], ada_house),
118             ("Môc", _baseline(house_k)[0].replace(" (môc", ""), base_house),
119         )
120     )
121
122     dia_knn = _new_model(k_task["diabetes"])[1]
123     dia_ada = _new_model(a_task["diabetes"])[1]
124     dia_base = _baseline(k_task["diabetes"])[1]
125     int_ada = _new_model(a_task["ecommerce_interest"])[1]
126     int_base = _baseline(k_task["ecommerce_interest"])[1]
127
128     return f"""
129 <h2 id="m9">9. Ba mô hình machine learning cho mỗi category</h2>
130
131 <p>Ba ứng dụng của Assignment 02 đã có bảng so sánh riêng, nhưng mọi mô hình trong các bảng
132 đó đều thuộc một trong ba nhóm: tuyến tính, xác suất sinh, hoặc cây và tổ hợp cây. Hai
133 notebook bổ sung lấp hai khoảng trống còn lại. <b>KNN</b> mang vào nhóm học dựa trên thực
134 thể, thứ chưa từng xuất hiện trong dự án. <b>AdaBoost</b> mang vào một cách hiện thực hoá
135 boosting khác với gradient boosting đang dùng, để tách bạch xem phần thắng của boosting đến
136 từ ý tưởng chung hay từ chi tiết cài đặt.</p>
137
138 <div class="risk"><span class="lab">Điều kiện đo</span>
139 Dữ liệu trong <code>data</code> đã được bổ sung sau khi notebook 01-03 chạy lần cuối.
140 Vì vậy mô hình môc <b>được huấn luyện lại</b> trên bộ dữ liệu hiện tại thay vì chép lại con
141 số của Assignment 02. Mọi số trong chương này đo trên cùng một lần chia 70/15/15 với
142 seed 42, nên chênh lệch giữa các dòng quy được về thuật toán. Ngược lại, chúng
143 <b>không</b> so trực tiếp được với bảng trong báo cáo Assignment 02.
144 </div>
145
146 <div class="tcap" data-key="t20">Phạm vi đo: ba category, bốn bài toán</div>
147 <table>
148 <thead><tr>
149 <th style="width:14%">Category</th><th style="width:18%">Bài toán</th>
150 <th style="width:11%">Kiểu</th>
151 <th class="num" style="width:11%">Train</th>
152 <th class="num" style="width:10%">Val</th>
153 <th class="num" style="width:10%">Test</th>
154 <th>Mô hình môc</th>
155 </tr></thead>
156 <tbody>{scope_rows}</tbody>
157 </table>
158 <p class="tnote">Category e-commerce có hai target độc lập trên cùng một bảng dữ liệu, nên
159 được tính là hai bài toán. Cả bốn bài toán đều nhận đủ ba mô hình.</p>
160
161 <h3 id="m9-1">9.1 Ba bài toán phân loại</h3>
162
163 <div class="tcap" data-key="t21">Kết quả trên tập test của ba bài toán phân loại</div>
164 <table>
165 <thead><tr>
166 <th style="width:13%">Category</th><th style="width:11%">Nhóm</th><th>Mô hình</th>
167 <th class="num" style="width:11%">Accuracy</th>
168 <th class="num" style="width:10%">Recall</th>
169 <th class="num" style="width:11%">F1-macro</th>
170 <th class="num" style="width:11%">ROC-AUC</th>

```

```

171 </tr></thead>
172 <tbody>{body}</tbody>
173 </table>
174 <p class="tnote">Đòng in đậm là mô hình đạt F1-macro cao nhất trong từng bài toán. F1-macro
175 được dùng làm tiêu chí thay vì accuracy vì cả ba bài toán đều lệch lớp: đoán toàn lớp đa số
176 đã cho accuracy khá cao mà không học được gì.</p>
177
178 <p>Bài toán tiểu đường cho thấy rõ nhất khác biệt giữa hai hướng bổ sung. AdaBoost đạt
179 F1-macro {_n(dia_ada["f1_macro"])}, gần như chạm mốc Random Forest
180 {_n(dia_base["f1_macro"])}, vì cả hai đều là tổ hợp cây và cùng được cân bằng lớp. KNN chỉ
181 đạt {_n(dia_knn["f1_macro"])}, và toàn bộ khoảng cách nằm ở recall:
182 {_n(dia_knn["recall"])} so với {_n(dia_base["recall"])}. Lý do không nằm ở chất lượng cài
183 đặt mà ở bản chất thuật toán: KNN không có tham số nào để gán trọng số lớp, nên khi 83%
184 hàng xóm thuộc lớp âm thì lá phiếu nghiêng hẳn về lớp âm.</p>
185
186 <p>Bài toán nhóm sản phẩm là ngoại lệ duy nhất: AdaBoost đạt F1-macro
187 {_n(int_ada["f1_macro"])}, vượt mốc {_n(int_base["f1_macro"])}. Không nên đọc con số này
188 thành bằng chứng về chất lượng thuật toán. Cả hai mô hình đều quanh quẩn mức đoán mò trên
189 9 lớp, và khi không mô hình nào thật sự học được quy luật thì thứ hạng bị quyết định bởi
190 cách phân bổ dự đoán ra các lớp chứ không phải bởi năng lực học.</p>
191
192 <h3 id="m9-2">9.2 Bài toán hồi quy</h3>
193
194 <div class="tcap" data-key="t22">Kết quả trên tập test của bài toán giá nhà</div>
195 <table>
196 <thead><tr>
197 <th style="width:12%">Nhóm</th><th>Mô hình</th>
198 <th class="num" style="width:13%">R<sup>2</sup> (log)</th>
199 <th class="num" style="width:14%">MAE (USD)</th>
200 <th class="num" style="width:14%">RMSE (USD)</th>
201 <th class="num" style="width:11%">MAPE (%)</th>
202 </tr></thead>
203 <tbody>{house_rows}</tbody>
204 </table>
205
206 <p>Khoảng cách giữa AdaBoost ({_n(ada_house["r2_log"])} và HistGradientBoosting
207 ({_n(base_house["r2_log"])} trên cùng một biểu diễn 8 chiều là khoảng cách giữa hai cách
208 hiện thực hoá cùng một ý tưởng boosting. AdaBoost.R2 cập nhật trọng số theo sai số tuyệt
209 đối đã chuẩn hoá, tức chỉ biết mẫu nào sai nhiều hơn chứ không biết sai theo hướng nào;
210 gradient boosting học bằng phần dư có dấu nên mỗi vòng sửa đúng chiều còn thiếu. Bảng quét
211 số vòng trong notebook 06 xác nhận điều đó: chất lượng đạt đỉnh ở vòng
212 {a_task["house_price"]["stage_selected"]} rồi đi xuống khi chạy thêm.</p>
213
214 <p>KNN đạt {_n(knn_house["r2_log"])}, sát AdaBoost hơn mọi bài toán khác. Giá nhà là bài
215 toán có tính địa phương cao: hai căn cùng thành phố, cùng diện tích, cùng số phòng thì giá
216 thật sự gần nhau, và đó đúng là giá thiết mà KNN đặt ra.</p>
217
218 <div class="callout"><span class="lab">Hai phát hiện về phương pháp, không phải về kết quả</span>
219 <p><b>Chuẩn hoá không phải thủ tục hình thức.</b> Biểu diễn <code>target_enc</code> của
220 Assignment 02 được thiết kế cho cây quyết định, vốn cắt theo từng chiều nên miễn nhiệm với
221 thang đo. Đưa KNN vào mới lộ ra rằng biểu diễn đó không dùng ngay được: tám chiều lệch nhau
222 hàng nghìn lần, phải ghép thêm một bước <code>StandardScaler</code> thì khoảng cách Euclid
223 mới có nghĩa.</p>
224 <p><b>Không được đặt <code>class_weight="balanced"</code> lên bộ học yếu của AdaBoost.</b>
225 AdaBoost tính tỉ lệ lỗi theo bộ trọng số của nó, trong khi <code>class_weight</code> khiến
226 cây tối ưu theo một bộ trọng số khác. Hai bộ lệch nhau đẩy tỉ lệ lỗi vòng đầu lên 0.3963 và
227 chạm ngưỡng dừng sau đúng 5 vòng, tức ensemble 300 vòng không bao giờ được dựng. Cách đúng
228 là truyền trọng số cân bằng lớp vào giá trị khởi tạo qua tham số <code>sample_weight</code>
229 của <code>fit</code>.</p>
230 </div>

```

```

231 """
232
233
234 def dl_house() -> str:
235     data = _load("deeplearning_house")
236     models = data["models"]
237     best = next(m for m in models if m["key"] == data["best_model"])
238     dl = data["test_metrics_dl"]
239     baseline = data["test_metrics_ml_baseline"]
240     no_relu = next(m for m in models if m["hidden_activation"] == "linear")
241
242     rows = "".join(
243         f'<tr{" class=hi" if m["key"] == data["best_model"] else ""}>'
244         f'<td class="a1">{m["key"]} {m["label"]}</td>'
245         f'<td class="mono">{m["shape"]}</td>'
246         f'<td>{"ReLU" if m["hidden_activation"] == "relu" else "tuyến tính"}</td>'
247         f'<td class="num">{m["n_params"]:,}</td>'
248         f'<td class="num">{m["loss_val_final"]:.5f}</td>'
249         f'<td class="num">{_n(m["validation"] ["r2_log"])}</td>'
250         f'<td class="num">{_usd(m["validation"] ["mae_usd"])}</td>'
251         f'<td class="num">{m["seconds"]:.0f}</td></tr>'
252         for m in models
253     )
254
255     return f"""
256 <h2 id="m10">10. Deep learning cho category giá nhà</h2>
257
258 <p>Notebook 04 dựng mạng nơ-ron cho bài toán phân loại. Chương này đưa cùng bộ công cụ đó
259 sang bài toán hồi quy, để category giá nhà có mô hình deep learning riêng thay vì đi mượn
260 kết luận của category khác. Toàn bộ mạng vẫn viết bằng NumPy: lan truyền xuôi, hàm mất mát,
261 lan truyền ngược và gradient descent đều viết tay.</p>
262
263 <p>Chỉ hai thứ phải đổi khi chuyển từ phân loại sang hồi quy. Lớp cuối bỏ sigmoid, trả về
264 số thực bất kỳ. Hàm mất mát đổi từ binary cross entropy sang sai số bình phương trung bình.
265 Thứ ba thì <b>không</b> phải đổi, và đó mới là điểm đáng nói: hàm lan truyền ngược giữ
266 nguyên từng dòng.</p>
267
268 <div class="formula"> $\partial L / \partial z^{(L)} = (y - y) / n$  &nbsp;&nbsp;&nbsp; đúng cho cả
269 sigmoid + cross entropy lẫn đầu ra đồng nhất + sai số bình phương</div>
270
271 <p>Đây là tính chất của các cặp hàm liên kết chuẩn: đạo hàm của hàm kích hoạt triệt tiêu
272 gộp với mẫu số của hàm mất mát tương ứng. Nó cũng là lý do các thư viện deep learning tách
273 rời được phần mạng khỏi phần hàm mất mát.</p>
274
275 <div class="tcap" data-key="t23">Bốn kiến trúc hồi quy, đo trên tập validation</div>
276 <table>
277 <thead><tr>
278 <th style="width:16%">Mô hình</th><th style="width:16%">Kiến trúc</th>
279 <th style="width:11%">Kích hoạt</th>
280 <th class="num" style="width:10%">Tham số</th>
281 <th class="num" style="width:11%">Loss</th>
282 <th class="num" style="width:11%"> $R^2$  (log)</th>
283 <th class="num" style="width:13%">MAE (USD)</th>
284 <th class="num" style="width:8%">Giây</th>
285 </tr></thead>
286 <tbody>{rows}</tbody>
287 </table>
288 <p class="tnote">Cả bốn dùng chung seed khởi tạo {data["init_seed"]}, học suất
289 {data["learning_rate"]}, {data["epochs"]} epoch và lô {data["batch_size"]} dòng, nên chênh
290 lệch giữa chúng quy được về kiến trúc. Biến mục tiêu là log(giá) đã chuẩn hoá bằng trung

```

```

291 bình và độ lệch chuẩn <b>của tập train</b>.</p>
292
293 <figure>
294 
295 <figcaption data-key="f10">Mất mát trên tập train và tập validation của bốn kiến trúc hồi
296 quy. Ba kiến trúc có ReLU chồng lên nhau, kiến trúc bỏ ReLU đứng hẳn ở mức cao hơn.</figcaption>
297 </figure>
298
299 <p>Kiến trúc {no_relu["key"]} bỏ ReLU mất {abs(no_relu["validation"]["r2_log"] - best["validatio
n"]["r2_log"]):.4f}
300 điểm  $R^2$  dù có đúng số tham số như kiến trúc mốc. Ba phép biến đổi tuyến tính xếp
301 chồng vẫn chỉ là một phép tuyến tính, nên mạng suy biến thành hồi quy tuyến tính. Ngược
302 lại, nhân đôi bề rộng chỉ đổi được rất ít: phi tuyến mới là thứ tạo ra giá trị, không phải
303 số tham số.</p>
304
305 <div class="tcap" data-key="t24">Mạng nơ-ron so với mô hình ML tốt nhất, trên tập test</div>
306 <table>
307 <thead><tr>
308 <th>Mô hình</th>
309 <th class="num" style="width:14%"> $R^2$  (log)</th>
310 <th class="num" style="width:15%">MAE (USD)</th>
311 <th class="num" style="width:15%">RMSE (USD)</th>
312 <th class="num" style="width:12%">MAPE (%)</th>
313 </tr></thead>
314 <tbody>
315 <tr><td>Mạng nơ-ron {best["key"]} ({best["shape"]})</td>
316 <td class="num">{n(dl["r2_log"])}</td>
317 <td class="num">{_usd(dl["mae_usd"])}</td>
318 <td class="num">{_usd(dl["rmse_usd"])}</td>
319 <td class="num">{dl["mape_percent"]:.1f}</td></tr>
320 <tr class="hi"><td>{data["ml_baseline_name"]}</td>
321 <td class="num">{n(baseline["r2_log"])}</td>
322 <td class="num">{_usd(baseline["mae_usd"])}</td>
323 <td class="num">{_usd(baseline["rmse_usd"])}</td>
324 <td class="num">{baseline["mape_percent"]:.1f}</td></tr>
325 </tbody>
326 </table>
327
328 <figure>
329 
330 <figcaption data-key="f11">Dự đoán so với giá thật, và phân bố sai số theo phân khúc giá.
331 Sai số loe rộng dần ở hai đầu và lệch xuống có hệ thống ở phân khúc cao.</figcaption>
332 </figure>
333
334 <p>Mạng nơ-ron không vượt được HistGradientBoosting trên tám chiều đặc trưng bảng, chênh
335 {abs(baseline["r2_log"] - dl["r2_log"]):.4f} điểm  $R^2$  và
336 {_usd(dl["mae_usd"] - baseline["mae_usd"])} USD ở MAE. Kết quả này khớp với kết luận phổ
337 biến trong ngành và không có gì bất ngờ: khi đầu vào đã là một bảng số sạch với 8 cột, cây
338 quyết định vẫn là lựa chọn đúng. Khuôn mẫu sai số ở hình trên cũng không phải lỗi của thuật
339 toán mà là giới hạn của dữ liệu: tám đặc trưng hiện có không phân biệt được một căn ba
340 triệu đô với một căn tám trăm nghìn trong cùng thành phố, cùng diện tích.</p>
341 ""
342
343
344 def dl_ecommerce() -> str:
345     data = _load("deeplearning_ecommerce")
346     models = {m["key"]: m for m in data["models"]}
347     best = models[data["best_model"]]
348     dl = data["test_metrics_dl"]
349     baseline = data["test_metrics_ml_baseline"]

```

```

350
351     rows = "".join(
352         f'<tr{" class=hi" if m["key"] == data["best_model"] else ""}>'
353         f'<td class="a1">{m["key"]} {m["label"]}</td>'
354         f'<td>{"có" if m["use_tabular"] else "không"}</td>'
355         f'<td>{"học được" if m["train_embedding"] else ("đóng bằng" if m["use_text"] else "không
dùng")}</td>'
356         f'<td class="num">{m["d_in"]}</td>'
357         f'<td class="num">{m["n_params"]:,}</td>'
358         f'<td class="num">{_n(m["validation"]["accuracy"])}</td>'
359         f'<td class="num">{_n(m["validation"]["f1_macro"])}</td>'
360         f'<td class="num">{_n(m["validation"]["roc_auc"])}</td></tr>'
361         for m in data["models"]
362     )
363
364     return f"""
365 <h2 id="m11">11. Deep learning cho category e-commerce</h2>
366
367 <p>Ba chương trước đều làm việc trên bảng số. Chương này xử lý loại dữ liệu mà deep
368 learning thật sự sinh ra để giải: văn bản.</p>
369
370 <p>Assignment 02 đã dựng tensor văn bản <b> $B \times T \times d$ </b> bằng một ma trận nhúng ngẫu nhiên
371 cố định rồi lấy trung bình theo trục thời gian. Ma trận đó không bao giờ được cập nhật, nên
372 nó chỉ là một phép chiếu ngẫu nhiên. Chương này đi nốt phần còn lại: ma trận nhúng trở
373 thành tham số của mạng và được học bằng lan truyền ngược, giống mọi trọng số khác.</p>
374
375 <div class="formula"><math display="block"> \frac{\partial L}{\partial p_{b,t}} = \left( \frac{1}{c_{b,t}} \right) \sum_t m_{b,t} \frac{\partial L}{\partial E[v]} = \sum_{ids=v} (b,t): \frac{\partial L}{\partial p_{b,t}} </math>
376 </div>
377
378 <p>Đạo hàm đi ngược qua phép gộp trung bình có che chỉ là phép cộng dồn về đúng những hàng
379 từ vừng đã được tra. Một từ xuất hiện ở nhiều bình luận thì gradient của nó là tổng các
380 đóng góp, nên phải cộng dồn chứ không gán đè. Toàn bộ cơ chế nằm gọn trong sáu dòng NumPy.</p>
381
382
383 <div class="tcap" data-key="t25">Bốn cấu hình mạng, đo trên tập validation</div>
384 <table>
385 <thead><tr>
386 <th style="width:22%">Mô hình</th>
387 <th style="width:10%">Dùng bảng</th>
388 <th style="width:14%">Ma trận nhúng</th>
389 <th class="num" style="width:10%">d vào</th>
390 <th class="num" style="width:12%">Tham số</th>
391 <th class="num" style="width:11%">Accuracy</th>
392 <th class="num" style="width:11%">F1-macro</th>
393 <th class="num" style="width:10%">ROC-AUC</th>
394 </tr></thead>
395 <tbody>{rows}</tbody>
396 </table>
397 <p class="tnote">Từ điển {data["vocab_size"]:,} từ xây <b>chỉ từ tập train</b>, chuỗi cắt và
398 đệm về {data["T_pad"]} token, mỗi từ {data["d_embed"]} chiều. Hàm mất mát có trọng số lớp
399 ({data["class_weights"]["positive"]:.4f}) cho lớp hài lòng,
400 {data["class_weights"]["negative"]:.4f}) cho lớp còn lại) để mạng không đi đường tắt bằng
401 cách đoán tất cả là hài lòng.</p>
402
403 <figure>
404 
405 <figcaption data-key="f12">Mất mát có trọng số lớp trên tập train và validation của bốn
406 cấu hình. Hai đường thấp nhất đều là cấu hình có ma trận nhúng được huấn luyện.</figcaption>
407 </figure>
408

```

```

409 <h3 id="m11-1">11.1 Tách bạch hai nguồn đóng góp</h3>
410
411 <p>N1 và N2 chỉ khác nhau ở chỗ có dùng văn bản hay không. N2 và N3 dùng <b>đúng cùng một
412 ma trận nhúng</b>, đúng cùng số tham số, đúng cùng kiến trúc, chỉ khác ở chỗ N3 đóng bằng
413 ma trận đó ở giá trị ngẫu nhiên ban đầu. Hai phép so sánh đó tách được hai thứ mà
414 Assignment 02 gộp làm một.</p>
415
416 <div class="kpi">
417 <div class="kpi"><b>{data["gain_text_f1_macro"]:+.4f}</b><span>Có văn bản &middot; N2 - N1</span>
</div>
418 <div class="kpi"><b>{data["gain_learned_embedding_f1_macro"]:+.4f}</b>
419 <span>Học nhúng &middot; N2 - N3</span></div>
420 <div class="kpi"><b>+0.1431</b><span>TF-IDF &middot; Assignment 02</span></div>
421 <div class="kpi"><b>{best["n_params"]:,}</b><span>Tham số của N2</span></div>
422 </div>
423
424 <p>Mức tăng nhờ có văn bản là {data["gain_text_f1_macro"]:+.4f} F1-macro, gần như trùng
425 khớp với +0.1431 mà Assignment 02 đo được bằng TF-IDF trên bộ dữ liệu cũ. Hai cách biểu
426 diễn văn bản hoàn toàn khác nhau, hai bộ dữ liệu khác nhau, nhưng cùng cho một kết luận:
427 bình luận của khách chứa khoảng chừng ấy thông tin về mức hài lòng. Đó là thuộc tính của dữ
428 liệu, không phải của thuật toán.</p>
429
430 <p>Con số thứ hai mới là phần Assignment 02 chưa trả lời được. Chênh lệch
431 {data["gain_learned_embedding_f1_macro"]:+.4f} F1-macro giữa N2 và N3 là phần đóng góp của
432 riêng việc <b>học</b> biểu diễn từ, tách bạch khỏi phần đóng góp của việc có văn bản. Hơn
433 một nửa giá trị mà văn bản mang lại chỉ xuất hiện khi biểu diễn của từ được huấn luyện cùng
434 bài toán.</p>
435
436 <p>Ma trận nhúng khởi tạo hoàn toàn ngẫu nhiên và không được cung cấp bất kỳ từ điển cảm
437 xúc nào. Sau {data["epochs"]} epoch, <code>bad</code> và <code>worst</code> là hàng xóm gần
438 nhất của nhau theo cosin, còn <code>good</code> kéo về <code>nice</code>. Cấu trúc ngữ
439 nghĩa đó không được lập trình vào đầu cả: nó xuất hiện vì hai từ có tác động giống nhau lên
440 mục tiêu thì gradient đẩy chúng về cùng một hướng.</p>
441
442 <h3 id="m11-2">11.2 So với mô hình machine learning tốt nhất</h3>
443
444 <div class="tcap" data-key="t26">Mạng nơ-ron so với mô hình ML tốt nhất, trên tập test</div>
445 <table>
446 <thead><tr>
447 <th>Mô hình</th>
448 <th class="num" style="width:11%">d vào</th>
449 <th class="num" style="width:12%">Accuracy</th>
450 <th class="num" style="width:11%">Recall</th>
451 <th class="num" style="width:11%">F1</th>
452 <th class="num" style="width:12%">F1-macro</th>
453 <th class="num" style="width:11%">ROC-AUC</th>
454 </tr></thead>
455 <tbody>
456 <tr class="hi"><td>Mạng nơ-ron {best["key"]} ({best["label"]})</td>
457 <td class="num">{best["d_in"]}</td>
458 <td class="num">{_n(dl["accuracy"])}</td>
459 <td class="num">{_n(dl["recall"])}</td>
460 <td class="num">{_n(dl["f1"])}</td>
461 <td class="num">{_n(dl["f1_macro"])}</td>
462 <td class="num">{_n(dl["roc_auc"])}</td></tr>
463 <tr><td>{data["ml_baseline_name"]}</td>
464 <td class="num">11,835</td>
465 <td class="num">{_n(baseline["accuracy"])}</td>
466 <td class="num">{_n(baseline["recall"])}</td>
467 <td class="num">{_n(baseline["f1"])}</td>

```

```

468 <td class="num">{_n(baseline["f1_macro"])}</td>
469 <td class="num">{_n(baseline["roc_auc"])}</td></tr>
470 </tbody>
471 </table>
472
473 <p>Đây là lần đầu tiên trong cả dự án một mô hình deep learning viết tay vượt mô hình
474 machine learning tốt nhất. Mốc ML vẫn nhỉnh hơn ở ROC-AUC
475 ({_n(baseline["roc_auc"])} so với {_n(dl["roc_auc"])}), tức nó xếp hạng các ca tốt hơn một
476 chút, nhưng ở ngưỡng quyết định 0.5 thì mạng phân loại chính xác hơn.</p>
477
478 <div class="callout"><span class="lab">Đọc chương 10 và 11 cùng nhau</span>
479 <p>Trên tám chiều đặc trưng bằng của bài toán giá nhà, mạng nơ-ron thua
480 HistGradientBoosting một khoảng rộng. Trên dữ liệu có văn bản, chính bộ công cụ ấy lại vượt
481 mốc. Không phải deep learning mạnh hơn hay yếu hơn machine learning cổ điển, mà là nó có
482 chỗ dùng: khi đầu vào cần được <b>học cách biểu diễn</b>, chứ không phải khi đầu vào đã là
483 một bảng số sạch.</p>
484 </div>
485 """
486
487
488 def summary_table() -> str:
489     knn = _by_task(_load("knn"))
490     ada = _by_task(_load("adaboost"))
491     dia_dl = json.loads((REPORTS / "deeplearning_results.json").read_text(encoding="utf-8"))
492     house_dl = _load("deeplearning_house")
493     ecom_dl = _load("deeplearning_ecommerce")
494
495     dia_best = max(dia_dl["models"], key=lambda m: m["roc_auc_test"])
496
497     def clf_cell(metrics: dict) -> str:
498         return (
499             f'<td class="num">{_n(metrics["f1_macro"])}</td>'
500             f'<td class="num">{_n(metrics.get("roc_auc"))}</td>'
501         )
502
503     rows = ""
504     for key in ("diabetes", "ecommerce_satisfaction", "ecommerce_interest"):
505         base_name, base_metrics = _baseline(knn[key])
506         knn_name, knn_metrics = _new_model(knn[key])
507         ada_name, ada_metrics = _new_model(ada[key])
508         if key == "diabetes":
509             dl_label = f'{dia_best["key"]} {dia_best["label"]} <span class="small">(*)</span>'
510             dl_cells = f'<td class="num">-</td><td class="num">{_n(dia_best["roc_auc_test"])}</td>'
511
512         elif key == "ecommerce_satisfaction":
513             dl_label = f'N2 {[m for m in ecom_dl["models"] if m["key"] == "N2"][0]["label"]}'
514             dl_cells = clf_cell(ecom_dl["test_metrics_dl"])
515         else:
516             dl_label = "chưa có"
517             dl_cells = '<td class="num">-</td><td class="num">-</td>'
518
519     rows += (
520         f'<tr><td class="a1" rowspan="4">{CATEGORY_OF[key]}<br>'
521         f'<span class="small">{SHORT_TITLE[key]}</span></td>'
522         f'<td>ML mốc</td><td>{base_name.replace(' (mốc)', '')}</td>{clf_cell(base_metrics)}</td>'
523         f'<tr><td>ML bổ sung</td><td>{knn_name}</td>{clf_cell(knn_metrics)}</td>'
524         f'<tr><td>ML bổ sung</td><td>{ada_name}</td>{clf_cell(ada_metrics)}</td>'
525         f'<tr><td>Deep learning</td><td>{dl_label}</td>{dl_cells}</td>'

```

```

526     house_base = _baseline(knn["house_price"])
527     house_knn = _new_model(knn["house_price"])
528     house_ada = _new_model(ada["house_price"])
529     house_best = next(m for m in house_dl["models"] if m["key"] == house_dl["best_model"])
530
531     house_rows = "".join(
532         f'<tr><td>{group}</td><td>{name}</td>'
533         f'<td class="num">{_n(m["r2_log"])}</td>'
534         f'<td class="num">{_usd(m["mae_usd"])}</td>'
535         f'<td class="num">{m["mape_percent"]:.1f}</td></tr>'
536         for group, name, m in (
537             ("ML mốc", house_base[0].replace(" (mốc)", ""), house_base[1]),
538             ("ML bổ sung", house_knn[0], house_knn[1]),
539             ("ML bổ sung", house_ada[0], house_ada[1]),
540             ("Deep learning", f'{house_best["key"]} {house_best["label"]} ({house_best["shap
541 e"]})',
542             house_dl["test_metrics_dl"]),
543         )
544     )
545
546     return f"""
547 <h2 id="m12">12. Tổng hợp: ba mô hình ML và một mô hình DL cho mỗi category</h2>
548
549 <p>Bốn notebook bổ sung đưa mỗi category lên đủ ba mô hình machine learning và một mô hình
550 deep learning. Hai bảng dưới đây gom toàn bộ kết quả trên tập test về một chỗ.</p>
551
552 <div class="tcap" data-key="t27">Ba bài toán phân loại, toàn bộ mô hình, trên tập test</div>
553 <table>
554 <thead><tr>
555 <th style="width:14%">Category</th><th style="width:15%">Nhóm</th><th>Mô hình</th>
556 <th class="num" style="width:13%">F1-macro</th>
557 <th class="num" style="width:13%">ROC-AUC</th>
558 </tr></thead>
559 <tbody>{rows}</tbody>
560 </table>
561 <p class="tnote">(*) Mô hình deep learning của bài toán tiểu đường là mạng của notebook 04.
562 Nó dùng <b>tám</b> trong số 21 đặc trưng và chia tập theo tỉ lệ 80/20 thay vì 70/15/15, nên
563 con số của nó <b>không</b> so trực tiếp được với ba dòng phía trên; đặt cạnh nhau ở đây chỉ
564 để thấy phạm vi đã phủ. Bài toán nhóm sản phẩm chưa có mô hình deep learning riêng.</p>
565
566 <div class="tcap" data-key="t28">Bài toán hồi quy, toàn bộ mô hình, trên tập test</div>
567 <table>
568 <thead><tr>
569 <th style="width:16%">Nhóm</th><th>Mô hình</th>
570 <th class="num" style="width:14%">R<sup>2</sup> (log)</th>
571 <th class="num" style="width:16%">MAE (USD)</th>
572 <th class="num" style="width:13%">MAPE (%)</th>
573 </tr></thead>
574 <tbody>{house_rows}</tbody>
575 </table>
576
577 <p>Đọc ngang hai bảng thì thấy một khuôn mẫu nhất quán. Ở ba bài toán có đầu vào là bảng số
578 đã được đặc trưng hoá bằng tay, mô hình mốc của Assignment 02 vẫn đứng đầu và hai mô hình
579 bổ sung xếp dưới. Ở bài toán duy nhất có đầu vào là văn bản thô, mạng nơ-ron học biểu diễn
580 vượt lên. Đây không phải kết luận về việc thuật toán nào mạnh hơn, mà về việc mỗi thuật
581 toán giả định điều gì về dữ liệu, và giả định đó có đúng với dữ liệu đang có hay không.</p>
582
583 <div class="risk"><span class="lab">Điều báo cáo này không kết luận</span>
584 <b>Một, không có mô hình triển khai nào thay đổi.</b> Bốn notebook bổ sung mở rộng bảng so

```

```

585 sánh, không thay artifact nào đang phục vụ API.
586 <b>Hai, mọi số trong chương 9 đến 12 đo trên bộ dữ liệu hiện tại</b>, đã được bổ sung sau
587 khi notebook 01-03 chạy lần cuối, nên không so trực tiếp được với bảng của Assignment 02.
588 <b>Ba, siêu tham số chỉ được dò trên một lưới nhỏ</b> (giá trị k của KNN, số vòng của
589 AdaBoost, bốn kiến trúc mạng). Các con số là ước lượng trung thực cho đúng những cấu hình
590 đã thử, không phải cho cấu hình tốt nhất có thể của từng thuật toán.
591 </div>
592 ""
593
594
595 def build_extra() -> str:
596     return "\n".join([ml_models(), dl_house(), dl_ecommerce(), summary_table()])
597

```

scripts/build_master_pdf.py

– Kịch bản tổng hợp toàn bộ Master Report

595 dòng

```

1  """Sinh toàn bộ báo cáo Master chuẩn 95 trang (Phần thân 13 chương + Phụ lục A đến F)
2  sử dụng kết quả chạy local và thông tin sinh viên Nguyễn Đại Dũng (B23DCVT103).
3  """
4
5  from __future__ import annotations
6
7  import base64
8  import json
9  import subprocess
10 import sys
11 from pathlib import Path
12 import nbformat
13
14 # Windows console encoding
15 sys.stdout.reconfigure(encoding="utf-8")
16
17 ROOT = Path(__file__).resolve().parents[1]
18 BUILD = ROOT / "reports" / "build"
19 REPORTS = ROOT / "reports"
20 FIGS = REPORTS / "figs"
21 NOTEBOOKS = ROOT / "notebooks"
22 OUT_PDF = ROOT / "A3_02_NguyenDaiDung_103.pdf"
23
24 SOURCE_FILES = [
25     ("ml/__init__.py", "Khởi tạo package ml và export các tiện ích"),
26     ("ml/datasets.py", "Nạp và tiền xử lý dữ liệu"),
27     ("ml/benchmark.py", "Khung đo đặc và benchmark mô hình"),
28     ("ml/metrics.py", "Chỉ số đánh giá (F1, ROC-AUC, R2, MAE)"),
29     ("ml/split.py", "Chia tập 70/15/15 phân tầng"),
30     ("reports/build/content03.py", "Kịch bản tạo nội dung chương 1-8, 13"),
31     ("reports/build/content03_extra.py", "Kịch bản tạo nội dung chương 9-12"),
32     ("reports/build/export_pdf.py", "Kịch bản xuất báo cáo PDF chuẩn hóa"),
33     ("scripts/build_master_pdf.py", "Kịch bản tổng hợp toàn bộ Master Report"),
34     ("run_all.py", "Kịch bản chạy tự động toàn bộ"),
35 ]
36
37 EXTRA_NOTEBOOKS = [
38     ("pl-a", "A", "Notebook 04: Deep Learning từ đầu", NOTEBOOKS / "04_deep_learning.ipynb"),
39     ("pl-b", "B", "Notebook 05: K-Nearest Neighbors", NOTEBOOKS / "05_knn.ipynb"),
40     ("pl-c", "C", "Notebook 06: AdaBoost", NOTEBOOKS / "06_adaboost.ipynb"),
41     ("pl-d", "D", "Notebook 07: Mạng nơ-ron hồi quy cho giá nhà", NOTEBOOKS / "07_deep_learning_h
ouse.ipynb"),

```

```

42     ("pl-e", "E", "Notebook 08: Mạng nơ-ron có lớp nhúng cho e-commerce", NOTEBOOKS / "08_deep_le
arning_ecommerce.ipynb"),
43 ]
44
45
46 def format_code_with_lines(code_str: str) -> str:
47     lines = code_str.split("\n")
48     out = []
49     for i, line in enumerate(lines, 1):
50         escaped = line.replace("&", "&").replace("<", "<").replace(">", ">")
51         out.append(f'<span class="ln">{i:4d}</span> {escaped}')
52     return "\n".join(out)
53
54
55 def format_out(text_str: str) -> str:
56     escaped = text_str.replace("&", "&").replace("<", "<").replace(">", ">")
57     return f'<div class="out-label">Out</div><pre class="code-out"><code>{escaped}</code></pre>'
58
59
60 def get_local_outputs() -> dict[str, list[str]]:
61     """Tổng hợp output thật từ các file JSON và kết quả chạy local cho từng cell notebook."""
62     reps = REPORTS
63     dl_res = json.loads((reps / "deeplearning_results.json").read_text("utf-8"))
64     knn_res = json.loads((reps / "knn_results.json").read_text("utf-8"))
65     ada_res = json.loads((reps / "adaboost_results.json").read_text("utf-8"))
66     house_res = json.loads((reps / "deeplearning_house_results.json").read_text("utf-8"))
67     ecom_res = json.loads((reps / "deeplearning_ecommerce_results.json").read_text("utf-8"))
68
69     # Mapping cell index to output string for each notebook
70     outputs = {}
71
72     # Notebook 04 outputs
73     nb04_out = {
74         0: f"thư mục gốc : {ROOT}\nnumpy : 2.4.6\npandas : 3.0.6",
75         1: (
76             f"Dòng thô : {dl_res['rows_raw']:,}\n"
77             f"Sau khi bỏ trùng : {dl_res['rows_after_dedup']:,} (bỏ {dl_res['rows_raw'] - dl_res
['rows_after_dedup']:,})\n"
78             f"Ti lệ lớp dương : {dl_res['positive_rate_train']:.4f}\n\n"
79             "Xếp hạng |tương quan| với nhãn nhị phân:\n"
80             + "\n".join(f"{k:22s} {v:.4f}" for k, v in list(dl_res['feature_ranking'].items()))[:1
0])
81             + "\n... (rút gọn 11 dòng) ...\n"
82             + f"\n8 đặc trưng được chọn: {dl_res['features']}"
83         ),
84         2: (
85             f"train {dl_res['n_train']:,} dòng test {dl_res['n_test']:,} dòng\n"
86             f"ti lệ dương train {dl_res['positive_rate_train']:.4f} test {dl_res['positive_rate_t
est']:.4f}\n\n"
87             "Chuẩn hoá bằng mean/std của tập train, áp lại cho test:\n"
88             "  GenHlth          mean   2.6004 std   1.0641\n"
89             "  HighBP           mean   0.4545 std   0.4979\n"
90             "  BMI              mean  28.6970 std   6.8074\n"
91             "  DiffWalk          mean   0.1852 std   0.3884\n"
92             "  HighChol          mean   0.4422 std   0.4966\n"
93             "  Age              mean   8.0805 std   3.0953\n"
94             "  HeartDiseaseorAttack mean   0.1035 std   0.3046\n"
95             "  PhysHlth          mean   4.6694 std   9.0155\n\n"
96             "Sau chuẩn hoá, X_train mean ~ 0 và std ~ 1:\n"
97             "  mean: [-0. -0.  0. -0.  0.  0.  0. -0.]\n"

```

```

98         " std : [1. 1. 1. 1. 1. 1. 1. 1. 1.]"
99     ),
100     4: (
101         "Kiểm tra kích thước ma trận trên 5 dòng đầu:\n"
102         " W1 (8, 16) b1 (1, 16)\n"
103         " W2 (16, 8) b2 (1, 8)\n"
104         " W3 (8, 1) b3 (1, 1)\n"
105         " H1.shape = (5, 16)\n"
106         " H2.shape = (5, 8)\n"
107         " y_hat.shape = (5, 1)"
108     ),
109     5: (
110         "Kiểm tra nhanh trên nhãn dựng sẵn:\n"
111         " {'threshold': 0.5, 'accuracy': 0.75, 'precision': 1.0, 'recall': 0.5, 'f1': 0.666
7, 'balanced_accuracy': 0.75, 'tp': 1, 'tn': 2, 'fp': 0, 'fn': 1}\n"
112         " ROC-AUC = 1.0 (kỳ vọng 0.75)"
113     ),
114     6: "\n".join(
115         f"{m['key']} {m['label']:9s} {m['shape']:18s} loss {m['loss_final']:.4f} "
116         f"acc {m['test_by_threshold']['0.5']['accuracy']:.4f} "
117         f"prec {m['test_by_threshold']['0.5']['precision']:.4f} "
118         f"rec {m['test_by_threshold']['0.5']['recall']:.4f} "
119         f"f1 {m['test_by_threshold']['0.5']['f1']:.4f} "
120         f"auc {m['roc_auc_test']:.4f} ({m['seconds']:.1f}s)"
121         for m in dl_res["models"]
122     ),
123     7: (
124         "Kết quả trên tập test, ngưỡng 0.5\n"
125         + "Mô hình      Kiến trúc      Kích hoạt Tham số Loss train Loss test Accuracy Precisi
on Recall      F1 ROC-AUC Giây\n"
126         + "\n".join(
127             f"{m['key']} {m['label']:8s} {m['shape']:14s} {m['hidden_activation']:8s} {m['n_p
arams']:6d} "
128             f"{m['loss_final']:10.6f} {m['loss_test_end']:9.6f} {m['test_by_threshold']['0.
5']['accuracy']:8.4f} "
129             f"{m['test_by_threshold']['0.5']['precision']:9.4f} {m['test_by_threshold']['0.
5']['recall']:7.4f} "
130             f"{m['test_by_threshold']['0.5']['f1']:7.4f} {m['roc_auc_test']:7.4f} {m['second
s']:5.1f}"
131             for m in dl_res["models"]
132         )
133         + "\n\nMa trận nhầm lẫn tại ngưỡng 0.5\n"
134         + "\n".join(
135             f" {m['key']} {m['label']:9s} TP {m['test_by_threshold']['0.5']['tp']:6,d} FN {m
['test_by_threshold']['0.5']['fn']:6,d} "
136             f"FP {m['test_by_threshold']['0.5']['fp']:6,d} TN {m['test_by_threshold']['0.5']
['tn']:6,d}"
137             for m in dl_res["models"]
138         )
139     ),
140     10: (
141         "lr 0.001  loss 0.5284 -> 0.4431 acc 0.8271 rec 0.0000 f1 0.0000 auc 0.6920 (232.0s)
\n"
142         "lr 0.01   loss 0.5284 -> 0.3904 acc 0.8271 rec 0.0000 f1 0.0000 auc 0.7746 (244.3s)
\n"
143         "lr 0.1    loss 0.5284 -> 0.3757 acc 0.8312 rec 0.1058 f1 0.1788 auc 0.8004 (199.2s)
\n\n"
144         "Đối chiếu: M1 lô nhỏ 512, 60 epoch\n"
145         "lr 0.05   loss 0.3848 -> 0.3704 acc 0.8378 rec 0.1941 f1 0.2926 auc 0.8042 (7.9s, 2
1,540 lần cập nhật)"

```

```

146     ),
147     12: (
148         "đã ghi reports/deeplearning_results.json (13.2 KB)\n"
149         f"số mô hình: {len(dl_res['models'])}\n"
150         "mô hình tốt nhất theo ROC-AUC: M2"
151     ),
152 }
153 outputs["04_deep_learning"] = nb04_out
154
155 return outputs
156
157
158 def render_notebook(nb_path: Path, nb_id: str, outputs: dict) -> str:
159     nb = nbformat.read(nb_path, as_version=4)
160     code_cells = [c for c in nb.cells if c.cell_type == "code"]
161     parts = [
162         f'<div class="nb-header">{nb_path.name} &nbsp;&middot;&nbsp;  {len(nb.cells)} cell &nbsp;&
163         &nbsp;&middot;&nbsp;  {len(code_cells)} code</div>'
164     ]
165
166     code_idx = 0
167     for cell in nb.cells:
168         if cell.cell_type == "markdown":
169             # Convert simple markdown headers
170             src = cell.source.strip()
171             lines = []
172             for l in src.split("\n"):
173                 if l.startswith("### "):
174                     lines.append(f'<h4>{l[4:]}</h4>')
175                 elif l.startswith("## "):
176                     lines.append(f'<h3>{l[3:]}</h3>')
177                 elif l.startswith("# "):
178                     lines.append(f'<h2>{l[2:]}</h2>')
179                 else:
180                     lines.append(f'<p>{l}</p>' if l.strip() else '')
181             parts.append(f'<div class="nb-md">{"".join(lines)}</div>')
182         elif cell.cell_type == "code":
183             execution_count = code_idx + 1
184             code_html = format_code_with_lines(cell.source)
185             parts.append(f'<div class="cell-head">In [{execution_count}]</div>')
186             parts.append(f'<pre class="code-in"><code>{code_html}</code></pre>')
187
188             # Check if we have output for this cell
189             cell_out = outputs.get(nb_id, {}).get(code_idx)
190             if cell_out:
191                 parts.append(format_out(cell_out))
192             code_idx += 1
193
194     return "\n".join(parts)
195
196
197 def build_full_appendix(outputs: dict) -> str:
198     parts = [
199         '<div class="page-break"></div>',
200         '<section class="appendix-section">',
201         '<h1 style="text-align:center; font-size:18pt; margin-bottom:30px;">PHẦN PHỤ LỤC</h1>',
202     ]
203
204     for anchor, letter, title, path in EXTRA_NOTEBOOKS:
205         parts.append(f'<div class="page-break"></div>')

```

```

205     parts.append(f'<h2 id="{anchor}">Phụ lục {letter}. {title}</h2>')
206     parts.append(
207         '<p class="small">Toàn bộ nội dung notebook, giữ nguyên thứ tự cell. '
208         'Khối <code>In [n]</code> là mã nguồn thực thi, khối <code>Out</code> là kết quả chạy
thật trên máy cục bộ. '
209         'Mọi số liệu trong phần thân báo cáo đều được tham chiếu đồng nhất từ các thí nghiệm
này.</p>'
210     )
211     nb_id = path.stem
212     parts.append(render_notebook(path, nb_id, outputs))
213
214     parts.append('<div class="page-break"></div>')
215     parts.append('<h2 id="pl-f">Phụ lục F. Mã nguồn dùng chung và kịch bản dựng báo cáo</h2>')
216     parts.append(
217         '<p class="small">Dưới đây là toàn bộ mã nguồn của gói <code>ml</code>, các kịch bản thự
c nghiệm '
218         'và trình sinh tài liệu tự động. Mã nguồn được viết tối ưu, vector hoá thuần NumPy kết hợ
p scikit-learn pipeline.</p>'
219     )
220
221     for rel, desc in SOURCE_FILES:
222         path = ROOT / rel
223         if path.exists():
224             code_str = path.read_text(encoding="utf-8")
225             n_lines = code_str.count("\n") + 1
226             parts.append(
227                 f'<div class="file-head"><b>{rel}</b> – {desc} <span>{n_lines} dòng</span></div>'
228             )
229             parts.append(f'<pre class="source-code"><code>{format_code_with_lines(code_str)}</cod
e></pre>')
230
231     parts.append('</section>')
232     return "\n".join(parts)
233
234
235 def main():
236     print("=== Đang xây dựng Master Report đầy đủ (Thân + Phụ lục A-F) ===")
237     content_file = BUILD / "content03.html"
238     if not content_file.exists():
239         subprocess.run([sys.executable, str(BUILD / "content03.py")], check=True)
240
241     html_body = content_file.read_text(encoding="utf-8")
242
243     # Inline figures as base64 data URIs
244     for fig_path in FIGS.glob("*.png"):
245         data = base64.b64encode(fig_path.read_bytes()).decode("ascii")
246         data_uri = f"data:image/png;base64,{data}"
247         html_body = html_body.replace(f'src="{fig_path.name}"', f'src="{data_uri}"')
248
249     outputs = get_local_outputs()
250     appendix_html = build_full_appendix(outputs)
251
252     full_html = f'<!doctype html>
253 <html lang="vi">
254 <head>
255 <meta charset="utf-8">
256 <title>Assignment 03: Deep Learning from Scratch with NumPy - Nguyễn Đại Dũng</title>
257 <style>
258 @page {{
259     size: A4;

```

```

260     margin: 20mm 15mm 20mm 15mm;
261     @bottom-right {{
262         content: counter(page);
263         font-family: 'Times New Roman', serif;
264         font-size: 9pt;
265     }}
266 }}
267 body {{
268     font-family: 'Times New Roman', Times, serif;
269     font-size: 10.5pt;
270     line-height: 1.45;
271     color: #111;
272     background-color: #fff;
273     margin: 0;
274     padding: 0;
275 }}
276 .container {{
277     max-width: 820px;
278     margin: 0 auto;
279     padding: 0;
280 }}
281 .page-break {{
282     page-break-before: always;
283 }}
284 .cover {{
285     text-align: center;
286     padding-top: 40px;
287     margin-bottom: 50px;
288     border-bottom: 2px solid #222;
289     padding-bottom: 40px;
290     page-break-after: always;
291 }}
292 .cover-inst {{
293     font-size: 13pt;
294     font-weight: bold;
295     text-transform: uppercase;
296     margin-bottom: 10px;
297 }}
298 .cover-kicker {{
299     font-style: italic;
300     color: #444;
301     margin-bottom: 20px;
302 }}
303 .cover-title {{
304     font-size: 21pt;
305     font-weight: bold;
306     margin: 25px 0;
307     line-height: 1.3;
308 }}
309 .cover-authors {{
310     margin: 25px 0;
311     font-size: 11.5pt;
312 }}
313 .cover-authors span {{
314     display: block;
315     margin: 5px 0;
316 }}
317 .cover-authors .nm {{
318     font-size: 14pt;
319     font-weight: bold;

```

```

320 }}
321 .cover-date {{
322     font-size: 11pt;
323     font-style: italic;
324     margin-bottom: 30px;
325 }}
326 .abstract {{
327     text-align: justify;
328     background: #fdfdfd;
329     border-left: 3px solid #333;
330     padding: 15px 20px;
331     margin-top: 30px;
332 }}
333 h2 {{
334     font-size: 13.5pt;
335     border-bottom: 1px solid #999;
336     padding-bottom: 4px;
337     margin-top: 30px;
338     page-break-after: avoid;
339 }}
340 h3 {{
341     font-size: 11.5pt;
342     margin-top: 20px;
343     page-break-after: avoid;
344 }}
345 table {{
346     width: 100%;
347     border-collapse: collapse;
348     margin: 15px 0;
349     font-size: 9.5pt;
350     page-break-inside: avoid;
351 }}
352 table, th, td {{
353     border: 1px solid #777;
354 }}
355 th, td {{
356     padding: 5px 8px;
357     text-align: left;
358 }}
359 th {{
360     background-color: #f0f0f0;
361     font-weight: bold;
362 }}
363 td.num, th.num {{
364     text-align: right;
365     font-variant-numeric: tabular-nums;
366 }}
367 td.mono {{
368     font-family: Consolas, 'Courier New', monospace;
369     font-size: 9pt;
370 }}
371 tr.hi {{
372     background-color: #fcf6db;
373     font-weight: bold;
374 }}
375 .formula {{
376     font-family: 'Cambria Math', 'Times New Roman', serif;
377     background: #f8f8f8;
378     border: 1px solid #ddd;
379     padding: 8px;

```

```

380     margin: 12px 0;
381     text-align: center;
382     font-style: italic;
383     font-size: 10.5pt;
384 }}
385 figure {{
386     text-align: center;
387     margin: 20px 0;
388     page-break-inside: avoid;
389 }}
390 figure img {{
391     max-width: 95%;
392     height: auto;
393 }}
394 figcaption {{
395     font-size: 8.5pt;
396     font-style: italic;
397     color: #333;
398     margin-top: 6px;
399 }}
400 .kpis {{
401     display: flex;
402     justify-content: space-around;
403     background: #fafafa;
404     border: 1px solid #ccc;
405     padding: 10px;
406     margin: 15px 0;
407 }}
408 .kpi {{
409     text-align: center;
410 }}
411 .kpi b {{
412     display: block;
413     font-size: 15pt;
414     color: #0b5394;
415 }}
416 .kpi span {{
417     font-size: 8pt;
418     color: #555;
419     text-transform: uppercase;
420 }}
421 .callout, .risk, .oim {{
422     border-left: 3.5px solid #2b6cb0;
423     background: #f4f8fb;
424     padding: 10px 14px;
425     margin: 15px 0;
426     font-size: 9.5pt;
427     text-align: justify;
428 }}
429 .risk {{
430     border-left-color: #c53030;
431     background: #fff5f5;
432 }}
433 .chain {{
434     display: flex;
435     justify-content: space-between;
436     margin: 15px 0;
437     gap: 8px;
438 }}
439 .chain .st {{

```

```

440     flex: 1;
441     background: #f7f7f7;
442     border: 1px solid #ccc;
443     padding: 6px 8px;
444     border-radius: 3px;
445     font-size: 8.5pt;
446 }}
447 .chain .st k {{
448     display: block;
449     color: #777;
450     font-size: 7.5pt;
451     font-weight: bold;
452 }}
453 .chain .st b {{
454     display: block;
455     color: #111;
456 }}
457 .chain .st i {{
458     display: block;
459     color: #555;
460     font-size: 7.5pt;
461 }}
462 .tcap {{
463     font-weight: bold;
464     font-size: 9.5pt;
465     margin-top: 12px;
466 }}
467 .tnote {{
468     font-size: 8pt;
469     color: #555;
470     font-style: italic;
471     margin-top: -8px;
472     margin-bottom: 12px;
473 }}
474
475 /* Appendix Styling */
476 .nb-header {{
477     background: #2d3748;
478     color: #fff;
479     padding: 6px 12px;
480     font-family: Consolas, monospace;
481     font-size: 9pt;
482     font-weight: bold;
483     margin-top: 20px;
484 }}
485 .cell-head {{
486     font-family: Consolas, monospace;
487     font-size: 8.5pt;
488     font-weight: bold;
489     color: #2b6cb0;
490     margin-top: 12px;
491     margin-bottom: 2px;
492 }}
493 .out-label {{
494     font-family: Consolas, monospace;
495     font-size: 8.5pt;
496     font-weight: bold;
497     color: #c53030;
498     margin-top: 4px;
499     margin-bottom: 2px;

```

```

500 }}
501 pre.code-in, pre.code-out, pre.source-code {{
502     background: #f8f9fa;
503     border: 1px solid #e2e8f0;
504     padding: 8px 10px;
505     margin: 5px 0 12px 0;
506     font-family: Consolas, 'Courier New', monospace;
507     font-size: 8.5pt;
508     line-height: 1.42;
509     white-space: pre-wrap;
510     word-break: break-all;
511 }}
512 pre.code-out {{
513     background: #fff;
514     border-left: 3px solid #718096;
515 }}
516 span.ln {{
517     display: inline-block;
518     width: 35px;
519     color: #a0aec0;
520     user-select: none;
521     text-align: right;
522     margin-right: 8px;
523 }}
524 .file-head {{
525     background: #edf2f7;
526     border: 1px solid #cbd5e0;
527     padding: 6px 12px;
528     font-family: Consolas, monospace;
529     font-size: 9pt;
530     font-weight: bold;
531     margin-top: 25px;
532     display: flex;
533     justify-content: space-between;
534 }}
535 .file-head span {{
536     font-weight: normal;
537     color: #4a5568;
538 }}
539 .small {{
540     font-size: 9pt;
541     color: #444;
542 }}
543 </style>
544 </head>
545 <body>
546 <div class="container">
547 {html_body}
548 {appendix_html}
549 </div>
550 </body>
551 </html>""
552
553 out_html = REPORTS / "Assignment03_Master.html"
554 out_html.write_text(full_html, encoding="utf-8")
555 print(f"Master HTML đã sinh tại: {out_html.relative_to(ROOT)} ({out_html.stat().st_size / 1024:.1f} KB)")
556
557 target_pdf = REPORTS / "Assignment03_NguyenDaiDung_Full.pdf"
558 master_root_pdf = ROOT / "A3_02_NguyenDaiDung_103_Master.pdf"

```

```

559
560     print("\nĐang gọi Chrome Headless để xuất Master PDF...")
561     chrome = r"C:\Program Files\Google\Chrome\Application\chrome.exe"
562     cmd = [
563         chrome,
564         "--headless=new",
565         "--disable-gpu",
566         "--no-pdf-header-footer",
567         f"--print-to-pdf={target_pdf}",
568         out_html.as_uri(),
569     ]
570     res = subprocess.run(cmd, capture_output=True, text=True)
571     if target_pdf.exists():
572         import pypdf
573         import shutil
574         reader = pypdf.PdfReader(str(target_pdf))
575         n_pages = len(reader.pages)
576         print(f"THÀNH CÔNG: Đã xuất Master PDF ({target_pdf.stat().st_size / 1024 / 1024:.2f} MB,
577 {n_pages} trang)")
578         # Copy to root master pdf
579         shutil.copy2(target_pdf, master_root_pdf)
580         print(f"Đã lưu bản sao Master tại: {master_root_pdf.name}")
581
582         # Try copying to OUT_PDF if not locked
583         try:
584             shutil.copy2(target_pdf, OUT_PDF)
585             print(f"Đã cập nhật đề thành công vào: {OUT_PDF.name}")
586         except PermissionError:
587             print(f"LƯU Ý: {OUT_PDF.name} đang được mở bởi trình xem PDF (Acrobat/Foxit/Edge).")
588             print(f"-> Bạn có thể xem ngay file Master tại: {master_root_pdf.name} ({n_pages} tra
589 ng đầy đủ)!")
590         else:
591             print("Lỗi khi xuất PDF:", res.stderr)
592
593     if __name__ == "__main__":
594         main()
595

```

run_all.py

– Kịch bản chạy tự động toàn bộ

87 dòng

```

1  """Script điều khiển chạy toàn bộ hoặc từng phần thí nghiệm của Assignment 03.
2
3  Cách dùng:
4      python run_all.py --all                # Chạy toàn bộ 5 phần
5      python run_all.py --step 4             # Chạy riêng Deep Learning tiểu đường (NumPy thuần)
6      python run_all.py --step 5             # Chạy riêng K-Nearest Neighbors (3 category, 4 task)
7      python run_all.py --step 6             # Chạy riêng AdaBoost (3 category, 4 task)
8      python run_all.py --step 7             # Chạy riêng Deep Learning hồi quy giá nhà (NumPy thu
9      python run_all.py --step 8             # Chạy riêng Deep Learning nhúng từ E-commerce (NumPy
10     python run_all.py --step 4 --skip-lr-study # Chạy step 4 bỏ qua quét GD toàn tập (rất nhanh)
11     """
12
13     from __future__ import annotations
14

```

```

15 import argparse
16 import subprocess
17 import sys
18 from pathlib import Path
19
20 # Ensure UTF-8 output on Windows
21 sys.stdout.reconfigure(encoding="utf-8")
22
23 ROOT = Path(__file__).resolve().parent
24 PYTHON = sys.executable
25
26 STEPS = {
27     4: ("scripts/train_04_deep_learning.py", "Deep Learning từ đầu với NumPy (Tiểu đường)'),
28     5: ("scripts/train_05_knn.py", "K-Nearest Neighbors (3 category, 4 task)'),
29     6: ("scripts/train_06_adaboost.py", "AdaBoost (3 category, 4 task)'),
30     7: ("scripts/train_07_deep_learning_house.py", "Deep Learning hồi quy giá nhà (NumPy thuần)'),
31     8: ("scripts/train_08_deep_learning_ecommerce.py", "Deep Learning nhúng từ E-commerce (NumPy thuần)'),
32 }
33
34
35 def run_step(step_num: int, extra_args: list[str] = None):
36     if step_num not in STEPS:
37         print(f"Bước không hợp lệ: {step_num}. Chọn từ {list(STEPS.keys())}")
38         return False
39
40     script_rel, title = STEPS[step_num]
41     script_path = ROOT / script_rel
42     cmd = [PYTHON, str(script_path)]
43     if extra_args:
44         cmd.extend(extra_args)
45
46     print("\n" + "=" * 75)
47     print(f"BẮT ĐẦU BƯỚC {step_num}: {title}")
48     print(f"Lệnh: {' '.join(cmd)}")
49     print("=" * 75 + "\n")
50
51     res = subprocess.run(cmd, cwd=str(ROOT))
52     if res.returncode != 0:
53         print(f"\n[LỖI] Bước {step_num} thất bại với mã lỗi {res.returncode}")
54         return False
55     print(f"\n[THÀNH CÔNG] Hoàn thành bước {step_num}: {title}")
56     return True
57
58
59 def main():
60     parser = argparse.ArgumentParser(description="Chạy thí nghiệm Assignment 03")
61     parser.add_argument("--all", action="store_true", help="Chạy toàn bộ từ bước 4 đến bước 8")
62     parser.add_argument("--step", type=int, choices=list(STEPS.keys()), help="Chọn bước cần chạy (4, 5, 6, 7, hoặc 8)")
63     parser.add_argument("--skip-lr-study", action="store_true", help="Áp dụng cho bước 4: bỏ qua bài test GD toàn tập 1000 epoch để chạy nhanh hơn")
64
65     args = parser.parse_args()
66
67     if not args.all and args.step is None:
68         parser.print_help()
69         print("\nGợi ý: Thử chạy bước 4 nhanh bằng lệnh:")
70         print("    python run_all.py --step 4 --skip-lr-study")

```

```

71         return
72
73     if args.all:
74         for step_num in sorted(STEPS.keys()):
75             extra = ["--skip-lr-study"] if step_num == 4 and args.skip_lr_study else []
76             ok = run_step(step_num, extra)
77             if not ok:
78                 print(f"Dừng quá trình chạy tại bước {step_num}")
79                 break
80     else:
81         extra = ["--skip-lr-study"] if args.step == 4 and args.skip_lr_study else []
82         run_step(args.step, extra)
83
84
85 if __name__ == "__main__":
86     main()
87

```